

Mobile Mapping

Eine Einführung mit Matlab

Hochschule München, Fakultät für Geoinformation
Prof. Dr. Thomas Abmayr

Stand: 18.09.2025 10:53:06

0	ÜBERSICHT ÜBER LEHRVERANSTALTUNGEN.....	4
0.1	MOBILE MAPPING (MASTER GEOMATIK).....	4
1	EINFÜHRUNG	7
1.1	GRUNDLEGENDE BEGRIFFE	7
1.2	MESSPRINZIPIEN	9
2	MATHEMATISCHE GRUNDLAGEN.....	11
2.1	DAS EUKLIDISCHE SKALARPRODUKT	11
2.2	DAS VEKTORPRODUKT.....	12
2.3	ROTATIONEN UND SPIEGELUNGEN	13
2.4	KOORDINATENSYSTEME	14
2.5	DARSTELLUNG VON ROTATIONEN	15
2.6	CARTESIAN MOTION	17
2.7	DIE SINGULÄRWERTZERLEGUNG UND ANWENDUNGEN	18
2.8	LÖSEN NICHTLINEARER GLEICHUNGSSYSTEME.....	24
3	KALIBRIERUNG VON SENSOREN.....	30
3.1	MODELLIERUNG UND KALIBRIERUNG VON KAMERAS	30
3.2	SENSORMOD. UND KALIBRIERUNG NACH ZHANG	32
3.3	MODELLIERUNG EINES STEREOKAMERASYSTEMS.....	37
4	STEREO- UND BEWEGTBILDVERARBEITUNG.....	39
4.1	EINFÜHRUNG.....	39
4.2	DIE STANDARDSTEREOGEOMETRIE	40
4.3	BILDREKTIFIZIERUNG	42
4.4	KORRESPONDENZANALYSE.....	45
4.5	DIE EPIPOLARGEOMETRIE.....	57
4.6	F-MATRIX BERECHNEN UND RANSAC.....	61
4.7	POSE BERECHNEN UND TRIANGULATION	65
4.8	BUNDLE ADJUSTMENT.....	70
4.9	MODIFIKATIONEN.....	74
5	LITERATUR	76
6	FAQS.....	77
6.1	MOBILE MAPPING.....	77
7	PRAKTIKA.....	87
7.1	MOBILE MAPPING (MATLAB)	87
7.2	PROJEKT: DENSE MATCHING (MATLAB)	106
7.3	MOBILE MAPPING (PYTHON/OPENCV)	116
7.4	PROJEKT: DENSE MATCHING (PYTHON/OPENCV).....	134
8	LÖSUNGSSKIZZEN PRAKTIKA.....	145
8.1	LÖSUNGSSKIZZE MOBILE MAPPING (MATLAB)	145
8.2	LÖSUNGSSKIZZE PROJEKT DENSE MATCHING (MATLAB).....	165
8.3	LÖSUNGSSKIZZE MOBILE MAPPING (PYTHON/OPENCV)	178
8.4	LÖSUNGSSKIZZE PRJ DENSE MATCH. (PYTHON/OPENCV).....	189

9	LÖSUNGSSKIZZE FAQs	190
9.1	MOBIEL MAPPING.....	190

0 Übersicht über Lehrveranstaltungen

0.1 MOBILE MAPPING (MASTER GEOMATIK)

0.1.1 ALLGEMEINES

Ziel der Lehrveranstaltung ist die prototyphafte Entwicklung von kamerabasierten Navigationsalgorithmen, zur Berechnung der zurückgelegten Trajektorie und zur Rekonstruktion der Umgebung. Der Fokus liegt auf bildbasierten Methoden unter Verwendung von

- Matlab unter Verwendung der Computer Vision Toolbox
- (Python unter Verwendung von openCV mit Einschränkungen)

0.1.1.1 FREIWILLIGER LEISTUNGSNACHWEIS

Im Rahmen des Praktikums kann ein freiwilliger Leistungsnachweis in einem Umfang von maximal **12.5 Punkten** – und damit von 2 ½ Notenstufen – wie folgt erzielt werden:

- Art der freiwilligen Übungsleistungen:
 - Bei 9 von 11 Lehrveranstaltungen:
 - Aktiver und vollständiger Versuch, die FAQ-Fragen zur nächsten Lehrveranstaltung zu beantworten. Abgabe vor der nächsten Lehrveranstaltung (**4 Punkte**).
 - Aktiver Versuch die Praktikumsaufgaben zu lösen (**4 Punkte**)
 - Abgabe der Ausarbeitungen (**4.5 Punkte**)
 - Ausgearbeitetes pdf inkl. kommentiertem Quellcode
 - Die Ausarbeitungen müssen zur Anerkennung hinreichend korrekt sein und Rückfragen beantwortet werden können

0.1.2 ZIEL DER LEHRVERANSTALTUNG UND FRAGESTELLUNGEN

Im Laufe der Lehrveranstaltung sollen **beispielsweise folgende Fragestellungen** beantwortet werden:

- Wie können unterschiedliche Aufnahmestandpunkte mathematisch beschrieben werden?
- Wie kann aus gegebenen (bzw. berechneten) Aufnahmestandpunkten eine Trajektorie berechnet werden?
- Wie kann eine Kamera mathematisch beschrieben werden?
- Wie kann eine Kamera kalibriert werden?

- Wie kann aus zwei Kamerabildern (deren relative Lage und Orientierung zueinander bekannt ist) Tiefeninformation extrahiert werden?
- Wie können zueinander korrespondierende Punkte in zwei Bildern beschrieben werden?
- Gegeben sei ein Punkt in dem einen Bild. Wo liegen mögliche korrespondierende Punkte in dem zweiten Bild?
- Wie kann über mehrere Bildaufnahmen die Genauigkeit gesteigert werden?

Demovideos

- Demovideo zum Praktikum → siehe Moodle
- <https://www.youtube.com/watch?v=qindx2LQ1r4>
- <https://www.youtube.com/watch?v=tP1GFapGalQ>

0.1.3 ÜBERSICHT ÜBER METHODEN UND BEGRIFFE

Im Folgenden werden für ein erstes Verständnis einige grundlegende Begriffe kurz umrissen (ohne an dieser Stelle einen Anspruch auf Exaktheit oder Vollständigkeit zu erheben):

Intrinsischen und extrinsische Kameraparameter: Intrinsisch: Beschreibt die innere Charakteristik einer Kamera (Brennweite, Verzeichnung ..). Extrinsisch: Lage und Orientierung relativ zu einem Weltkoordinatensystem

Standardstereogeometrie: Idealisierte Kameraanordnung: Bildebenen Parallel in derselben Ebene ausgerichtet

Levenberg Marquardt und Bündelblockausgleichung: Algorithmus zum Lösen nichtlinearer Optimierungsprobleme; sehr häufiger Einsatz in der Computer Vision; LM: Algorithmus zur Lösung nichtlinearer Ausgleichsprobleme Bündelblockausgleichung: Vereinfacht: Berechnung der Lage und Orientierung mehrerer Standpunkte einer Kamera (auch bspw. eines Lasers) zueinander. Dabei wird jeder Standpunkt zu jedem ausgeglichen und nicht nur jeder Standpunkt zu einem Masterstandpunkt berechnet

Kamerakalibrierung nach Zhang: (a) Mathematische Beschreibung einer Kamera, um 3D Punkte auf die Bildebene eines Bildes zu projizieren (b) sowie die dazugehörige Methode zur Berechnung der dazu benötigten Parameter, um insbesondere die inneren Kameraparameter zu berechnen

Disparitätsbild ("disparity"): Gegeben seien zwei Kameras in einer spez. Anordnung, der sog. Standardstereogeometrie. Aus der Differenz der horizontalen Pixelkoordinaten korrespondierender Pixel (=Disparität) lässt sich die 3D Koordinate des Punktes berechnen

SIFT: Algorithmus zum Finden korrespondierender Punkte in zwei Bildern

Epipolargeometrie berechnen: Berechnet relative Lage und Orientierung zweier Bilder zueinander (falls inneren Kameraparameter bekannt)

Dense 3D Modell: Für jedes Pixel wird ein 3D Punkt rekonstruiert

Sparse 3D Modell: Nur für explizite / markante Punkte wird ein 3D Punkt rekonstruiert

Bildrektifizierung ("rectify images"): Paralleles Ausrichten von 2 Bildern“ um daraus 3D Punkte rekonstruieren zu können. Dieser Ablauf heißt Überführung in Standardstereogeometrie.

Triangulation ("triangulate"): Berechnung eines 3D Punktes als Schnitt zweier Geraden

Singulärwertzerlegung: Verfahren aus der linearen Algebra mit Anwendungen in unterschiedlichsten Bereichen in der Computer Vision (z.B. Lösen von linearen Gleichungssystemen, der Berechnung der numerischen Stabilität eines Gleichungssystems oder dem Lösen von Eigenwertproblemen).

RANSAC: Robustes, effizientes Verfahren zur Suche von Ausreißern

Korrespondenzsuche: Suche nach korrespondierenden Punkten

ply und pld Datenformat: (a) ply Polygon File Format oder Stanford Triangle Format zum Speichern von 3D Punktwolken; Wahlweise binär oder ascii; b) pcd: Alternatives Format mit dem Vorteil, auch organized datasets (wie z.B. kinect) abspeichern zu können und dadurch effizienter

Pose und Homogene Matrix: (a) Pose: Begriff aus Robotik: Position und Orientierung (hier eines Scans) (b) Hom. Matrix: Matrix, die Position und Orient. beinhaltet um „damit rechnen zu können“

1 Einführung

Im Folgenden werden grundlegende Methoden und Begriffe eingeführt.

1.1 GRUNDLEGENDE BEGRIFFE

1.1.1 BEGRIFF (LOKALISATION UND WELTKOORDINATENSYSTEM)

Die Fähigkeit eines (mobilen) Roboters seinen Ort relativ zu einem Bezugskoordinatensystem zu bestimmen heißt **Lokalisierung**. Dieses Bezugskoordinatensystem wird im Folgenden oft **Weltkoordinatensystem** genannt.

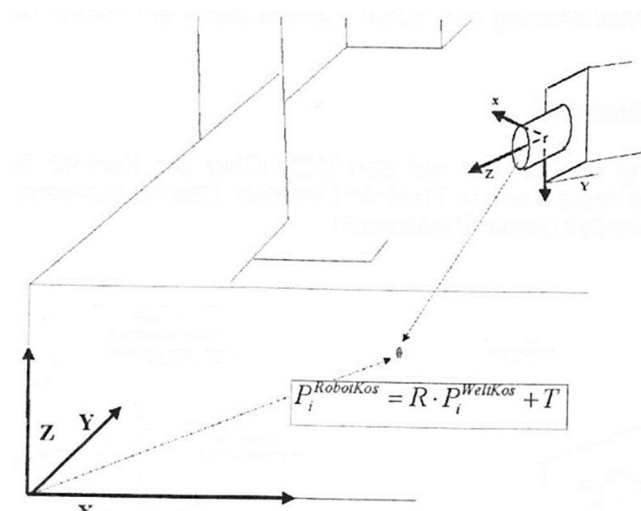


Abbildung 1 Schematische Darstellung zur Lokalisierung: Der Ort eines mobilen Roboters soll in einem Bezugskoordinatensystem bestimmt werden

1.1.2 BEGRIFF (3D REKONSTRUKTION)

Die Berechnung eines digitalen 3D Modells (z.B. eine Punktwolke oder ein Mesh) aus mit Sensoren aufgenommenen Daten der Umgebung heißt im Folgenden **3D Rekonstruktion**.

Wurden die Sensorinformationen aus unterschiedlichen Standpunkten aufgenommen, so muss die Lage und Orientierung der Standpunkte zueinander bekannt sein.

1.1.3 BEGRIFF (MOBILE MAPPING SYSTEM)

Mobile Mapping Systeme (MMS) sind mit folgenden Eigenschaften charakterisiert:

- **Mobile Plattform** (z.B. Auto, Bahn, Flugzeug)
- **Multisensorieller Aufbau** zur Erfassung der Umgebung in zwei bzw. dreidimensionaler Form

- Die **Auswertung** der Daten geschieht zum Teil offline (Genauigkeitssteigerung!)

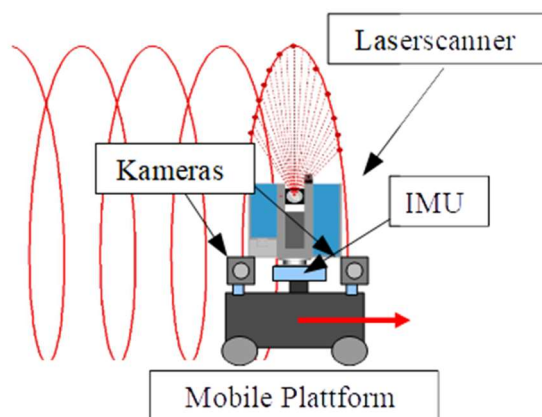


Abbildung 2 Schematischer Aufbau eines MMS bestehend aus einer mobilen Plattform und einem Multisensorsystem

1.1.4 BEISPIEL (MOBILE MAPPING SYSTEME)

1.1.4.1 DAS STRASSENERFASSUNGSSYSTEM MOSES

- Firma:** 3D Mapping Solutions
- Aufgabe:** Erstellung digitaler Bestandspläne im Strassenbau
- Eigenschaften**
 - Erfassungsgeschwindigkeit: 100 km/h
 - Positionsbestimmung:
 - DGPS (über SaPOS und eigene Referenzstationen)
 - Hochgenaue IMU (Applanix, >0.005o && 0.02o)
 - Odometer
 - Erfassung der Objektinformation:
 - Semantische Information über Videokameras
 - 2 CCD Kameras (Stereobildverarbeitung)
 - Laserscanner (Z+F Imager 5010)

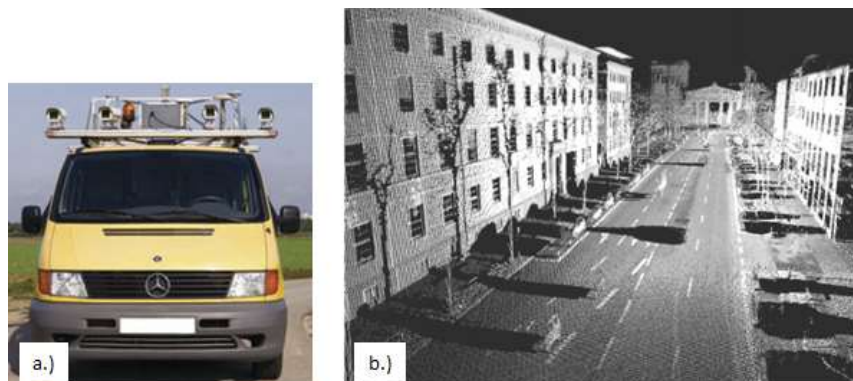


Abbildung 3 (a) Strassenerfassungssystem Moses (b) 3D Punktwolke der Brienner Straße[Quelle: 3D Mapping Solutions], 3D Punktwolke der Brienner Straße[Quelle: 3D Mapping Solutions, 2012]

1.1.4.2 GOOGLE STREET VIEW

Firma: Google

- **Aufgabe:** Darstellung der Strassenperspektive als 360° Bilder
- Positionsbestimmung mit DGPS, MU und Odometer
- Erfassung der Objektinformation mit Bildern aus Kamerasystem

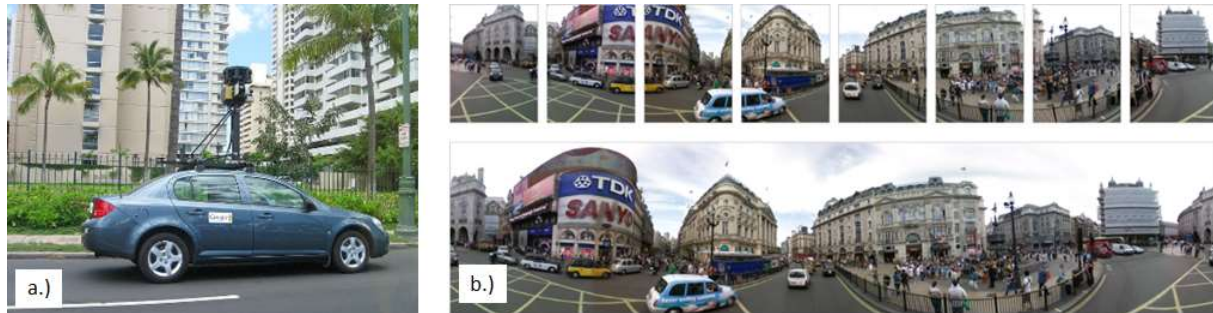


Abbildung 4 (a) MMS System für Google Streetview **(b)** Datenaufnahme aus 8 Bildern und Bildstichung zu einem Panoramabild

1.1.4.3 WEITERE BEISPIELE

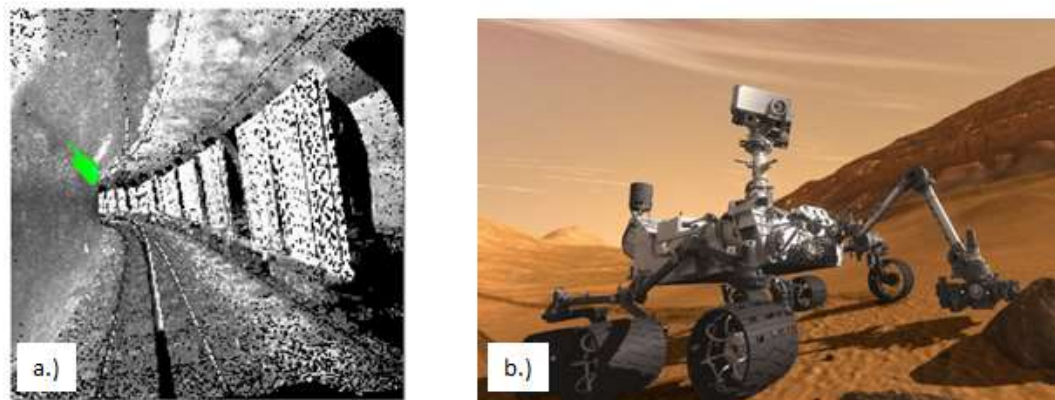


Abbildung 5 (a) Kinematisches Laserscannen (Zoller + Fröhlich GmbH) und **(b)** Symbolbild des Marsrovers Curiosity [Keystone]

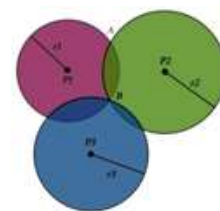
1.2 MESSPRINZIPIEN

1.2.1 EINFÜHRENDE BEGRIFFE

1.2.1.1 BEGRIFF (TRILATERATION)

Berechnung der Lokalisation aus **Entfernungsmessung**.

- Hierzu sind 3 Sender notwendig.
- Mathematisch bedeutet dies den Schnitt 3er Kugeln zu berechnen
- Dies resultiert in zwei Lösungen.



- Ein Plausibilitätstest ergibt dann die richtige Lösung). Beispiele sind UWB und GPS.

1.2.1.2 BEGRIFF (TRIANGULATION)

Berechnung der Lokalisation aus **Schnitt 2er Geraden**.

- Beispiele sind Kameras, Theodoliten, etc.

2 Mathematische Grundlagen

2.1 DAS EUKLIDISCHE SKALARPRODUKT

2.1.1 BEGRIFF (EUKLIDISCHES SKALARPRODUKT)

Gegeben seien $u := (u_0, \dots, u_n)$ und $v := (v_0, \dots, v_n)$ Dann definiert

$$\langle u, v \rangle := (u_0 v_0 + \dots + u_n v_n)$$

ein Skalarprodukt. Es heißt **euklidisches Skalarprodukte**.

Des Weiteren definiert

$$\|v\| := \sqrt{\langle v, v \rangle}$$

eine **Norm**. Es gilt (mit α Winkel zwischen u, v):

$$\langle u, v \rangle = \|u\| \|v\| \cos \alpha \text{ mit } \alpha \in [0, \pi]$$

2.1.2 BEGRIFF (ORTHOGONAL)

Das Skalarprodukt zweier von Null verschiedener Vektoren ist genau dann Null, wenn sie aufeinander senkrecht stehen. Also wenn

$$\langle u, v \rangle = 0$$

Dann steht u senkrecht auf v . Solche Vektoren heißen **orthogonal**.

Beweis:

Sei $u \perp v$ dann gilt:

$$\|u - v\|^2 = \|u\|^2 + \|v\|^2 \quad (1)$$

Also

$$\sum_{i=0}^{N-1} u_i^2 + \sum_{i=0}^{N-1} v_i^2 = \sum_{i=0}^{N-1} u_i^2 - \sum_{i=0}^{N-1} (2u_i v_i) + \sum_{i=0}^{N-1} v_i^2 \quad (2)$$

Also

$$0 = - \sum_{i=0}^{N-1} (2u_i v_i) \quad (3)$$

Also

$$\langle u, v \rangle = 0$$

2.2 DAS VEKTORPRODUKT

2.2.1 BEGRIFF (VEKTORPRODUKT)

Gegeben seien $u := (u_0, u_1, u_2)$ und $v := (v_0, v_1, v_2)$. Dann definiert

$$u \times v := \|u\| \|v\| \sin(\theta) n \quad (4)$$

mit

$$n \in \mathbb{R}^3; \|n\| = 1; n \perp u \wedge n \perp v \quad (5)$$

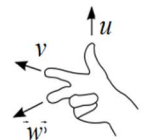
eine Abbildung die **Vektorprodukt** heißt. Hierbei bezeichnet θ den Winkel zwischen u und v , wobei

$$0^\circ \leq \theta \leq 180^\circ \quad (6)$$

2.2.2 GEOMETRISCHE INTERPRETATION

Geometrisch lässt sich das Vektorprodukt als eine Abbildung interpretieren, die einen neuen Vektor $w \in \mathbb{R}^3$ berechnet, m.d.E.

- w orthogonal zu u und v
- u, v, w bilden ein Rechtssystem



2.2.3 EIGENSCHAFTEN (VEKTORPRODUKT)

Es gilt:

$$\|u \times v\| = \|u\| \|v\| \sin \theta$$

$$\|u \times u\| = 0 \quad (7)$$

$$\|u \times v\| = 1 \Leftrightarrow u \perp v; \|u\| = 1; \|v\| = 1$$

$$u \times v = -v \times u \quad (\text{antikommutativ})$$

2.2.4 MATRIX/ VEKTORFORM

Ein Vektorprodukt lässt sich i.A. in der folgenden Form darstellen

$$u \times v = \begin{pmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{pmatrix} \Rightarrow u \times v = \underbrace{\begin{pmatrix} 0 & -u_3 & u_2 \\ u_3 & 0 & -u_1 \\ -u_2 & u_1 & 0 \end{pmatrix}}_S \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \quad (8)$$

Und heißt **Matrix-/ Vektorform des Vektorproduktes**. Außerdem ist S antisymmetrisch, d.h.

$$S^t = -S \quad (9)$$

2.3 ROTATIONEN UND SPIEGELUNGEN

2.3.1 BEGRIFF (SPEZIELLE ORTHOGONALE GRUPPE)

Gegeben sei die Menge

$$SO(3) := \{ R \in \mathbb{R}^{3 \times 3} \mid R^{-1} = R^t \wedge \det(R) = 1 \} \quad (10)$$

Die Menge umfasst alle Rotationsmatrizen und bildet mit der Matrizenmultiplikation eine Gruppe, die oft spezielle orthogonale Gruppe genannt wird.

Anmerkung: Die Gruppe umfasst also alle 3x3 Drehmatrizen. Gilt $\det(R) = -1$, so handelt es sich um eine Spiegelung.

2.3.2 EIGENSCHAFTEN (SPEZIELLE ORTHOGONALE GRUPPE)

$$R R^t = I \quad \text{und} \quad \sum_{i=1}^3 r_{ij} r_{kj} = \begin{cases} 0 & \text{if } i = k \\ 1 & \text{if } i \neq k \end{cases} \quad (11)$$

Folgerung: Jede Rotation kann durch drei Parameter beschrieben werden

2.3.3 BEGRIFF (BEWEGUNG DES $\mathbb{R}^3$)

Die Abbildung

$$b_{R,T}(x) = Rx + T \quad (12)$$

heißt Bewegung in $\mathbb{R}^3$. Die Menge

$$B := \{ b_{R,T} \mid R \in SO(3), T \in \mathbb{R}^3 \} \quad (13)$$

der Bewegungen bildet mit der Komposition $\circ$ eine Gruppe (Abgeschlossenheit, Assoziativität, Neutrales Element, Inverses Element).

2.3.4 BEGRIFF (HOMOGENE MATRIX)

Jede Rotation und Translation lässt sich als sog. **Homogene Matrix** in der folgenden Form darstellen

$$M = \begin{pmatrix} R & T \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (14)$$

Anm.: Der Vorteil von Homogene Matrizen ist u.a., dass Hinereinanderausführungen leichter berechnet werden können und eine gegebene Transformation leichter invertiert werden kann.

2.3.5 BEGRIFF (EXTERNE KAMERAPARAMETER)

Die für die Transformation

$$X^{(c)} = RX^{(w)} + T \quad (15)$$

von Welt- nach Kamerakoordinaten benötigte Rotation **R** und Translation **T** heißen **externe Kameraparameter**.

2.3.6 BEGRIFF (ORIENTIERUNG, POSITION UND POSE)

Die für die Transformation

$$X^{(w)} = RX^{(c)} + T \quad (16)$$

von Kamerakoordinaten (= lokalen Koordinaten) nach Weltkoordinaten benötigte Rotation **R** heißt **Orientierung** und die Translation **T** heißt **Position** (=Location) **L**. Das Tupel (R, T) wird mit **pose** bezeichnet. Insgesamt gilt für den Zusammenhang zwischen externen Kameraparametern und pose

$$Ori = R' \quad \text{und} \quad L = -R'T \quad (17)$$

2.4 KOORDINATENSYSTEME

Gegeben

$$M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{mit} \quad \begin{matrix} \vec{t} = (t_1 & t_2 & t_3) \\ \vec{r}_i = (r_{i1} & r_{i2} & r_{i3}) \end{matrix} \quad (18)$$

dann beschreibt

$$K = (\vec{t} \quad \vec{r}_1 \quad \vec{r}_2 \quad \vec{r}_3) \quad (19)$$

Ein affines, orthogonales, orientierungstreues Koordinatensystem.

2.4.1.1 BEWEISSKIZZE:

- "Alter Ursprung" (also (0, 0, 0,1)) mit Matrix transformieren ergibt "neuen Ursprung" (=4. Spalte der Matrix M)
- "Alte" x Achse (also (1,0,0)) mit Matrix transformieren ergibt "neuen x Achse" (=1. Spalte der Matrix M)
- "Alte" y Achse (also (0,1,0)) mit Matrix transformieren ergibt "neuen y Achse" (=2. Spalte der Matrix M)
- "Alte" z Achse (also (0,0,1)) mit Matrix transformieren ergibt "neuen z Achse" (=3. Spalte der Matrix M)

Anmerkung: Orientierungstreu heißt, dass das Koordinatensystem mit der rechten Handregel aufgespannt werden kann.

2.5 DARSTELLUNG VON ROTATIONEN

2.5.1 ROTATIONEN ("ROLL, PITCH, YAW" DARSTELLUNG)

Jede Rotation lässt sich darstellen durch

$$R_{rpy} = R_y R_p R_r \quad (20)$$

Mit

$$R_r = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos r & -\sin r \\ 0 & \sin r & \cos r \end{pmatrix} R_p = \begin{pmatrix} \cos p & 0 & \sin p \\ 0 & 1 & 0 \\ -\sin p & 0 & \cos p \end{pmatrix} R_y = \begin{pmatrix} \cos y & -\sin y & 0 \\ \sin y & \cos y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (21)$$

Und insgesamt

$$R_{rpy}(r, p, y) = \begin{pmatrix} \cos p \cos y & \cos y \sin r \sin p - \cos r \sin y & \sin r \sin y + \cos r \cos y \sin p \\ \cos p \sin y & \sin r \sin p \sin y + \cos r \cos y & \cos r \sin p \sin y - \cos y \sin r \\ -\sin p & \cos p \sin r & \cos r \cos p \end{pmatrix} \quad (22)$$

2.5.2 RÜCKRECHNUNG (RPY DARSTELLUNG)

Gegeben sei die Rotationsmatrix R durch

$$R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix} \quad (23)$$

Gesucht seien die Winkel r , p , y .

Lösung: Aus Koeffizientenvergleich folgt:

$$y = \text{atan2}(\sin y, \cos y) = \text{atan2}(\sin y \cos p, \cos y \cos p) = \text{atan2}(r_{21}, r_{11}) \quad (24)$$

$$p = \text{atan2}(\sin p, \cos p) = \text{atan2}\left(+\sin p, \cos p \underbrace{(\cos y \cos + \sin y \sin y)}_{=1}\right) \quad (25)$$

$$= \text{atan2}(-(-\sin p), \cos y \cos p \cos y + \sin y \cos p \sin y) = \text{atan2}(-r_{31}, r_{11} \cos y + r_{21} \sin y)$$

$$r = \text{atan2}(\sin r, \cos r) = \text{atan2}\left(\frac{\cos p \sin r}{\cos p}, \frac{\cos r \cos p}{\cos p}\right) = \text{atan2}\left(\frac{r_{32}}{\cos p}, \frac{r_{33}}{\cos p}\right) \quad (26)$$

2.5.3 ROTATIONEN (AXIS/ANGLE DARSTELLUNG)

Jede Rotation lässt sich darstellen durch

$$R = I \cos(\Theta) + (1 - \cos(\Theta)) \begin{pmatrix} n_1 n_1 & n_1 n_2 & n_1 n_3 \\ n_2 n_1 & n_2 n_2 & n_2 n_3 \\ n_3 n_1 & n_3 n_2 & n_3 n_3 \end{pmatrix} + \sin(\Theta) \begin{pmatrix} 0 & -n_3 & n_2 \\ n_3 & 0 & -n_1 \\ -n_2 & n_1 & 0 \end{pmatrix} \quad (27)$$

Mit

$$\langle n, n \rangle = \sqrt{n_1^2 + n_2^2 + n_3^2} = 1$$

$$Rn = n \quad (28)$$

$1, \cos \Theta + i \sin \Theta, \cos \Theta - i \sin \Theta$ sind Eigenwerte

2.5.4 RÜCKRECHNUNG (AXIS ANGLE DARSTELLUNG)

Die Rückrechnung erfolgt aus Berechnung der Eigenvektoren und dem 2. und 3. Eigenwert (siehe oben).

2.6 CARTESIAN MOTION

2.6.1 EINFÜHRUNG

Folgendes Szenario: Die **Steuerbefehle** an Ihren Roboter erfolgen in einer Frequenz von **0.5 Hz** (also $dt=2$ [sek]). Es liegen Ihnen also alle 2 Sekunden neue **Odometriedaten** des Roboters vor, aus denen Sie jetzt mit Koppelnavigation eine Pose berechnen können. Für spätere **Filterberechnungen** wollen Sie jetzt aber mit einer Frequenz von **20 Hz** eine Pose vorliegen haben (und damit $dt=0.05$ [sek]).

2.6.2 INTERPOLATION VON ORIENTIERUNGEN

Gegeben seien zwei Rotationsmatrizen R_0 und R_N die zwei Orientierungen zu Beginn und Ende eines Vorgangs angegeben.

Gesucht seien Rotationsmatrizen R_1 bis R_{N-1} die diese Rotationen "glatt" ineinander interpolieren

Lösung: (siehe auch Corke) Verwende die RPY Darstellung für Rotationen. Bezeichne hierzu die Winkel der i-ten Rotation mit

$$(r^{(i)}, p^{(i)}, y^{(i)}). \quad (29)$$

Berechne jetzt die Winkel der gesuchten Rotationen, durch komponentenweises lineares interpolieren

$$\begin{aligned} r^{(i)}(s) &:= r^{(0)} + s(r^{(N)} - r^{(0)}) \\ p^{(i)}(s) &:= p^{(0)} + s(p^{(N)} - p^{(0)}) \\ y^{(i)}(s) &:= y^{(0)} + s(y^{(N)} - y^{(0)}) \end{aligned} \quad \text{mit} \quad s := \frac{k}{N+1}; \quad k \in \{1, \dots, N-1\} \quad (30)$$

Also

$$\begin{aligned} r^{(i)}(s) &:= (1-s)r^{(0)} + sr^{(N)} \\ p^{(i)}(s) &:= (1-s)p^{(0)} + sp^{(N)} \\ y^{(i)}(s) &:= (1-s)y^{(0)} + sy^{(N)} \end{aligned} \quad (31)$$

Anmerkung: Ein direktes Interpolieren der 3x3 Matrixeinträge würde den „Raum der Drehungen verlassen“.

2.6.3 INTERPOLATION VON KOORDINATENSYSTEMEN

Gegeben seien zwei homogene Matrizen M_0 und M_N die die Lage und Orientierung zweier affiner, rechtshändiger und orientierungstreuer Koor.-systeme zu Beginn und Ende eines Vorgangs angeben.

Gesucht seien Matrizen M_1 bis M_{N-1} die diese Koordinatensysteme glatt ineinander interpolieren

Lösung: Verwende (*) um die Orientierungen zu interpolieren und interpoliere die Ursprünge beider Koordinatensysteme ebenfalls komponentenweise

$$\begin{aligned} T_x^{(i)}(s) &:= (1-s)T_x^{(0)} + sT_x^{(N)} \\ T_y^{(i)}(s) &:= (1-s)T_y^{(0)} + sT_y^{(N)} \\ T_z^{(i)}(s) &:= (1-s)T_z^{(0)} + sT_z^{(N)} \end{aligned} \quad \text{mit} \quad s := \frac{k}{N+1}; \quad k \in \{1, \dots, N-1\} \quad (32)$$

2.7 DIE SINGULÄRWERTZERLEGUNG UND ANWENDUNGEN

Ziel: Folgende Aufgaben können (zumindest teilweise) mit der Singulärwertzerlegung gelöst werden

- Lösen von **überbestimmten, linearen und inhomogenen Gleichungssystemen** mit Anwendungen bspw. in der Kamerakalibrierung, Registrierung 3D/3D und Polynomapproximation/Regression
- Lösen von **linearen homogenen Gleichungssystemen** mit Anwendungen bspw. in der Kamerakalibrierung, Bildbasierten 3D Rekonstruktion oder Ebenen-/Geraden-/Zylinderpassung,
- **Test auf Singularität** (numerische Stabilität) einer Matrix
- **Inversenberechnung** / Pseudoinverse einer Matrix
- **Hauptachse**

Im Folgenden wird in die Singulärwertzerlegung eingeführt, bevor danach auf Eigenschaften und Anwendungen eingegangen wird (siehe bspw. [1]):

2.7.1 DIE SINGULÄRWERTZERLEGUNG

2.7.1.1 BEGRIFF (EIGENWERT UND EIGENVEKTOR)

Gegeben seien eine Matrix A und ein Vektor e , so dass $Ae = \lambda e$ für ein $\lambda \in \mathbb{R}$. Dann heißt e ein **Eigenvektor** zum **Eigenwert** λ der Matrix A .

2.7.1.2 BEGRIFF (SINGULÄRWERTZERLEGUNG)

Jede $n \times m$ Matrix A lässt sich schreiben als:

$$A = \underbrace{U}_{m \times m} \underbrace{D}_{m \times n} \underbrace{V^t}_{n \times n} \quad (33)$$

mit

$$A = \underbrace{\begin{pmatrix} u_{0,0} & \cdots & u_{0,m-1} \\ \vdots & & \vdots \\ u_{m-1,0} & \cdots & u_{m-1,m-1} \end{pmatrix}}_{\substack{\text{Spalten zueinander} \\ \text{orthogonale Einheitsvektoren}}} \underbrace{\begin{pmatrix} \sigma_0 & \cdots & 0 \\ \vdots & & \vdots \\ 0 & \cdots & \sigma_{n-1} \end{pmatrix}}_{\substack{\sigma_0 \geq \cdots \geq \sigma_{n-1} \geq 0 \\ \text{hei\ss en Singulärwerte}}} \underbrace{\begin{pmatrix} v_{0,0} & \cdots & v_{0,n-1} \\ \vdots & & \vdots \\ v_{n-1,0} & \cdots & v_{n-1,n-1} \end{pmatrix}^t}_{\substack{\text{Spalten zueinander} \\ \text{orthogonale Einheitsvektoren}}} \quad (34)$$

Die Zerlegung heit **Singulärwertzerlegung** (engl. Singular Value Dekomposition, kurz SVD).

2.7.1.3 GEOMETRISCHE INTERPRETATION

Abbildung 6 zeigt die geometrische Interpretation der Singulärwertzerlegung.

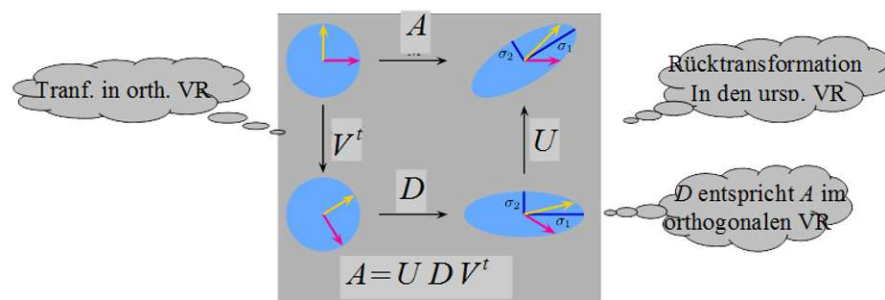


Abbildung 6: Quelle: Wikipedia. Die Abbildung zeigt die geometrische Interpretation der Singulärwertzerlegung: V^t bildet das ursprüngliche Koordinatensystem in ein neues Koordinatensystem K^{new} ab; D beinhaltet dieselbe Abbildung wie A , jedoch bzgl. des neuen Koordinatensystems; U bildet K^{new} wieder in das ursprüngliche Koordinatensystem ab

2.7.2 EIGENSCHAFTEN (SINGULÄRWERTZERLEGUNG)

Es gilt

$$AA^t u_i = \sigma_i^2 u_i \quad \text{und} \quad A^t A v_i = \sigma_i^2 v_i \quad (35)$$

wobei

- u_i ist Eigenvektor von AA^t und σ_i^2 der dazugehörige Eigenwert
- v_i ist Eigenvektor von $A^t A$ und σ_i^2 der dazugehörige Eigenwert

2.7.2.1 FOLGERUNG

- Die quadrierten Singulärwerte ungleich Null von A sind Eigenwerte von $A^t A$ bzw. AA^t
- Wenn $A = UDV^t$, dann sind die Spaltenvektoren von U bzw. V gleich den Eigenvektoren von AA^t bzw. $A^t A$.

2.7.3 ANWENDUNG (PSEUDOINVERSE UND TEST AUF SINGULÄR)

2.7.3.1 BEGRIFF (SINGULÄR)

Ist mindestens ein Singulärwerte σ_i^2 gleich Null, so ist eine Matrix nicht invertierbar und heißt **singulär**.

2.7.3.2 BEGRIFF (PSEUDOINVERSEN)

Nicht jede Matrix kann bekanntlich invertiert werden. Setze

$$A^+ = VD_0^{-1}U^t \quad \text{mit} \quad D_0^{-1} = \begin{pmatrix} \frac{1}{\sigma_0} & .. & 0 \\ : & \frac{1}{\sigma_i} & : \\ & & 0 \\ 0 & .. & : \end{pmatrix} \quad (36)$$

Dann heißt die so berechnete Matrix **Pseudoinverse** A^+ . Sie bildet die auf singuläre Matrizen verallgemeinerte Inverse einer Matrix.

Hinweise:

Die Herleitung erfolgt durch

$$A = UDV^t \Rightarrow A^+ = (UDV^t)^+ \Rightarrow A^+ = (V^t)^{-1} D_0^{-1} U^{-1} = VD_0^{-1}U^t \quad (37)$$

- U und V sind orthogonale Matrizen und können invertiert werden

- Für D werden nur die Elemente invertieren, die ungleich Null sind. Sind alle Diagonalelemente ungleich Null, so sind Pseudoinverse und Inverse identisch

2.7.3.3 BEGRIFF (SCHLECHT KONDITIONIERT)

Ist der Unterschied zwischen größtem Singulärwert und kleinstem Singulärwert sehr groß, also

$$\frac{\sigma_{n-1}}{\sigma_0} \approx 0$$

so ist eine Matrix „numerisch nicht gut invertierbar“ und heißt **schlecht konditioniert**.

Anmerkung: Ist eine Matrix schlecht konditioniert, so kann das Gleichungssystem nicht stabil gelöst werden!

2.7.4 ANWENDUNG (ÜBERBESTIMMTE LIN. INHOM. GLEICHUNGSS.)

2.7.4.1 PROBLEMSTELLUNG

Gegeben sei ein lineares Gleichungssystem mit mehr Gleichungen als Unbekannten

$$\begin{aligned} a_{00}x_0 + \dots + a_{0n-1}x_{n-1} &= b_0 \\ &\dots \\ a_{m-10}x_0 + \dots + a_{m-1n-1}x_{n-1} &= b_{m-1} \end{aligned} \quad (38)$$

(**Anm.:** x_i unbekannt, a_{ij} bekannt und „voneinander unabhängig“, b_i bekannt und ungleich Null)

Oder in Matrix Vektor Darstellung:

$$Ax = b \quad \text{wobei} \quad A = \begin{pmatrix} a_{0,0} & \dots & a_{0,n-1} \\ \vdots & \ddots & \vdots \\ a_{m-1,0} & \dots & a_{m-1,n-1} \end{pmatrix}; \quad x = \begin{pmatrix} x_0 \\ \vdots \\ x_{n-1} \end{pmatrix}; \quad b = \begin{pmatrix} b_0 \\ \vdots \\ b_{m-1} \end{pmatrix} \quad (39)$$

2.7.4.2 LÖSUNG

Zur **Lösung** wird der quadratische Fehler minimiert

$$\begin{aligned} r_0 &= (a_{00}x_0 + \dots + a_{0n-1}x_{n-1} - b_0)^2 \\ &\dots \\ r_{m-1} &= (a_{m-10}x_0 + \dots + a_{m-1n-1}x_{n-1} - b_{m-1})^2 \end{aligned} \quad (40)$$

Also

$$r(x) = (Ax - b)^t (Ax - b) \quad (41)$$

oder

$$\underbrace{\|Ax - b\|^2}_{\substack{r(x) \\ \text{Residuum}}} \rightarrow \min \quad (42)$$

r ist eine **quadratische Funktion** in Abhängigkeit von x, deren Nullstelle nun durch „Ableiten und Null setzen“ berechnet werden können. Ausmultiplizieren von r ergibt folglich

$$r(x) = (Ax - b)^t (Ax - b) = (b - Ax)^t (b - Ax) = x^t A^t Ax - 2x^t A^t b + b^t b \quad (43)$$

und ableiten ergibt

$$r'(x) = 2A^t Ax - 2A^t b \quad (44)$$

und durch Null setzen folgt

$$\begin{aligned} r'(x_0) = 0 &\Rightarrow 2A^t Ax_0 - 2A^t b = 0 \Rightarrow 2A^t Ax_0 = 2A^t b \Rightarrow \\ x_0 &= (A^t A)^+ A^t b \Rightarrow x_0 = (VD_0^{-1}U^t)A^t b \end{aligned} \quad (45)$$

2.7.4.3 BEGRIFF (NORMALENGLEICHUNGEN)

Die oben dargestellten Gleichungen $A^t A x_0 = A^t b$ heißen **Normalengleichungen**.

2.7.5 ANWENDUNG (HOMOGENEN LINEAREN GLEICHUNGSSYSTEME)

2.7.5.1 PROBLEMSTELLUNG UND TRIVIALE NULLLÖSUNG

Gesucht ist die Lösung für ein **Lineares homogenes** Gleichungssystem mit mehr Gleichungen als Unbekannten

$$\left. \begin{aligned} a_{00}x_0 + \dots + a_{0n-1}x_{n-1} &= 0 \\ \dots & \\ a_{m-10}x_0 + \dots + a_{m-1n-1}x_{n-1} &= 0 \end{aligned} \right\} \text{ mit } \begin{aligned} m &\geq n-1 \\ \text{rank}(A) &= n-1 \end{aligned} \quad (46)$$

(Anm.: x_i unbekannt, a_{ij} bekannt (und „voneinander unabhängig“), b_i gleich Null)

Oder in Matrix Vektor Darstellung

$$Ax = 0 \quad \text{wobei} \quad A = \begin{pmatrix} a_{0,0} & \dots & a_{0,n-1} \\ \vdots & \ddots & \vdots \\ a_{m-1,0} & \dots & a_{m-1,n-1} \end{pmatrix}; \quad x = \begin{pmatrix} x_0 \\ \vdots \\ x_{n-1} \end{pmatrix} \quad (47)$$

Problem: Die Lösung ist nicht eindeutig. Lösen als überbestimmtes Gl.-System führt zur trivialen Nulllösung (also $x_0=0$).

2.7.5.2 LÖSUNG DURCH FORMULIERUNG EINES ERSATZPROBLEMS

Minimiere das 'Ersatzproblem' (durch sog. Lagranges Optimierungsproblem)

$$\underbrace{\|Ax\|^2}_{:=F(x)} \rightarrow \min \quad \text{with} \quad \|x\| = 1 \quad (48)$$

ergibt

$$L(x) := (Ax)^t \underbrace{Ax}_{F(x)} - \lambda \underbrace{(x^t x - 1)}_{=0} = x^t A^t Ax - \lambda \underbrace{(x^t x - 1)}_{=0} \quad (*) \quad (49)$$

die Ableitung von L ergibt

$$L'(x) = 2A^t Ax - 2\lambda x \quad (50)$$

.. und berechne das Extremum x_λ von L

$$2A^t Ax_\lambda - 2\lambda x_\lambda = 0 \Rightarrow A^t Ax_\lambda = \lambda x_\lambda \Rightarrow x_\lambda \text{ ist Eigenvektor von } A^t A \quad (51)$$

.. (51) in (49) ergibt

$$L(x_\lambda) = x_\lambda^t \underbrace{A^t Ax_\lambda}_{\lambda x_\lambda} - \lambda \underbrace{(x_\lambda^t x_\lambda - 1)}_{=0} \Rightarrow L(x_\lambda) = x_\lambda^t \lambda x_\lambda \Rightarrow L(x_\lambda) = \lambda \quad (52)$$

λ ist Eigenwert von $A^t A$ und x der dazugehörige Eigenvektor und damit ist λ Singulärwert von A und x der dazugehörige Spaltenvektor von U (vgl. Abschnitt 2.7.2).

$L(x)$ wird kleiner, je kleiner λ ist. Bei n Singulärwerten ist also der **Spaltenvektor x** der Lösungsvektor, **der zum kleinsten Singulärwert korrespondiert**

2.7.5.3 FOLGERUNG

(a) Die Lösung ist der Spaltenvektor x , der zum kleinsten Singulärwert korrespondiert. **(b)** Die Lösung ist nur bis auf einen Faktor eindeutig, also $a \cdot x$ ist auch eine Lösung

2.7.6 AUSBLICK (ANWENDUNGEN IN DER MULTIVARIATEN STATISTIK)

Eine Anwendung der Singulärwertzerlegung in der multivariaten Statistik ist die sog. **Hauptkomponentenanalyse**, bzw. engl. **principal component analysis, pca**: Hier wird mittels Singulärwertzerlegung aus der der Kovarianzmatrix die Korrelation von Merkmale durch einen Basiswechsel in einen neuen Vektorraum minimiert (vgl. Abbildung 6).

Anhand der Singulärwerte kann jetzt der n dimensionale Merkmalsraum auf signifikante Merkmale reduzieren werden, indem nur der Unterraum der Diagonalmatrix D beibehalten wird, der Singulärwerte $\sigma_i > 0$ beinhaltet. Die Hauptkomponenten ergeben sich dann aus den zu den σ_i korrespondierenden Vektoren u_i der Matrix U .

Schließlich heißt die Hauptachsentransformation in der digitalen Bildverarbeitung oft **Kohene-Loeve Transformation** mit Anwendungen z.B. in der Geradenpassung, Zylinderpassung (bei 3D Punktwolken) oder Bilddatenkompression.

2.8 LÖSEN NICHTLINEARER GLEICHUNGSSYSTEME

2.8.1 EINFÜHRUNG

2.8.1.1 PROBLEMSTELLUNG

Gegeben sei eine nichtlineare Funktion f_β mit bekannten $\mathbf{y}=(y_0, .. y_{n-1})$ und $\mathbf{x}=(x_0, .. x_{n-1})$ sowie einem unbekannten, zu bestimmenden Parametervektor β , so dass

$$\sum_{i=0}^{N-1} \|y_i - f_\beta(x_i)\| \rightarrow \min \quad (53)$$

Der Vektor $\mathbf{x}$ wird im Folgenden Istwert, der Vektor $\mathbf{y}$ wird Sollwert sowie f_β der funktionale Zusammenhang genannt.

Gesucht ist also Parametervektor β_0 , so dass so “gut wie möglich” gilt

$$y_i \approx f_{\beta_0}(x_i) \quad (54)$$

Anmerkung: (1) Hierbei wird angenommen, dass in den Messfehlern keine systematische Information mehr enthalten ist, sondern sie zufällig um die Null schwanken.

(2) Von S können die Nullstellen wegen dem nichtlinearen funktionalen Zusammenhang i.A. nicht effizient oder nicht analytisch berechnet werden.

“So gut wie möglich” für β_0 bedeutet oft den quadratischen Fehler, also

$$\underbrace{\sum_{i=0}^{N-1} \underbrace{\left[y_i - f_{\beta}(x_i) \right]}_{F_i(\beta)}}_{S(\beta)} \rightarrow \min \quad (55)$$

Und damit

$$\underbrace{\sum_{i=0}^{N-1} \underbrace{\left[y_i - f_{\beta}(x_i) \right]^2}_{S(\beta)}}_{S(\beta)} = \sum_{i=0}^{N-1} \underbrace{\left[F_i(\beta) \right]^2}_{S(\beta)} = \|F(\beta)\|^2 \quad (56)$$

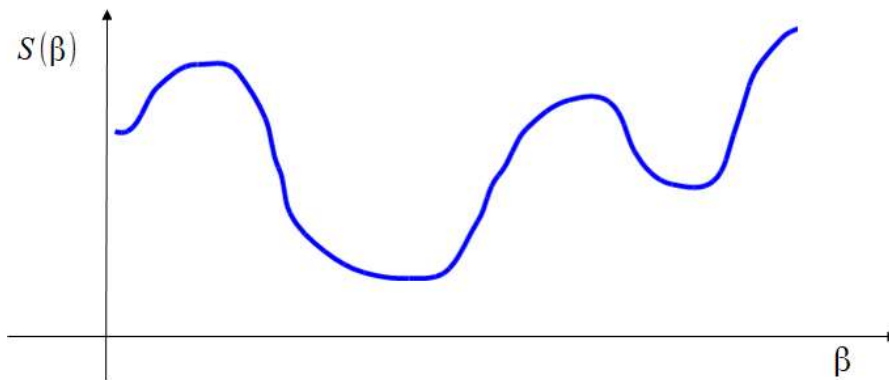


Abbildung 7: Die Abbildung zeigt die schematische Darstellung der in (55) gezeigten Formel

2.8.2 LÖSEN DURCH LINEARISIEREN

2.8.2.1 PROBLEMSTELLUNG UND LÖSUNGSANSATZ

Gesucht: Globales Minimum

Idee: “Wenn die erste Ableitung von S Null ist, dann hat S an dieser Position ein Extremum“

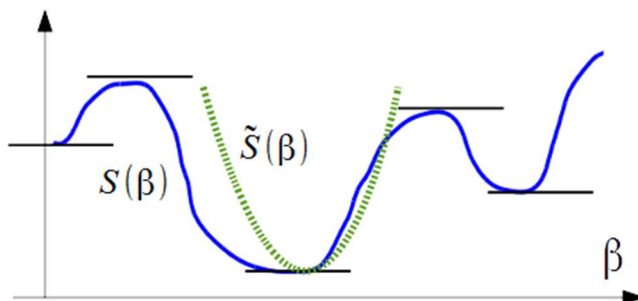


Abbildung 8

Lösungsansatz:

Algorithmus Gauss Newton

Schritt 1: Taylorapproximation 1. Ordnung um β_0 ergibt "Ersatzfunktion" $S\sim$

Schritt 2: Berechne die Ableitung von $S\sim$ und setze sie Null

2.8.2.2 SCHRITT 1 (TAYLORAPPROXIMATION 1. ORDNUNG)

Aus

$$\sum_{i=0}^{N-1} \underbrace{\left[y_i - \underbrace{f_{\beta}(x_i)}_{F_i(\beta)} \right]}_{S(\beta)}^2 \rightarrow \min \quad (57)$$

folgt

$$F(\beta) \approx F(\beta_0) + J(\beta - \beta_0) \quad (58)$$

wobei J die Jacoby Matrix bezeichnet (also die Matrix der ersten Ableitungen nach β) mit

$$J = \begin{pmatrix} \frac{\partial F_1}{\partial \beta_1} & \dots & \frac{\partial F_1}{\partial \beta_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial \beta_1} & \dots & \frac{\partial F_m}{\partial \beta_n} \end{pmatrix} \quad (59)$$

.. Und damit (um den Entwicklungspunkt β_0)

$$S(\beta) \approx \sum_{i=0}^{N-1} \underbrace{\left[F_i(\beta_0) + J_i(\beta - \beta_0) \right]}_{\tilde{S}(\beta - \beta_0) := \tilde{S}(\delta)}^2 \quad (60)$$

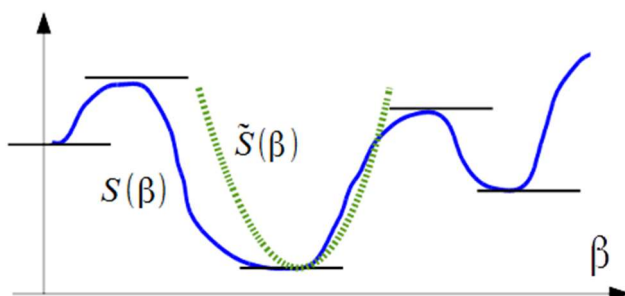


Abbildung 9: Die Abbildung zeigt die Überführung in ein Ersatzproblem: Ist der Entwicklungspunkt gut gewählt, lässt sich das Minimum mit $\tilde{S}$ gut bestimmen

2.8.2.3 SCHRITT 2 (BERECHNE DIE ABLEITUNG VON $S\sim(\Delta)$ UND SETZE SIE NULL)

Aus

$$S(\beta) \approx \underbrace{\sum_{i=0}^{N-1} [F_i(\beta_0) + J_i(\beta - \beta_0)]^2}_{\tilde{S}(\beta - \beta_0) := \tilde{S}(\delta)} \quad (61)$$

folgt in Matrix Schreibweise

$$\tilde{S}(\beta - \beta_0) = \|F_i(\beta_0) + J_i(\beta - \beta_0)\|^2 \quad (*)$$

.. leite (*) nach $\delta = \beta - \beta_0$ ab und setze (*) gleich Null.

$$\underbrace{(J^t J)}_A \underbrace{\delta}_x = \underbrace{J^t F(\beta_0)}_b \quad (**)$$

Herleitung

$$r(x) = (Ax - b)^t (Ax - b) = (b - Ax)^t (b - Ax) = x^t A^t Ax - 2x^t A^t b + b^t b$$

und ableiten ergibt

$$r'(x) = 2A^t Ax - 2A^t b \quad (62)$$

und durch Null setzten folgt

$$\begin{aligned} r'(x_0) = 0 &\Rightarrow 2A^t Ax_0 - 2A^t b = 0 \Rightarrow 2A^t Ax_0 = 2A^t b \Rightarrow \\ x_0 &= (A^t A)^+ A^t b \Rightarrow \delta_0 = (J^t J)^+ J^t (J^t F(\beta_0)) \end{aligned} \quad (63)$$

$$\begin{aligned} &= (b + A\sigma)^t (b + A\sigma) & (b + Ax)^t (b + Ax) &= x^t A^t Ax + 2x^t A^t b + b^t b \\ &\rightarrow & & \end{aligned}$$

$$\begin{aligned} &\text{.. und ableiten} \\ &r'(x) = 2A^t Ax + 2A^t b \end{aligned}$$

.. und Null setzen:

$$\begin{aligned} r'(x_0) = 0 &\Rightarrow 2A^t A x_0 - 2A^t b = 0 & A^t A x_0 = A^t b \rightarrow J^t J \sigma_0 = -J^t F(\beta_0) \\ &\rightarrow & \end{aligned}$$

$$\begin{aligned} \sigma_0 &= (J^t J)^+ J^t F(\beta_0) & \sigma_0 &= (VDU)^+ J^t F(\beta_0) & (\sigma_0 \text{ heit auch Schrittweite}) \\ &\rightarrow & \end{aligned}$$

Anmerkungen

- Jetzt handelt es sich um ein berbestimmtes inhomogenes lineares Gleichungssystem.
- Die Jacobymatrix muss nicht notwendigerweise analytisch berechnet werden, sondern kann numerisch berechnet werden.

2.8.2.4 BEGRIFF (NORMALENGLEICHUNGEN)

Die in (**) hergeleiteten Gleichungen heißen **Normalengleichungen**. Sie können als lineares Ausgleichsproblem gelöst werden (siehe Kapitel 2.7.4).

2.8.3 DAS GAUSS NEWTON VERFAHREN

2.8.3.1 DER GAUSS-NEWTON ALGORITHMUS

Algorithmus Gauss Newton

Input: Fehlerfunktion S , Startwerte β_0

Linearisierung von S an der Position β_0 ergibt

Berechnen des Minimums δ^* der Ersatzfunktion $\hat{S}(\delta)$

Setze $\beta_0 := \beta_0 + \delta$ und wiederhole (1) und (2) bis Abbruchkriterium erfüllt

Output: Parametervektor β_0

2.8.3.2 KONVERGENZEIGENSCHAFTEN

- Kann quadratische Konvergenz erreichen
 $\Rightarrow \|x_{k+1} - x^*\| < c \|x_k - x^*\|^2$
- Konvergenzgeschwindigkeit abhängig von Wahl der Startwerte
- Konvergenz kann nicht garantiert werden
- Bei schlechten Startwerten: lokale Minima

2.8.4 ERWEITERUNG DURCH LEVENBERG-MARQUARDT

2.8.4.1 PROBLEMSTELLUNG UND LÖSUNGSANSATZ

Jacobymatrix enthält Information über die Steigung von S ; Die **Hessematrix** würde zusätzlich Information über die Krümmung enthalten. Also

$$S(\beta) \approx \sum_{i=0}^{N-1} \underbrace{\left[F_i(\beta_0) + J_i(\beta - \beta_0) + H_i(\beta - \beta_0)^2 \right]}_{\hat{S}(\beta - \beta_0)}^2 \quad (64)$$

Aber leider aufwändig zu berechnen

2.8.4.2 IDEE DES LEVENBERG-MARQUARDT ALGORITHMUS

Approximation der Hessematrix aus Jacobymatrix wegen

$$\text{diag}(H) = \text{diag}(J^T J) \quad (65)$$

und also

$$\left(J^T J + \text{diag}(J^T J)\right) \delta_o = J^T F(\beta_0) \quad (66)$$

Und durch Hinzunahme eines **Schrittweitenfaktors** gilt schließlich

$$\left(J^T J + \lambda \text{diag}(J^T J)\right) \delta = J^T F(\beta_0) \quad (67)$$

Wenn Krümmung groß, dann wird Schrittweite σ_0 klein. So werden lokale Extrema nicht übersprungen

2.8.4.3 KONVERGENZEIGENSCHAFTEN

- Durch Erweiterung werden absteigende Funktionswerte erzwungen
- Ähnlich Gauss-Newton, aber wesentlich robuster

3 Kalibrierung von Sensoren

3.1 MODELLIERUNG UND KALIBRIERUNG VON KAMERAS

Gesucht ist eine **Transformation**, die einen in einem Weltkoordinatensystem beschriebenen Punkt auf das korrespondierende Pixel im digitalen Bild projiziert. Hierzu muss die zugrundeliegende **Abbildungsgeometrie** beschrieben und die dazugehörigen, standortspezifischen und kameraspezifischen **Parameter** bestimmt werden.

Im Folgenden wird das Funktionsprinzip einer Kamera erläutert.

3.1.1 FUNKTIONSPRINZIP EINER KAMERA

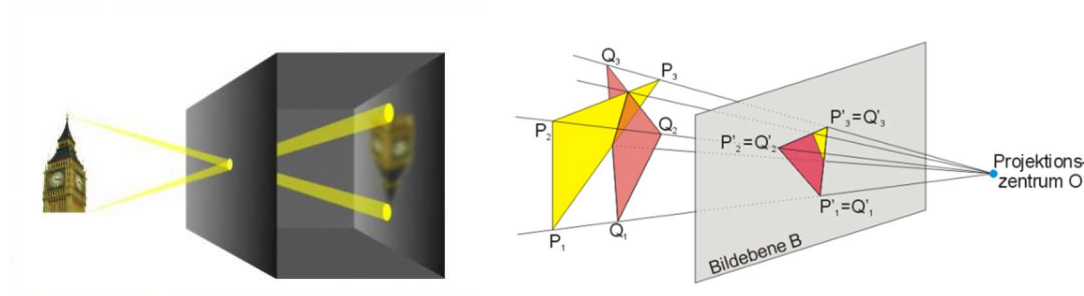


Abbildung 10: Quelle Wikipedia. **(a)** Die Abbildung zeigt das Funktionsprinzip einer Kamera: Lichtstrahlen werden durch eine Blende auf eine photoempfindliche Platte projiziert **(b)** Die Abbildung zeigt schematisch, dass diese Projektion von 3D nach 2D ist und hierbei also eine Dimension – die Tiefeninformation - verloren geht

Abbildung 10 (a) zeigt den **schematischen Aufbau** einer einfachen Kamera. Lichtstrahlen werden als Bündel durch eine Blende **spiegelverkehrt** auf eine photoempfindliche Platte projiziert. Falls **Punkte** auf demselben Sichtstrahl liegen, werden diese - bis auf den am nächsten liegenden Punkt – **abgeschattet**. Insgesamt geht also Information verloren (vgl. Abbildung 10 (b)).

Im Folgenden werden grundlegende Begriffe eingeführt.

3.1.2 GRUNDLEGENDE BEGRIFFE

3.1.2.1 BEGRIFF (LOCHKAMERAMODELL)

Mathematisches Modell zur idealisierten Beschreibung einer Kamera

- Objektiv/Blende als Punkt,
- keine Verzeichnung,
- Chip entspricht einer Ebene

3.1.2.2 ZENTRALPROJEKTION

Strahlen gehen durch einen **gemeinsamen Punkt** und werden auf eine gemeinsame **Ebene projiziert** (siehe Abbildung 10 (b)).

3.1.2.3 BILDKOORDINATENSYSTEM

2D Koordinatensystem, eingebettet in den 3D Raum. Einheit Pixel. Ursprung i.A. linke obere Bildecke (vgl. Abbildung 11).

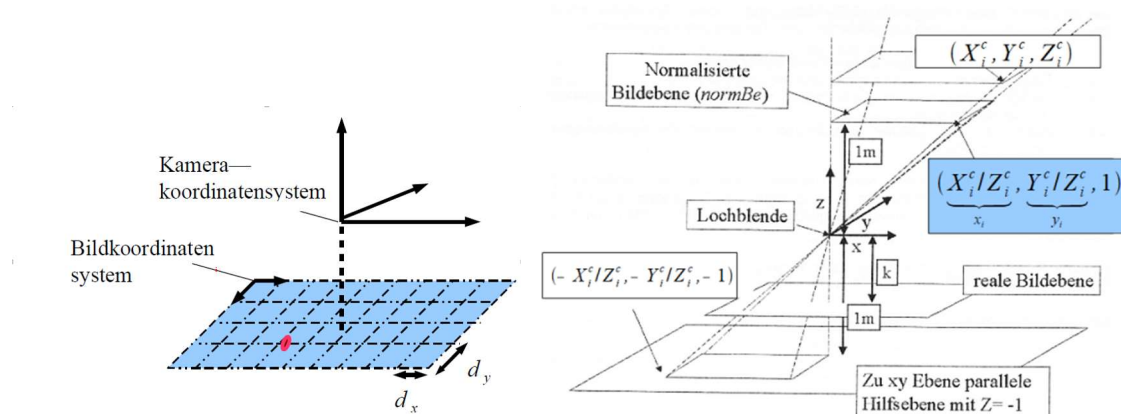


Abbildung 11: (a) Die Abbildung zeigt das Bild- sowie das Kamerakoordinatensystem. (b) schematische Darstellung der zugrunde liegenden Koordinatensysteme und Projektionsebenen

3.1.2.4 WELTKOORDINATENSYSTEM

3D Koordinatensystem (i.A. rechtshändig, orthogonal); z.B. eine Punktwolke, die von einem Laserscanner aufgenommen wurde.

3.1.2.5 KAMERA KOORDINATENSYSTEM

3D Koordinatensystem (i.A. rechtshändig, orthogonal); Beschreibt die Lage und Orientierung einer Kamera relativ zum W-Kos.; Einheit i.A. Meter (oder inch oder feet); Ursprung im Zentrum der (Loch)Blende; Z-Achse so, dass der Schnitt denjenigen Punkt trifft, der keine Verzeichnung hat; X-Achse entlang der Sensorzeile, Y-Achse entlang der Spalte des Sensors (siehe Abbildung 11).

3.1.2.6 BRENNWEITE

Abstand k zwischen Ursprung und Kamerakoordinatensystem (=Lochblende) zum Bildhauptpunkt auf dem Chip (siehe Abbildung 11 (b)).

3.1.2.7 BILDEBENE

Chip idealisiert als Ebene mit Abstand k zur Lochblende (siehe Abbildung 11 (b)).

3.1.2.8 NORMALISIERTE BILDEBENE

Ebene mit Abstand 1 zur Lochblende (siehe Abbildung 11 (b)).

3.1.2.9 BILDHAUPTPUNKT:

Schnitt der Z Achse („=optischen Achse“) des Kamerakoordinatensystems mit der Bildebene (siehe Abbildung 11 (a)).

3.1.2.10 EXTERNE KAMERAPARAMETER:

Beschreiben der **Translation** und **Rotation** vom W-Kos in das Kamerakoordinatensystem. Müssen für jeden Standpunkt neu bestimmt werden

3.1.2.11 INNERE KAMERAPARAMETER I:

Beschreiben die **Abbildungscharakteristik** der Kamera. Pro Kamera nur einmal, solange Mechanik der Kamera unverändert.

3.1.2.12 FREIHEITSGRAD („DEGREE OF FREEDOM - *DOF*“):

Alle Parameter, die **bestimmt** werden müssen, um die gesuchte Transformation zu beschreiben.

3.1.2.13 VERZEICHNUNG

Aufgrund der Brechung der Linse, Schiefstellung des Chips etc. hervorgerufene, **bauartbedingte Abbildungsfehler**.

3.1.2.14 KALIBRIERUNG

Die **Bestimmung** der unbekannten externen und internen Kameraparameter heißt (Kamera)kalibrierung.

3.2 SENSORMOD. UND KALIBRIERUNG NACH ZHANG

3.2.1 PROBLEMSTELLUNG

Gesucht ist eine **Transformation**, die einen in einem Weltkoordinatensystem beschriebenen Punkt auf das korrespondierende Pixel im digitalen Bild projiziert.

3.2.2 BESCHREIBUNG DES SENSORMODELLS

3.2.2.1 FUNKTION *WORLD_2_CAM*

Punkt wird von **Welt- nach Kamerakoordinaten** transformiert (=Drehung und Verschiebung). Die Transformation lautet wie folgt

$$\underbrace{\begin{pmatrix} X^c \\ Y^c \\ Z^c \end{pmatrix}}_M = R \underbrace{\begin{pmatrix} X^w \\ Y^w \\ Z^w \end{pmatrix}}_M + T \quad (6 \text{ DOF}) \quad (68)$$

Und hat folgende **Eigenschaften** (vgl. Abbildung 12 (a)):

- Beide Koordinatensysteme sind dreidimensional,
- insgesamt wird die Transformation mit 6 Parametern, nämlich 3 für die Rotation und drei für die Translation, beschrieben.

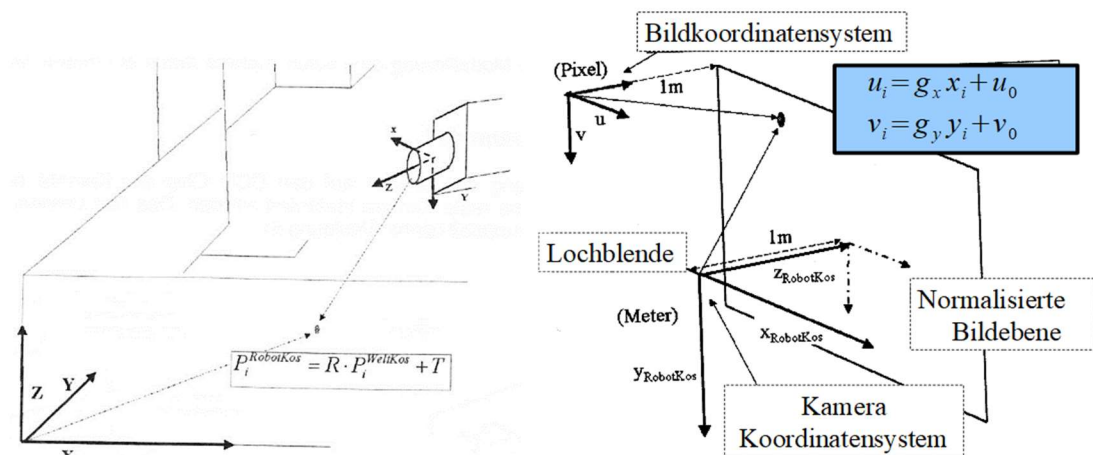


Abbildung 12: (a) schematische Darstellung der Transformation *world_2_cam* (b) schematische Darstellung der Transformation vom Kamerakoordinatensystem in das Bildkoordinatensystem

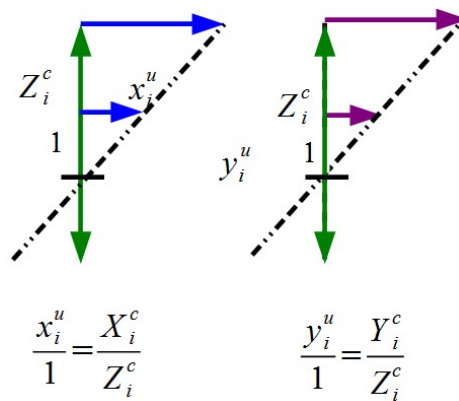
3.2.2.2 FUNKTION CAM_2_NORMBE

Der jetzt im Kamerakoordinatensystem beschriebene Punkt (X^c , Y^c , Z^c) wird als **Zentralprojektion** auf die oben eingeführte normalisierte Bildebene abgebildet. Die Transformation lautet wie folgt

$$\underbrace{x^u = \frac{X^c}{Z^c} \quad ; \quad y^u = \frac{Y^c}{Z^c}}_p \quad (0 \text{ DOF}) \quad (69)$$

und hat folgende **Eigenschaften** (vgl. auch Abbildung 13):

- ($x^u, y^u, 1$) sind also immer noch im Kamerakoordinatensystem beschrieben und unverzeichnet
- Es werden keine Freiheitsgrade (*DOF*) benötigt

Abbildung 13: Schematische Darstellung der Abbildung *cam_2_normBE*

3.2.2.3 FUNKTION *NORMBE_2_CAMDIST*

Diese Abbildung **korrigiert** vorhandene **Verzeichnungen**. Hierbei werden die folgenden beiden Verzeichnungsarten unterschieden.

Radiale Verzeichnung:

Diese Verzeichnungsart wird durch die **Linse** hervorgerufen (vgl. Abbildung 15). Die Korrekturpolynome zur Korrektur lauten wie folgt

$$\begin{aligned} x^{rad} &= x^u (1 + k_1 r^2 + k_2 r^4 + k_3 r^6); & \text{mit} \quad r &= \sqrt{x^u x^u + y^u y^u} \\ y^{rad} &= y^u (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \end{aligned} \quad (70)$$

Die Abbildung hat folgende **Eigenschaften**

- r gibt den Abstand zum Bildhauptpunkt an → keine Verzeichnung im Bildhauptpunkt
- Der Polynomgrad kann variiert werden → Je nach Polynomgrad die Unbekannten k_i und also 1-3 *DOF*

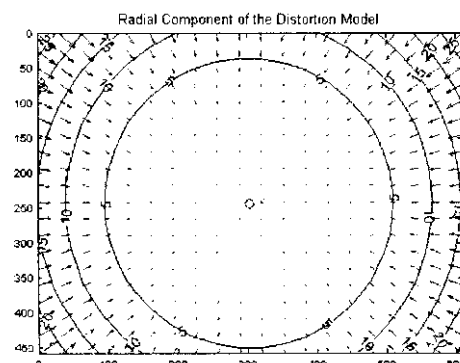
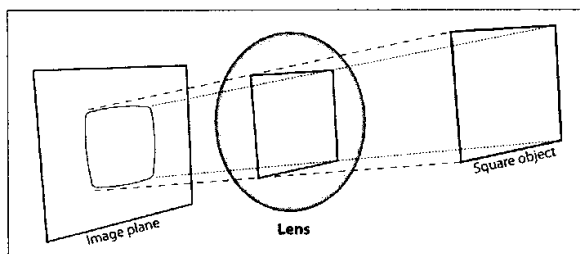


Abbildung 14: Quelle O'Reilly. Die Abbildung zeigt die Auswirkung der radialen Verzeichnung

Tangentiale Verzeichnung:

Diese Verzeichnungsart wird durch die **Verkipfung des Chips** hervorgerufen (vgl. auch Abbildung 15). Die Korrekturpolynome zur Korrektur lauten wie folgt

$$\begin{aligned} x^{\tan} &= 2p_1y'' + p_2(r^2 + 2x''x''); & \text{mit } r &= \sqrt{x''x'' + y''y''} \\ y^{\tan} &= p_1(r^2 + 2y''y'') + 2p_2x'' \end{aligned} \quad (71)$$

Die Abbildung hat folgende **Eigenschaften**:

- r gibt den Abstand zum Bildhauptpunkt an
- Die Unbekannten (p_1, p_2) ergeben zusammen 2 *DOF*

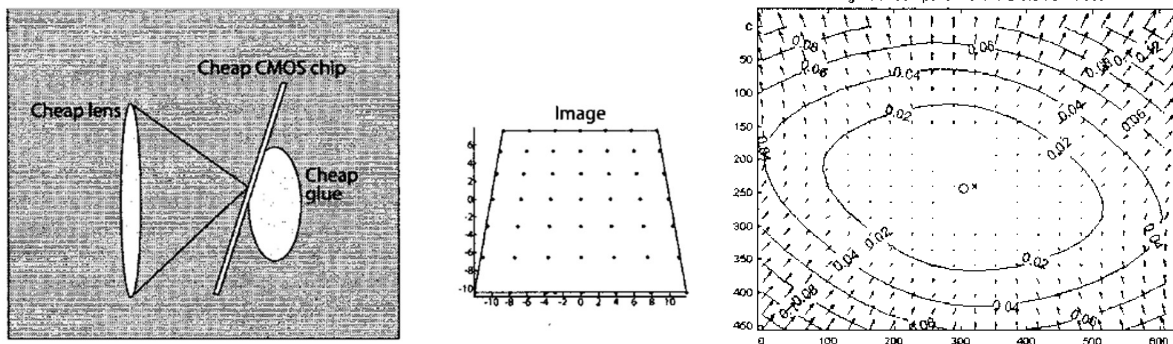


Abbildung 15: Quelle O'Reilly. Die Abbildung zeigt die Auswirkung der tangentialen Verzeichnung

Funktion `normBE_2_camDist`

Die Transformation `normBE_2_camDist` von unverzeichneten nach verzeichneten Punkten lautet dann insgesamt

$$\begin{aligned} x^d &= x^u + x^{rad} + x^{\tan} \\ y^d &= y^u + y^{rad} + y^{\tan} \end{aligned} \quad (1-5 \text{ DOF}) \quad (72)$$

und hat dann je nach Polynomgrad der verwendeten Korrekturpolynome 1-5 *DOF*.

3.2.2.4 FUNKTION `CAMDIST_2_PIXEL`

Diese Abbildung **transformiert** jetzt den immer noch im Kamerakoordinatensystem beschriebenen Punkt $(x^d, y^d, 1)$ vom 3D Kamerakoordinatensystem in das **2D Bildkoordinatensystem**. Die Abbildung lautet

$$\underbrace{\begin{pmatrix} u \\ v \end{pmatrix}}_B = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \end{pmatrix} \begin{pmatrix} x^d \\ y^d \\ 1 \end{pmatrix} \quad (4 \text{ DOF}) \quad (73)$$

und hat folgende **Eigenschaften**

- Die Parameter (c_x, c_y) beschreiben die Translation vom Kamera-KOS in das Bild-KOS und damit die Lage des Bildhauptpunktes im Bildkoordinatensystem
- Die Parameter (f_x, f_y) beinhalten die Skalierung in Pixeleinheiten sowie die Skalierung von der normalisierten Bildebene in die reale Bildebene (und damit die Brennweite).
- Die Parameter (f_x, f_y, c_x, c_y) sind unbekannt $\rightarrow 4 \text{ DOF}$

3.2.2.5 FUNKTION WORLD_2_PIXEL

Insgesamt gilt also dann für die Projektion eines 3D Punktes vom Welt-KOS in das Bild-KOS die Hintereinanderausführung obiger Teilabbildungen und insgesamt

$$\begin{pmatrix} u \\ v \end{pmatrix} = \underbrace{B}_{4 \text{ DOF}} \circ \underbrace{V}_{1-5 \text{ DOF}} \circ \underbrace{P}_{0 \text{ DOF}} \circ \underbrace{M}_{6 \text{ DOF}} \begin{pmatrix} X^w \\ Y^w \\ Z^w \end{pmatrix} \quad (11-15 \text{ DOF}) \quad (74)$$

mit insgesamt also 11-15 Degrees of Freedom (DOF).

3.2.3 KALIBRIERUNG

3.2.3.1 OPTIMIERUNGSMODELL

Seien genügend viele korrespondierende Punkte zwischen mehreren Bildern und einem Weltkoordinatensystem gegeben. Dann lässt sich folgendes Fehlermodell aufstellen (siehe auch Abbildung 16).

$$\sum_{k=0}^M \sum_{i=0}^N \left\| \begin{pmatrix} \tilde{u}_i^k \\ \tilde{v}_i^k \end{pmatrix} - \underbrace{B}_{4 \text{ DOF}} \circ \underbrace{V}_{1-5 \text{ DOF}} \circ \underbrace{P}_{0 \text{ DOF}} \circ \underbrace{M_k}_{k*6 \text{ DOF}} \begin{pmatrix} X_i^w \\ Y_i^w \\ Z_i^w \end{pmatrix} \right\|^2 \rightarrow \min \quad (75)$$

Ablauf Kalibrierung

Schritte der Kalibrierung (Überblick)

Die Berechnung der Kalibrierparameter erfolgt durch

- 1. Schritt:** Startwertberechnung aus Homographie
- 2. Schritt:** Nachiterieren der Lösung mit Levenberg-Marquardt

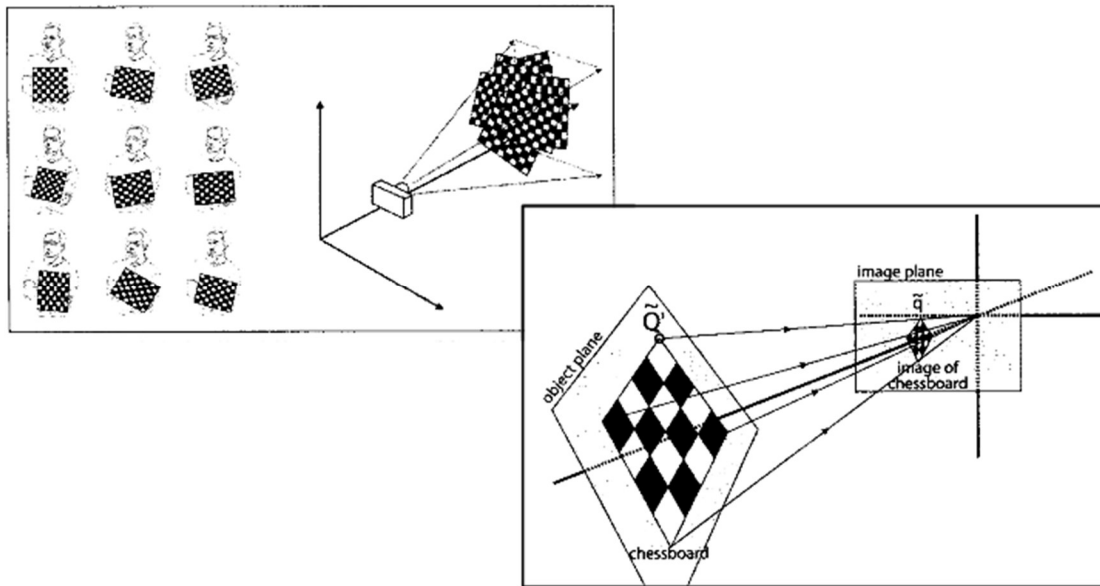


Abbildung 16: Schematischer Ablauf der Mehrbildkalibrierung

3.3 MODELLIERUNG EINES STEREOKAMERASYSTEMS

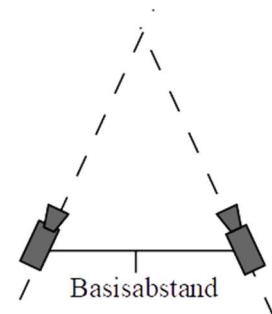
3.3.1 EINFÜHRUNG UND BEGRIFFE

3.3.1.1 BEGRIFF (KALIBRIERUNG STEREOSYSTEM)

Gegeben seien **zwei Kameras** die **fest** zueinander fixiert sind. Gesucht sei die relative Lage und Orientierung beider Kameras zueinander.

3.3.1.2 BEGRIFF (BASISABSTAND)

Der **Abstand** der **optischen Zentren** beider Kameras zueinander heißt Basisabstand.



3.3.1.3 PARAMETER EINES STEREOSYSTEMS

Inneren Kameraparameter (6 + 2*(5-9) DOF):

- Inneren Kameraparameter pro Kamera:
 - $(c_x, c_y, f_x, f_y, p_1, p_2, k_1, k_2, k_3)$ → Je nach Verzeichnung also 5-9 DOF
- Relative Lage und Orientierung zwischen beiden Kameras
 - (R, T) → 6 DOF

Externen Kameraparameter (6 DOF)

6 DOF beschreiben die Lage und Orientierung des Systems relativ zum Weltkoordinatensystem.

3.3.2 BESTIMMUNG DER PARAMETER

Gegeben seien jeweils die externen Parameter beider Kameras zu einem W-KOS als homogenen Matrizen durch M_r, M_l . Dann gilt folgende Beziehung für die Transformation von der rechten Kamera in die linke Kamera

$$M_{rl} = M_l M_r^{-1} \quad (76)$$

4 Stereo- und Bewegtbildverarbeitung

4.1 EINFÜHRUNG

4.1.1 PROBLEMSTELLUNG UND GRUNDBERGRIFFE

Im Folgenden wird auf das Problem und grundlegende Begriffe eingeführt.

4.1.1.1 BEGRIFF (STATISCHE STEREO- UND BEWEGTBILDANALYSE)

Die Untersuchung des **geometrischen Zusammenhangs** zwischen einer Folge von Stereobildpaaren, mit dem Ziel die **Tiefeninformation** von Objekten einer statischen Szene zu rekonstruieren, heißt **statische Stereo- und Bewegtbildanalyse**.

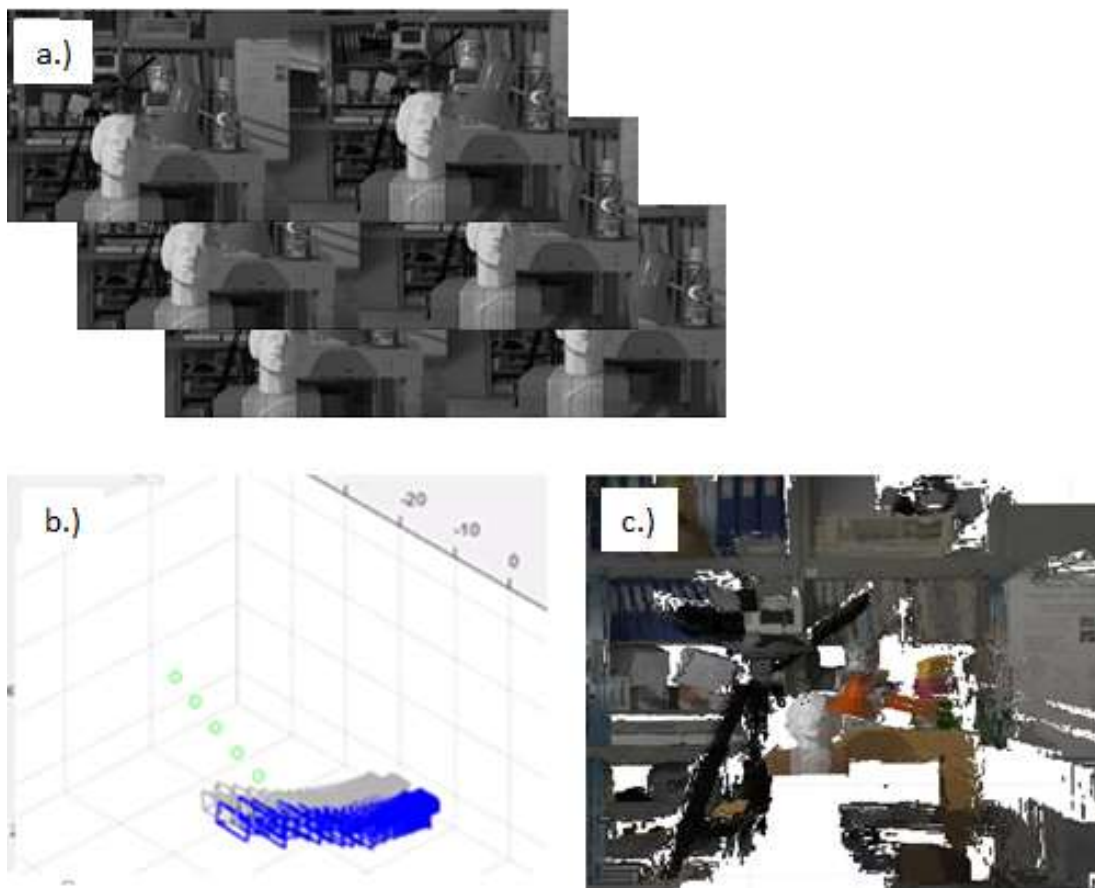


Abbildung 17 (a) Folge aus Stereobildpaaren (b) Daraus rekonstruierte Trajektorie (c) Aus a.) und b.) berechnetes dichtes 3D Modell

Im Folgenden wird auf zwei Begriffe eingegangen, die in der Literatur zusammen oft als die **zwei Probleme der Stereo- und Bewegtbildverarbeitung** bezeichnet werden.

4.1.1.2 BEGRIFF (KORRESPONDENZPROBLEM)

Gegeben sei ein Pixel im linken Bild. Welches Pixel ist das korrespondierende Pixel im rechten Bild? Dieses Problem **das Korrespondenzproblem** (*correspondence problem*):

4.1.1.3 BEGRIFF (3D REKONSTRUKTIONSPROBLEM)

Gegeben seien korrespondierende Pixel. Wie lassen sich daraus 3D Punkte rekonstruieren? Dieses Problem heißt **das 3D Rekonstruktionsproblem** (*reconstruction problem*).

4.2 DIE STANDARDSTEREOGEOMETRIE

4.2.1 MODELLANNAHMEN

Die **Standardstereogeometrie** beinhaltet folgende Modellannahmen

- Kameras liegen in derselben Bildebene
- Kameras besitzen die gleichen inneren Kameraparameter und sind verzeichnungsfrei
- Die optischen Achsen sind exakt parallel, genauso wie die x-Achsen und y-Achsen
- Die optischen Achsen sind senkrecht zur Basislinie dB

Folgerung: Korrespondierende Punkte p1 und p2 liegen in derselben Bildzeile!

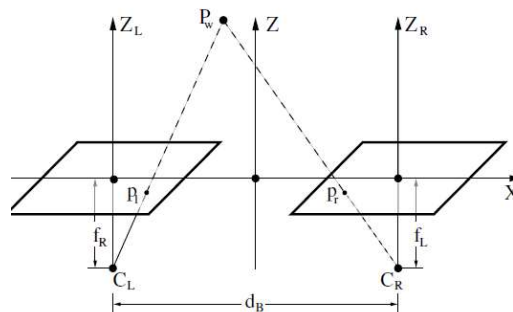


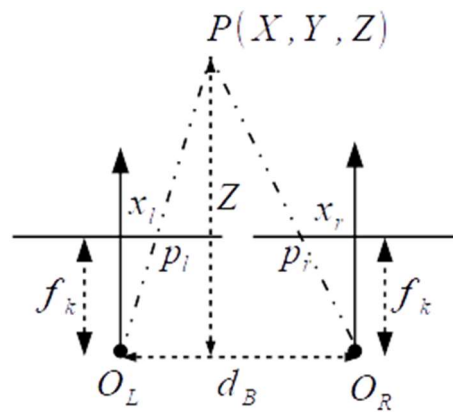
Abbildung 18 Quelle: Klimentjew, Universität Hamburg

4.2.2 3D REKONSTRUKTION EINES PUNKTES

Gegeben Sei eine Aufnahmekonfiguration in Standardstereogeometrie mit der **Brennweite** (focal length) f_k und dem **Basisabstand** d_B . Außerdem seien **korrespondierende Punkte** p_l und p_r gegeben, beide im **linken Koordinatensystem** als Referenzkoordinatensystem beschrieben. Gesucht sei der 3D Punkt $P(X, Y, Z)$.

Lösung: (Für Z) Aus kongruenten Dreiecken

$$\Delta(P, O_R, O_L) \equiv \Delta(p_L, P, p_R) \quad (77)$$



Folgt

$$\frac{p_r - p_l}{Z - f_k} = \frac{d_B}{Z} \quad (78)$$

und somit für $p_l = x_l$ und $p_r = d_B + x_r$

$$\frac{d_B + x_r - x_l}{Z - f_k} = \frac{d_B}{Z} \quad (79)$$

und damit folgt durch Umformung:

$$\frac{d_B + x_r - x_l}{d_B} = \frac{Z - f_k}{Z} \Rightarrow 1 + \frac{x_r - x_l}{d_B} = 1 - \frac{f_k}{Z} \Rightarrow Z = \frac{f_k d_B}{\underbrace{x_l - x_r}_{:=d}} \quad (80)$$

Lösung: (Für X und Y) Aus kongruenten Dreiecken:

$$X = \frac{d_B x_l}{d} \quad , \quad Y = \frac{d_B y_l}{d} \quad (81)$$

Oder alternativ

$$X = \frac{x_l Z}{f} \quad , \quad Y = \frac{y_l Z}{f} \quad (82)$$

Herleitung: $Z/X = (Z - f_k)/(X - x_l) \Rightarrow (Z - f_k)/Z = (X - x_l)/X \Rightarrow (Z - f_k)/Z = 1 - x_l/X \Rightarrow x_l/X = 1 - (Z - f_k)/Z$
 $\Rightarrow x_l/X = (Z - Z + f_k)/Z \Rightarrow x_l/X = f_k/Z \Rightarrow X = x_l/f_k * Z$

4.2.3 DEFINITION (DISPARITÄT)

Der Unterschied $d = x_l - x_r$ der Abbildungskordinaten ein und desselben Punktes der abgebildeten Szene heißt **Disparität**.

4.2.4 FAZIT

Dieses Modell ist ein idealisiertes Modell, da:

- reale Kameras nie zueinander parallel gefertigt werden können
- keine Kamera verzeichnungsfrei gefertigt werden kann

4.2.5 ZUSATZ (DIE KONVERGENTE STEREOGEOMETRIE)

4.2.5.1 MODELLANNAHMEN

- Kameras haben dieselbe **Brennweite**
- Kameras liegen in derselben **Ebene** $\implies$ Optischen Achsen schneiden sich im sog. **Konvergenzpunkt**
- Das gemeinsame **Referenzkoordinatensystem** liegt exakt zwischen beiden Kameras

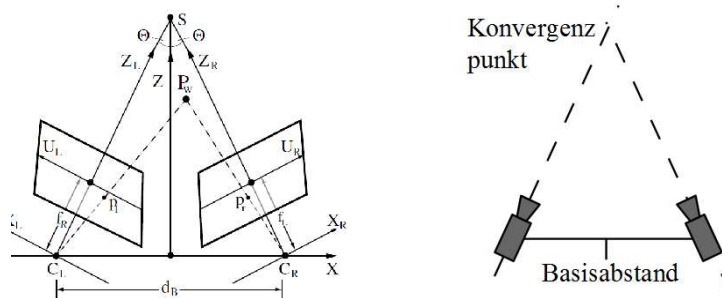


Abbildung 19 (a) Konvergente Kamerageometrie, Quelle: Klimentjew, Univ. Hamburg, (b) Aufbau inkonvergenter Anordnung

4.2.5.2 FAZIT

- Dieses Modell ist **immer noch idealisiert**, da sich laut Modellannahme die optischen Achsen beider Kameras schneiden.
- Anders als bei der Standardstereogeometrie gilt hier jedoch die **Lagebeziehung** zueinander korrespondierender Punkte nicht mehr
- Im Vergleich zur Standardstereogeometrie haben die Kameras jedoch eine bessere **Anordnung** zueinander

4.3 BILDREKTIFIZIERUNG

Ziel dieses Abschnittes ist es, ein gegebenes Stereobildpaar in Standardstereogeometrie zu überführen.

4.3.1 EINFÜHRUNG UND GRUNDLEGENDE BEGRIFFE

4.3.1.1 BEGRIFF (BILDREKTIFIZIERUNG)

Die Überführung des Stereopaars in die Standardgeometrie heißt **Bildrektifizierung**!

4.3.1.2 MODELLANNAHMEN

Die **Kameraparameter** werden als **bekannt** vorausgesetzt. O.B.d.A. kann dann weiter angenommen werden:

- Die Kameras werden als verzeichnungsfrei angenommen.
- Die Daten werden im Kamerakoordinatensystem beschrieben
- Die Kamerabrennweite ist für beide Kameras gleich und wird mit f bezeichnet

Anmerkung: Diese Annahmen sind zulässig, da bei bekannten inneren Kameraparameter die Bilder vorab nach (73) durch Umkehrung der *cam2image* Abbildung verzeichnungsfrei gerechnet werden können.

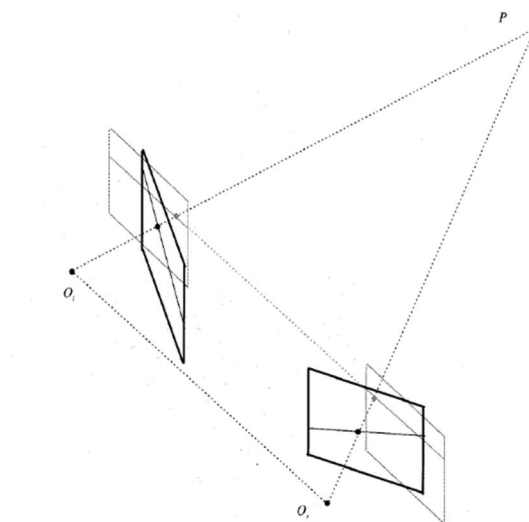


Abbildung 20: Quelle [1] Die Abbildung zeigt die Rektifizierung, d.h. die Überführung eines Stereobildpaares in Standardstereogeometrie

4.3.2 DER ALGORITHMUS RECTIFY

4.3.2.1 KONSTRUKTION EINER DREHMATRIX

Idee: Konstruiere eine Drehmatrix, so dass die horizontale Achse parallel zum Basisvektor liegt.

Die Drehmatrix $R = (e_x, e_y, e_z)$ lässt sich dann wie folgt beschreiben

1. Spalte (entspricht der x -Achse und bewirkt, dass x -Achse parallel zur Basislinie)

$$e_x = \frac{B}{\|B\|} \quad (83)$$

2. Spalte (entspricht der y-Achse und bewirkt, dass e_x und e_y senkrecht auf der optischen Achse stehen)

$$e_y = e_x \times \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (84)$$

3. Spalte (entspricht der z-Achse und ergänzt die Achsen zu einem rechtshändigem Koordinatensystem)

$$e_z = e_x \times e_y \quad (85)$$

Folgende Matrix bewirkt folglich für das im Referenzkoordinatensystem beschriebene Bild die gewünschte Drehung

$$R_{rect} = (e_x, e_y, e_z) \quad (86)$$

4.3.2.2 ALGORITHMUS (RECTIFY)

Algorithmus (Rectify)

Input: externen Kameraparameter R, T , Bildbrennweiten f

1.) Setze für die linke Kamera $R_l = R_{rect}$ und für die rechte $R_r = R_{rect} R$

2.) Drehe jeden Punkt des linken Bildes durch

$$(x', y', z') = R_l(x, y, f) \quad (87)$$

und projiziere diesen wieder auf die gedrehte Bildebene durch

$$(x'', y'', f) = \frac{f}{z'}(x', y', z') \quad (88)$$

3.) Wiederhole Schritt 2.) auch für das rechte Bild, jetzt mit R_r

Output: die rektifizierten Bilder

4.3.3 FAZIT

In diesem Abschnitt wurde gezeigt, wie bei bekannten internen und externen Kameraparametern ein Stereobildpaar in Standardstereogeometrie überführt werden kann.

4.4 KORRESPONDENZANALYSE

4.4.1 EINFÜHRUNG UND BEGRIFFE

4.4.1.1 PROBLEMSTELLUNG

Pixel der einen Kamera sollen dem korrespondierenden Pixel der anderen Kamera **zugeordnet** werden. Falls Kameras in **Standardstereogeometrie** vorliegen, reduziert sich die Suche **entlang derselben Zeile**

4.4.1.2 BEGRIFF (DISPARITÄTSBILD)

Seien ein Stereopaar Img_1 und Img_2 gegeben mit Breite w und Höhe h . Wird jedem Pixel aus Img_1 eine **Disparität** zu ihrem korrespondierenden Pixel in Img_2 zugeordnet (also der Unterschied der Abbildungsordinate (in Pixeln)) und in einem Array abgespeichert, so heißt das so berechnete Bild **Disparitätsbild**.

Anmerkung: Ein Disparitätsbild ist also ein Array (=Bild) mit der Dimension der Eingangsbilder. Jeder Arrayeintrag (m,n) enthält die Disparität des korrespondierenden Pixelpaares (p,q) , also in Standardstereogeometrie die Differenz $Disp(m,n) = n - q$.

Die Verfahren werden in Flächen- und Merkmalsbasierte Verfahren unterteilt.

4.4.1.3 UNTERTEILUNG DER VERFAHREN

Flächenbasierte Verfahren

- Für jedes Pixel im linken Bild soll ein korrespondierender Pixel im rechten Bild gefunden werden
- Bilder müssen in Standardstereogeometrie vorliegen
- Ziel hier: Disparitätsbild, um daraus dichtes 3D Modell zu erstellen

Merkmalsbasierte Verfahren

- Nur für einzelne markante Punkte/Bildmerkmale (Punkte, Kanten) im linken Bild sollen korrespondierende Punkte im rechten Bild gefunden werden.
- Anwendung:
 - Berechnung der relativen Lage und Orientierung beider Bilder zueinander (um Bilder z.B. in Std Geometrie überführen zu können oder um die Eigenbewegung zu schätzen)
 - Ggf. auch um dünnes 3D Modell zu berechnen (i.A. zum Überprüfen)

4.4.2 ANNAHMEN UND EINSCHRÄNKUNGEN

(vgl. Klementjew)

- **Epipolare Geometrie:** Zu einem Punkt im linken Bild korrespondierende Punkte im rechten Bild liegen entlang der Epipolarlinie. Damit reduziert sich der Suchraum auf eine 1D Linie
- **Eindeutigkeit der Zuordnung:** Für jeden Punkt im linken Bild existiert i.A. Nur ein Punkt im rechten Bild.
- **Verträglichkeit von Intensitätswerten:** Zwei Punkte eines Stereopaars sind nur dann korrespondent, wenn die Differenz der Intensitätswerte klein ist

$$\left| \text{Img}_{pl}(u_l, v_l) - \text{Img}_{pr}(u_r, v_r) \right| \quad (89)$$

- **Kontinuität der Disparität:** Sind zwei Punkte eines Stereopaars korrespondent, so können die Nachbarpunkte nur dann korrespondieren, wenn die Differenz

$$\left| \sqrt{(u_{l1} - u_{r1})^2 + (v_{l1} - v_{r1})^2} - \sqrt{(u_{l2} - u_{r2})^2 + (v_{l2} - v_{r2})^2} \right| \quad (90)$$

der Disparitäten gering ist

- **Das Disparitätslimit:** Für sämtliche Punkte im linken Bild kommen nur Kandidaten im rechten Bild in Frage die eine maximale Disparität nicht überschreiten

$$\sqrt{(u_{l1} - u_{r1})^2 + (v_{l1} - v_{r1})^2} < D_{max} \quad (91)$$

- **Reihenfolge der Punkte in den Bildern:** Liegen die Punkte eines Stereobildpaares in einem Bild auf einer Epipolarlinie, so werden diese in gleicher Reihenfolge auf einer korrespondierenden Epipolarlinie des jeweils anderen Bildes abgebildet. Diese Annahme gilt nur, wenn sich die Objekte der Szene ungefähr im gleichen Abstand zur Kamera befinden.

4.4.3 FLÄCHENBASIERTE VERFAHREN

4.4.3.1 BEGRIFF (FLÄCHENBASIERTE VERFAHREN)

Gegeben Bilder in Standardstereogeometrie

Gesucht ist ein Disparitätsbild, um daraus ein dichtes 3D Modell zu berechnen → Die Korrespondenzsuche erstreckt sich also über alle Punkte des Referenzbildes/ Bildausschnittes.

4.4.3.2 BEGRIFF (KORRELATIONSBASIERTES MATCHING, BLOCK MATCHING)

Beim **korrelationsbasiertem Matching** oder **Blockmatching** mit Anwendung in flächenbasierten Verfahren, wird der Bildausschnitt um das aktuelle Pixel pix_l successive mit Bildausschnitten gleicher Größe des anderen Bildes unter Verwendung einer „Ähnlichkeitsmetrik“ verglichen. Es wird dann der Pixel pix_r als korrespondierend betrachtet, dessen Bildausschnitt die größte Ähnlichkeit aufweist.

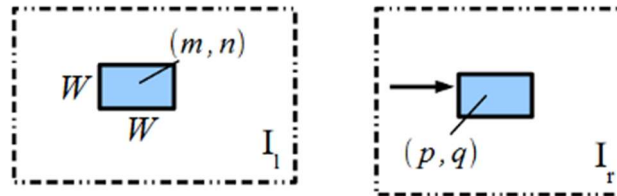


Abbildung 21: Prinzip des korrelationsbasierten Matchings, Blockmatchings

Algorithmus *Disparitätsbild durch Blockmatching*

Input: (a) Bilder I_r und I_l in Std Geometrie (b) Ähnlichkeitsmetrik

Für jedes Pixel (m, n) in I_l

 Für jedes Pixel (m, q) in I_r

 Vergleiche Nachbarschaft $I_l(m, n)$ und $I_r(m, q)$ ob „ähnlich „

 Nehme Pixel q , das am ähnlichsten ist und
 berechne daraus die Disparität durch $Disp(m, n) = n - q$

Output: Disparitätsbild $Disp$

Im Folgenden werden Ähnlichkeitsmetriken vorgestellt, die im Rahmen des Blockmatchings verwendet werden können.

4.4.3.3 ÄHNLICHKEITSMETRIKEN

Normalized cross correlation (NCC)

$$NCC(m, n, p, q) = \frac{\sum_{j=-N/2}^{N/2} \sum_{i=-N/2}^{N/2} (I_l(m-i, n-j) - \bar{I}_l) (I_r(p-i, q-j) - \bar{I}_r)}{\sqrt{\sum_{j=-N/2}^{N/2} \sum_{i=-N/2}^{N/2} (I_l(m-i, n-j) - \bar{I}_l)^2 \sum_{j=-N/2}^{N/2} \sum_{i=-N/2}^{N/2} (I_r(p-i, q-j) - \bar{I}_r)^2}} \quad (92)$$

Mit $\bar{I}_l$ den Mittelwert des linken Bildausschnittes bezeichnet, also

$$\bar{I}_l = \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} I_l(m-i, n-j) \quad (93)$$

Und $\bar{I}_r$ den Mittelwert des rechten Bildausschnittes bezeichnet, also

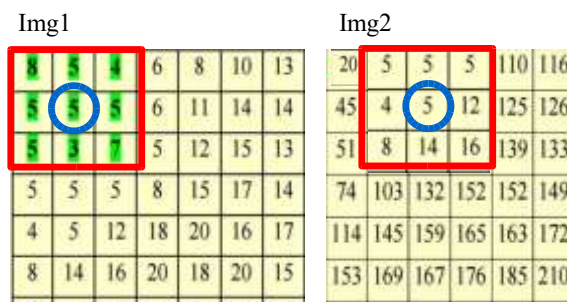
$$\bar{I}_{I_r} = \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} I_l(p-i, q-j) \quad (94)$$

Diese Metrik heißt (um den jeweiligen Mittelwert) **normalisierte Kreuzkorrelation** und wird umso größer, je ähnlicher sich die Intensitätsverläufe beider Bilder sind. Insgesamt gilt:

Anmerkungen:

- Maximale Ähnlichkeit hat den Wert 1, minimale Ähnlichkeit den Wert 0.
- ➔ Bildausschnitte als Vektoren angeordnet werden, dann Berechnung wie Skalarprodukt!

Beispiel (NCC)



- **Schritt 1:** (Normalisierung um den Mittelwert des Ausschnittes)
 - $\text{mean_Ausschnitt_1} = 47 / 9 = 5$
 - $\text{mean_Ausschnitt_2} = 74 / 9 = 8$
- **Schritt 2:** (Berechnung mit normierten Bildern)

Img1			Img2		
3	0	-1	-3	-3	-3
0	0	0	-4	-3	4
0	-2	2	0	6	8

$$\text{NCC}(2, 2, 3, 2) = \frac{((+3)*(-3) + (0*(-3) + (-1)*(-3) + 0*(-4) + 0*(-3) + 0*4 + 0*0 + (-2)*6 + 2*8))}{[\text{sqrt}(3^2 + (-1)^2 + (-2)^2 + 2^2)] * \text{sqrt}((-3)^2 + (-3)^2 + (-3)^2 + (-4)^2 + (-3)^2 + (4)^2 + 6^2 + 8^2)]}$$

Sum of squared differences (SSD)

$$SSD(m,n,p,q) = \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} -\left(I_l(m-i,n-j) - I_r(p-i,q-j)\right)^2 \quad (95)$$

Diese Metrik berechnet die **Summe der quadrierten Differenzen** und wird auch **umso größer, je ähnlicher** sich die Intensitätsverläufe beider Bilder sind (jedoch ohne Normierung des minimalen Wertes; jedoch mit der oberen Schranke 0).

Beispiel (SSD)

$$\begin{aligned} SSD(2, 2, 3, 2) = & \\ & - ((8-5)^2 + (5-5)^2 + (4-5)^2 + \\ & ((5-4)^2 + (5-5)^2 + (5-12)^2 + \\ & ((5-8)^2 + (3-14)^2 + (7-16)^2) = \\ & - 89 \end{aligned}$$

Sum of the absolut differences (SAD)

$$SAD(m, n, p, q) = \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} -|I_l(m-i, n-j) - I_r(p-i, q-j)| \quad (96)$$

Diese Metrik berechnet die Summe der **absoluten Differenzen** und wird auch umso größer, je ähnlicher sich die Intensitätsverläufe beider Bilder sind (jedoch ohne Normierung des minimalen Wertes; jedoch mit der oberen Schranke 0).

Mean square error (MSE)

$$MSE(m, n, p, q) = \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} -(I_l(m-i, n-j) - I_r(p-i, q-j))^2 \quad (97)$$

Diese Metrik berechnet den **mittleren quadratischen Fehler** und wird auch umso **größer**, je **ähnlicher** sich die Intensitätsverläufe beider Bilder sind (jedoch ohne Normierung des minimalen Wertes; jedoch mit der oberen Schranke 0).

Mean Absolute Error (MAE)

$$MAE(m, n, p, q) = \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} -|I_l(m-i, n-j) - I_r(p-i, q-j)| \quad (98)$$

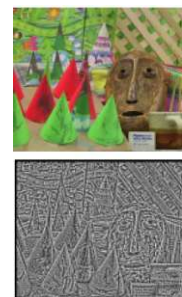
Diese Metrik berechnet **den mittleren absoluten Fehler** und wird auch **umso größer**, je **ähnlicher** sich die Intensitätsverläufe beider Bilder sind (jedoch ohne Normierung des minimalen Wertes; jedoch mit der oberen Schranke 0).

Rank Transform (rank filter)

Schritt 1:

Berechne für jedes Pixel seinen **rank** relativ zu seinen Nachbarpixel

- Eine **Umgebung** um ein Pixel wird nach ihren Grauwerten **aufsteigend geordnet**.
- Dann wird geschaut, an **wievielter Stelle** (=rank) das Pixel in dieser geordneten Folge steht.
- Diese Stelle (=rank) wird **an der Stelle des Pixels** abgespeichert;



- falls dies für alle Pixel durchgeführt wird, **entsteht so jeweils ein neues Bild**, das das sog. **rank-Bild**

$$rank(u, v) = rank_{u, v} \left(\text{sort} \left(\text{Img}_{u-N/2, v-N/2}, \dots, \text{Img}_{u+N/2, v+N/2} \right) \right) \quad (99)$$

Die so entstehenden Bilder heißen **Rank-Images** Rk_l und Rk_r

Schritt 2:

Anwendung einer der Metriken (NCC, SSD, ..) auf die Rank-Images R_{k_l} und R_{k_r} . Diese Metrik vergleicht Pixel paarweise nur relativ, jedoch nicht nach ihren Intensitäten. Der Vorteile dieser Metrik ist, dass sie **sehr robust** gegen Ausreißer, Belichtungsunterschiede und Rauschen ist. Nachteile sind **Informationsverlust**, und damit keine signifikante Beschreibung jedes Pixels.



Abbildung 22 [<http://siddhantahuja.wordpress.com/>]

Beispiel (rank):

Schritt 1 (Rank um das Pixel (2,2)):

$$(8, 5, 4, 5, 5, 5, 5, 3, 7) \implies (3, 4, 5, 5, 5, 5, 7, 8) \\ \implies \text{Rank Im}g_1(2, 2) = 5$$

Schritt 2 (Blockmatching):

Auf beide rank-Bilder wird jetzt das Blockmatching mit z.B. SSD angewendet

Figure 1 shows two images, **Img1** and **Img2**, each containing a 6x4 grid of numbers. In **Img1**, a 3x3 red bounding box highlights the top-left region, and a blue circle marks the center pixel (value 5). In **Img2**, a 3x3 red bounding box highlights the top-right region, and a blue circle marks the center pixel (value 5).

Zensus Transformation

- Vergleich Grauwert des zentralen Pixels mit Nachbarn (z.B. 3x3 Nachbarschaft üblich)
- Gehe zeilenweise durch und berechne daraus Bit String, einen sog. **Signatur Vektor** mit „0“ falls Wert größer und "1" falls Wert kleiner als Grauwert des zentralen Pixels
- ➔ Bei einer 3x3 Umgebung und ohne Vergleich mit sich sind dies genau 1 Byte
- Dieser Signatur-Vektor kann jetzt mit anderen Signatur-Vektoren auf Ähnlichkeit verglichen werden kann.
- **Vorteil:** ebenfalls robust gegen Ausreißer, Rauschen, Belichtungsunterschiede

4.4.4 MERKMALSBASIERTE VERFAHREN

Wie in Abschnitt 4.4.1.3 beschrieben, ist es das Ziel der merkmalsbasierten Verfahren, nur spezifisch ausgewählte Punkte im linken Bild korrespondierenden Punkten im rechten Bild zuzuordnen, um daraus bspw. die Eigenbewegung einer Kamera zu schätzen.

Im Folgenden werden hierzu grundlegende Begriffe eingeführt, bevor dann auf konkrete Verfahren eingegangen wird.

4.4.4.1 GRUNDLEGENDE BEGRIFFE

Definition (Markanter Punkt, Corner Point, Keypoint)

Ist ein Punkt (z.B. Ecken, Schnitt von Geraden etc.) im Bild eindeutig in seiner lokalen Umgebung zu identifizieren, so heißt dieser Punkt **markanter Punkt** oder **Keypoint**.

Begriff (Merkmalsvektor, Feature Point/ Keypoint Deskriptor)

Charakteristische Beschreibung eines markanten Punktes durch spezifische Merkmale, anhand derer der Punkt (eindeutig) von anderen Punkten unterschieden werden kann. Der Vektor, indem diese Merkmale für jeden Punkt gespeichert werden, heißt **Merkmalsvektor**, **Feature Point Deskriptor** oder **Keypoint Deskriptor**.

Begriff (Feature Matching)

Die Zuordnung markanter Punkte zweier Bilder anhand ihrer Feature Point Deskriptoren heißt **Feature Matching**.

Begriff (Point Detection, Feature Point Extraction)

Die Extraktion markanter Punkte in einem Bild heißt **Point Detection**. Wird den extrahierten Punkten noch ein Merkmalsvektor zugeordnet, so heißt dieser Schritt allgemein Feature Point Extraktion.

Im Folgenden wird gezeigt, wie markante Punkte in einem Bild bestimmt werden können.

4.4.4.2 CORNER POINT DETECTION

Ziel: Suche markanter Punkte im Bild

Der Moravec-Operator

Idee: Berechne die mittlere quadratische Gradienten Summe in 4 Hauptrichtungen eines Bildfensters. 

$$\begin{aligned}
V_1^{(u,v,\delta)} &:= \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} (I(u-i, v-j) - I(u-i, v-j+\delta))^2 \\
V_2^{(u,v,\delta)} &:= \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} (I(u-i, v-j) - I(u-i, v-j-\delta))^2 \\
V_3^{(u,v,\delta)} &:= \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} (I(u-i, v-j) - I(u-i-\delta, v-j))^2 \\
V_4^{(u,v,\delta)} &:= \frac{1}{(N+1)^2} \sum_{i=-N/2}^{N/2} \sum_{j=-N/2}^{N/2} (I(u-i, v-j) - I(u+i, v-j))^2
\end{aligned} \tag{100}$$

Ist das Ergebnis für alle 4 Hauptrichtungen groß, dann wird gefolgt, dass sich das Pixel signifikant von seiner Umgebung unterscheidet und ein markanter Punkt vorliegt. Also

Input Bild I

```

Durchlaufe das Bild für alle Pixel (u,v)

  Berechne V1, V2, V3, V4 für das Pixel (u,v)
  if min(V1,V2,V3,V4) > threshold
    füge (u,v) in M ein

```

Output Liste M mit Koordinaten der markanten Punkte

Beispiel für $V_3^{(3,3,1)}$

100	102	99	100	101
99	101	99	110	102
95	99	97	87	103
109	100	99	97	103
111	99	105	98	102

$$\begin{aligned}
V_1^{(3,3,1)} &= 1/9 * ((101-99)^2 + (99-101)^2 + (110-99)^2 + \\
&\quad (99-95)^2 + (97-99)^2 + (87-97)^2 + \\
&\quad (100-109)^2 + (99-100)^2 + (97-99)^2) \\
&= 1/9 * (2^2 + 2^2 + 11^2 + 4^2 + 2^2 + 10^2 + 9^2 + 1^2 + 2^2)
\end{aligned}$$

Fazit: Einfach zu implementieren, jedoch nicht subpixelgenau

Ausblick (Polynomapproximation und Harris Corner Detektor)

Andere Verfahren approximieren die lokale Grauwertumgebung um das zentrale Pixel durch eine Polynomfläche p , um dann aus dessen Hauptkrümmungsrichtungen Eigenschaften abzuleiten (siehe Abbildung). Der Vorteil hierbei ist die Subpixelgenauigkeit und Rotationsinvarianz. Vergleiche hierzu auch den **Harris Corner Detector** und den **Förstner Operator**.

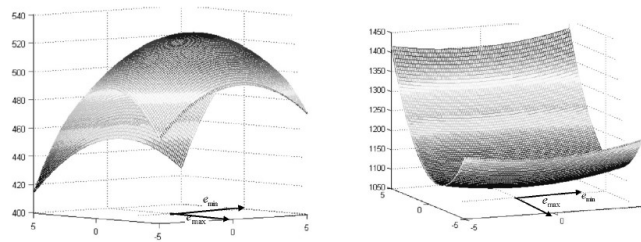


Abbildung 23 Quelle: Abmayr [2] Die Abbildung zeigt eine durch die umliegenden Grauwerte approximierte Polynomfläche (a.) Für eine Ecke (b.) für eine Kante.

Im Folgenden wird das Template Matching zur Suche korrespondierender Pixel in zwei Bildern vorgestellt.

4.4.4.3 TEMPLATE MATCHING ZUR KORRESPONDENZSUCHE

Ziel: Suche von Punktkorrespondenzen in zwei Bildern.

Begriff (template Matching)

Seien zwei verschieden große Ausschnitt zweier Bilder gegeben und die Pixelposition der größten Übereinstimmung zwischen diesen Bildausschnitten gesucht.



Wird jetzt der kleinere Bildausschnitt sukzessive durch den größeren Bildausschnitt "geschoben" und ein *Score*, d.h. ein Grad der Übereinstimmung für die aktuelle Pixelposition anhand einer Ähnlichkeitsmetrik (vgl. Abschnitt 4.4.3.3) berechnet, so wird dieser Ablauf mit **Template Matching** bezeichnet.

Algorithmus (Template Matching zur Korrespondenzsuche)

Schritt 1 Corner Point Detection

Schritt 2 Feature Matching using Template Matching

Schritt 1 (Corner Point Detection)

⇒ vgl. hierzu Absatz 4.4.4.2

Schritt 2 (Feature Matching using Template Matching)

Der folgende Algorithmus zeigt einen Feature Matching Ansatz unter Verwendung von des in Absatz 0 eingeführten Template Matching.

Algorithmus Feature based Matching applying Template Matching

Input: (a) Bilder Img_r und Img_l , (b) Merkmalspunkte F_r und F_l (c) Ähnlichkeitsmetrik

```

For jedes Pixel (m,n) aus  $F_l$  {
    For jedes Pixel (p,q) aus  $F_r$  {

```

```
// template Matching zwischen  $I_1(m,n)$  und  $I_2(p,q)$ 
```

```
 $d_{p,q} := \text{TemplateMatching über Bildausschnitte mit Ähnlichkeitsmetrik } \phi(m,n,p,q)$ 
```



```
zu (m,n) korr. Punkt :=  $\max_{(p,q) \in F_p} (\{d_{p,q}\})$ 
```

Output: Korrespondierende Punktepaare

Fazit Der Algorithmus ist robust und performant, jedoch nicht rotationsinvariant und nicht skalierungsinvariant. In der Bewegtbildverarbeitung findet er insbesondere Anwendung, wenn es sich um eine Bildfolge handelt, bei der sich benachbarte Bilder relativ wenig voneinander unterscheiden.

4.4.4.4 SCALE INVARIANT FEATURE TRANSFORM (SIFT)

SIFT (**Scale Invariant Feature Transform**) liefert eine Methode, die sowohl skalen- als auch rotationsinvariant ist.

Der Algorithmus lässt sich in folgende Schritte unterteilen:

Übersicht

Algorithmus SIFT (Übersicht)

Pro Bild Feature Extraktion

Schritt 1: Suche markanter Punkte im Skalenraum
 Schritt 2: Lokale Orientierungen für jeden markanten Punkt berechnen
 Schritt 3: Feature Deskriptor erstellen

Pro Bildpaar

Schritt 4: Feature Matching

Schritt 1 (Suche markanter Punkte im Skalenraum)

Ziel: Markante Punkte müssen in unterschiedlichen Entfernungen markant sein.

Hierzu werden die folgenden Schritte durchgeführt:

- a.) **Simulation unterschiedlicher Entfernungen (=Skalenraum)** Berechnung einer neuen Bildsequenz durch Gauss-Tiefpassfilterung (=Weichzeichnungsfilter) mit unterschiedlichen Kernelgrößen.

Beispiel

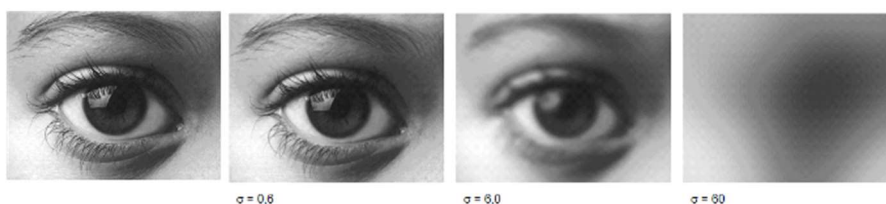


Abbildung 24: Die Abbildung zeigt die Auswirkung eines Gauss-Tiefpass Filters, mit unterschiedlichen Filterkernel.

$$\frac{1}{16} \underbrace{\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}}_{\sigma=0,3} ; \quad \frac{1}{256} \underbrace{\begin{pmatrix} 0 & 2 & 4 & 2 & 0 \\ 2 & 14 & 28 & 14 & 2 \\ 4 & 28 & 56 & 28 & 4 \\ 2 & 14 & 28 & 14 & 2 \\ 0 & 2 & 4 & 2 & 0 \end{pmatrix}}_{\sigma=0,6} \quad (101)$$

- b.) Berechnung von Extrema aus Difference of Gaussian Bilder** Je nach Filtergröße wird das Ergebnisbild unterschiedlich stark weichgezeichnet. Die Differenz zweier so unterschiedlich stark weichgezeichneter Bilder heißt Difference of Gaussian (DoG).
- c.) Aus DoG: Berechnung einer Liste L aller Extrema** Falls Pixel (u,v) Extrema im Bild DoG_i sowie im Bild DoG_{i-1} und DoG_{i+1} dann wird er in die Liste L der Extrema Punkte aufgenommen.
- d.) Identifikation markanter Punkte:** Falls ein Extrempunkt aus L nach dem Harris Kriterium (vgl. Abschnitt 4.4.4.2) ein markanter Punkt ist, dann wird er in die Liste M der markanten Punkte eingeordnet und als markanter Punkt M_i definiert.

2. Schritt (Markanten Punkten ihre lokale Orientierung zuordnet)

Ziel: Invarianz gegen Rotationen, indem jedem Punkt $M_i = (u_i, v_i)$ eine lokale Orientierung zugeordnet wird. Dies erfolgt in zwei Schritten

- a.) Lokale Orientierungen der Nachbarschaft** Berechne hierzu für jedes Pixel (p_k, q_k) in einer $N \times N$ Region um $M_i = (u_i, v_i)$ seines Gradienten $\text{grad} = (d_x, d_y)$ aus seinen approx. diskreten Ableitungen $\text{grad} \approx (I(q_k, p_{k+1}) - I(q_k, p_{k-1}), I(q_{k+1}, p_k) - I(q_{k-1}, p_k))$ und leite hiervon dessen Orientierung und Betrag ab durch $\alpha = \text{atan}(dy, dx)$ und $w := \|\text{grad}\|$

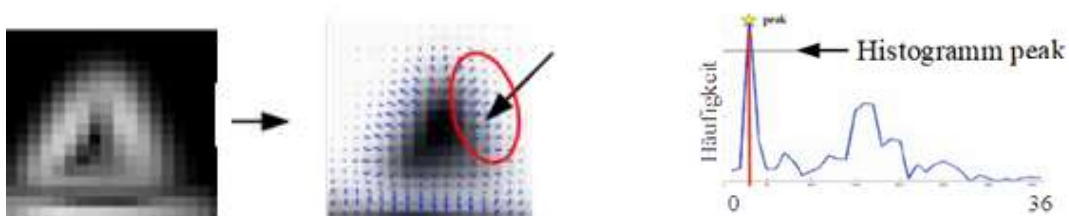


Abbildung 25 Quelle: Wikipedia Die Abbildung zeigt Schritte des SIFT Algorithmus. **(a.)** Lokale Gradienten um die Nachbarschaft, sowie **(b.)** die daraus über das Histogramm der mit dem Gradienten Betrag gewichteten Orientierung abgeleitete dominante Gradienten Richtung

Beispiel: Gegeben sei die 3x3 Umgebung durch

$$GV = \begin{pmatrix} 125 & 150 & 130 \\ 103 & 100 & 110 \\ 88 & 70 & 72 \end{pmatrix} \quad (102)$$

Dann lässt sich der Gradient approximieren durch $\text{grad} = (110 - 100, 150 - 100)$ und daraus die Orientierung durch $\alpha = \text{atan}(150 - 70, 110 - 103) = 84^\circ$ und seine Norm durch $\|\text{grad}\| = 80,3$ ableiten.

b.) Lokale Orientierung O_i für Markanten Punkt M_i bestimmen: Die Region mit dem größten Gradienten Betrag definiert die Orientierung des markanten Punktes: Stelle hierzu die lokalen Orientierungen als Histogramm dar (gewichtet mit dem Gradienten Betrag) und wähle Bin mit größter Häufung (üblich sind 36 Bins, also 10 Grad Schritte).

3. Schritt (Keypoint Deskriptor erstellen)

Ziel: Charakteristische Beschreibung des Punktes durch Nachbarschaft

Die Orientierungen θ_k um einen Punkt $M_i = (u_i, v_i)$ werden hierzu zu Regionen zusammengefasst und als Histogramm in einen Merkmalsvektor geschrieben. Die Einträge erfolgen in relativ zur lokalen Orientierung O_i (üblich sind 4x4 Regionen, mit 8 Bins pro Region, als 4x4x8 Einträgen).

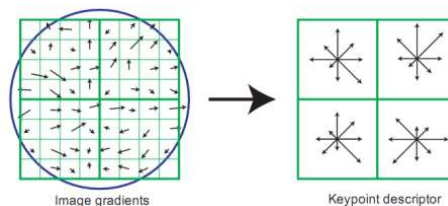


Abbildung 26 Quelle: Wikipedia **(a.)** Die Abbildung zeigt wie die lokalen Orientierungen in z.B. 4 Regionen aufgeteilt werden **(b.)** Die Abbildung zeigt, wie in den 4 Regionen jeweils die Histogramme der Orientierung mit z.B. 8 Bins pro Region gerechnet wurden. Insgesamt gibt sich so ein Keypoint Deskriptor der Größe 4x8.

4. Schritt (Feature Matching):

Ziel: Suche übereinstimmender Punkte in zwei Bildern.

Hierzu werden die in obigem Schritt erstellten *Keypoint Deskriptoren* für Punkte unterschiedlicher Bilder bzgl. des euklidischen Skalarproduktes miteinander verglichen. Gut übereinstimmende Punkte werden als Matches betrachtet.

Fazit: Robuster Algorithmus zum Finden von korrespondierenden Punkten in zwei Bildern. Falls die Abweichung „nicht zu groß ist“ ist der Algorithmus invariant gegen Rotation und Skalierung. Aufwendiger zu implementieren als Template Matching, weniger performant.

4.5 DIE EPIPOLARGEOMETRIE

4.5.1 MODELLANNAHMEN UND BEGRIFFE

4.5.1.1 MODELLANNAHMEN

Folgende Modellannahmen gelten:

- Kameras können beliebige Brennweiten haben
- Kameras liegen nicht notwendig in derselben Ebene
- Das Referenz- bzw. Weltkoordinatensystem liegt in der linken Kamera
- Die Kameras werden o.B.d.A. als verzeichnungsfrei angenommen. Diese Annahme ist zulässig, da bei bekannten inneren Kameraparameter die Bilder vorab nach (73) durch Umkehrung der *cam2image* Abbildung verzeichnungsfrei gerechnet werden können.

4.5.1.2 BEGRIFF (EPIPOLAREBENE)

Die durch die Punkte X, O_L und O_R aufgespannte Ebene heißt **Epipolarebene** (siehe Abbildung 27). Die auf die Bildebene projizierten Punkte x_L und x_R liegen ebenfalls in dieser Ebene.

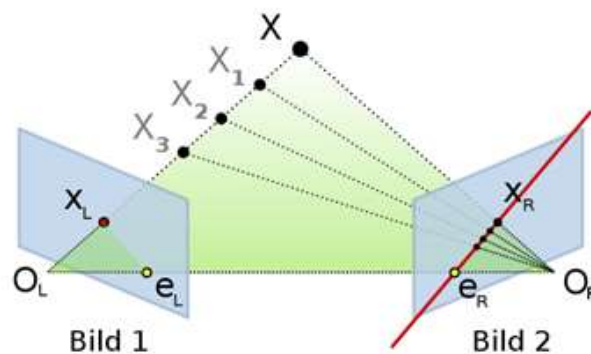


Abbildung 27: Epipolargeometrie Quelle: Wikipedia

4.5.1.3 BEGRIFF (EPIPOLARLINIE)

Die aus dem Schnitt zwischen Epipolarebene und den jeweiligen Bildebenen entstehenden Linien $l_L^{(X)}$ und $l_R^{(X)}$ heißen **Epipolarlinien** (vgl. Abbildung 27 (b)).

4.5.1.4 BEGRIFF (EPIPOL)

Der Schnittpunkt e_L bzw. e_R der Basislinien mit der Bildebenen heißt **Epipol**. Er ist der Projektionspunkt des jeweils anderen optischen Zentrums auf die Bildebene (vgl. Abbildung 27 (b)).

4.5.1.5 FOLGERUNGEN

- Der zu einem Punkt x_L korrespondierende Punkt x_R muss nur entlang der Epipolarlinie $l_R^{(x_L)}$ gesucht werden und umgekehrt (vgl. Abbildung 27). Diese Eigenschaft folgt auch aus dem **epipolar constraint**, vgl. auch den folgenden Abschnitt 4.5.2
- Punkte die oberhalb/unterhalb einer Epipolarebene liegen, erzeugen eine andere Epipolarebene und somit auch andere Epipolarlinien
- Alle Epipolarebenen haben die Basislinie als **gemeinsame Schnittlinie**
- Alle Epipolarlinien einer Kamera gehen durch ihren Epipol

4.5.1.6 FAZIT

Die Verwendung der Epipolargeometrie schränkt den Suchraum für korrespondierende Punkte stark ein. Trotzdem ist die Suche noch sehr rechenaufwendig.

4.5.2 DIE KOMPLANARITÄTSBEDINGUNG

4.5.2.1 GEOMETRISCHER ZUSAMMENHANG

Bezeichne die Transformation von der linken zur rechten Kamera mit R (Rotation) und B (Translation). Bezeichne den Punkt P_w mit P_L , wenn er im linken Kamerakoordinatensystem beschrieben ist und bezeichne ihn mit P_R , wenn er im rechten Kamerakoordinatensystem beschrieben ist (vgl. Abbildung 28). Dann gilt

$$P_R = RP_L - RB \Rightarrow P_R = R(P_L - B) \quad (103)$$

und

$$p_L = \frac{f_L}{Z_L} P_L \quad \text{und} \quad p_R = \frac{f_R}{Z_R} P_R \quad (104)$$

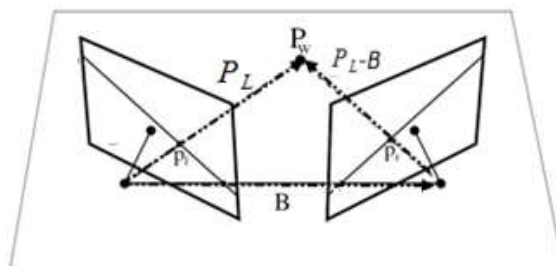


Abbildung 28: Epipolargeometrie

4.5.2.2 DIE KOMPLANARITÄTSBEDINGUNG

Unter Verwendung der durch B und die Vektoren P_L und $P_L - B$ aufgespannten Epipolarebene gilt folgende Bedingung

$$\underbrace{(P_L - B)^t \left(\underbrace{B \times P_L}_{\text{Vektor} \perp \text{Epipolarebene}} \right)}_{\text{Vektoren} \perp \text{zueinander}} = 0 \quad (105)$$

und heißt **Komplanaritätsbedingung** (eng. **epipolar constraint**).

4.5.3 DIE ESSENTIELLE MATRIX

4.5.3.1 HERLEITUNG

Es gilt mit (105) und unter Verwendung von (103)

$$(P_L - B)^t B \times P_L = 0 \Rightarrow (R^t P_R)^t B \times P_L = 0 \Rightarrow P_R^t R B \times P_L = 0 \quad (106)$$

Dies lässt sich schreiben wie folgt

$$P_R^t R \underbrace{\begin{pmatrix} 0 & -B_Z & B_Y \\ B_Z & 0 & -B_X \\ -B_Y & B_X & 0 \end{pmatrix}}_{\substack{:=S \\ :=E}} P_L = 0 \quad (107)$$

Also gilt insgesamt

$$P_R^t \underbrace{E}_{=RS} P_L = 0 \quad (108)$$

Außerdem gilt unter Verwendung von (104) folgender Zusammenhang für auf die Bildebene projizierte Punkte

$$P_R^t E P_L = 0 \Rightarrow \left(p_R \frac{Z_R}{f_R} \right)^t E \left(p_L \frac{Z_L}{f_L} \right) = 0 \Rightarrow p_R^t E p_L = 0 \quad (109)$$

4.5.3.2 BEGRIFF (ESSENTIELLE MATRIX)

Die in (108) bzw. (109) gefundene Matrix E liefert eine Beziehung zwischen den epipolar constraint und den externen Kameraparameter und heißt **Essentielle Matrix** (eng. **essential matrix**).

4.5.3.3 FAZIT

Die in (109) gezeigte Beziehung gilt für alle Punkte p_i auf der Epipolarlinie l_{p_i} . Die **Essentielle Matrix** liefert diesen Zusammenhang.

4.5.4 DIE FUNDAMENTALMATRIX

Bis jetzt erfolgten alle Herleitungen in **Kamerakoordinaten**. In Bildern liegen jedoch Pixelkoordinaten vor. Der Übergang zu Pixelkoordinaten wird in diesem Abschnitt dargestellt. Gesucht ist jetzt also ein Zusammenhang zwischen **korrespondierenden Pixeln**.

4.5.4.1 HERLEITUNG

Bezeichne die Matrix der inneren Kameraparameter mit M_l und M_r . Dann gilt für die Umrechnung in Pixel $\tilde{p}_L = Mp_L$ bzw. $\tilde{p}_R = Mp_R$. Folglich lässt sich (109) dann schreiben zu

$$p_R^t E p_L = 0 \Rightarrow (M_R^{-1} \tilde{p}_R)^t E M_L^{-1} \tilde{p}_L = 0 \Rightarrow \tilde{p}_R^t \underbrace{(M_R^{-1})^t E (M_L^{-1})}_{=F} \tilde{p}_L = 0 \quad (110)$$

Also gilt insgesamt

$$p_R^t E p_L = 0 \Rightarrow \tilde{p}_R^t F \tilde{p}_L = 0 \quad (111)$$

4.5.4.2 BEGRIFF (FUNDAMENTALMATRIX)

Die in (111) hergeleitete Matrix heißt **Fundamentalmatrix** und liefert einen Zusammenhang zwischen den Kameras ohne Kenntnis der Kameraparameter (jedoch unter der Annahme, dass die Kameras verzeichnungsfrei sind).

4.5.5 VERGLEICH ESSENTIELLE MATRIX UND FUNDAMENTALMATRIX

Essentielle Matrix

- Beinhaltet nur Information über die externen Kameraparameter
- Es gilt $\text{rang}(E) = 2 \rightarrow$ Ein Eigenwert muss gleich Null sein
- Die zwei (ggf. komplexen) Eigenwerte ungleich Null sind gleich¹

Fundamentalmatrix

- Beinhaltet Information über die externen und internen Kameraparameter
- Es gilt $\text{rang}(F) = 2 \rightarrow$ Ein Eigenwert muss gleich Null sein, die anderen sind ggf. komplex

¹ Dass die Eigenwerte von schiefsymmetrischen 3x3 Matrizen komplex sein können, lässt sich wie folgt zeigen:
 $Ax = \pi x \rightarrow Ax - \pi x = 0 \rightarrow (A - \pi I)x = 0 \rightarrow (A - \pi I) = 0 \rightarrow (a_{11} - \pi, a_{12}; -a_{12}, a_{22} - \pi) = 0 \rightarrow (a_{11} - \pi)(a_{22} - \pi) + a_{12}^2 = 0 \rightarrow$
 $\pi^2 + (-a_{11} - a_{22})\pi + a_{11}a_{22} + a_{12}^2 = 0 \rightarrow$ Diskriminante: $(a_{11} + a_{22})^2 - 4a_{11}a_{22} - 4a_{12}^2 = (a_{11} - a_{22})^2 - 4a_{12}^2 > 0$ Insgesamt gilt also, falls $(a_{11} - a_{22})^2 < 4a_{12}^2$ dann komplexe Lösung

- Es gilt $F = (M_r^{-1})^t E (M_l^{-1})$ (112)

4.6 F-MATRIX BERECHNEN UND RANSAC

Ziel: Gegeben seien n korrespondierende Punkte zwischen zwei Bildern in Epipolargeometrie. In folgenden Abschnitt wird gezeigt, wie daraus durch den sog. 8-Punkte Algorithmus die Fundamental Matrix berechnet werden kann, danach wird darauf eingegangen, wie fehlerhaft zugeordnete Punktpaare mit dem sog. Ransac Algorithmus eliminiert werden können.

4.6.1 DER 8-PUNKTE ALGORITHMUS

Gegen seien korrespondierende Punktpaare (keine Outlier!), **die nicht auf einer Ebene** liegen und keine fehlerhaften Zuordnungen enthalten. Die Berechnung der Fundamentalmatrix kann dann wie folgt untergliedert werden:

1. **Schritt:** Lineares homogenes Gleichungssystem aufstellen
2. **Schritt:** Gleichungssystem in Matrix Vektor Darstellung überführen
3. **Schritt:** Gleichungssystem mit SVD lösen
4. **Schritt:** Für Lösung mit SVD Eigenschaft einer F Matrix erzwingen

Im Folgenden wird auf die einzelnen Schritte eingegangen:

1. Schritt (Lineares homogenes Gleichungssystem aufstellen)

Jedes Paar $(p_R^{(i)}, p_L^{(i)})$ korrespondierender Punkte gibt eine Gleichung der Form

$$(u_R^{(i)}, v_R^{(i)}, 1) F (u_L^{(i)}, v_L^{(i)}, 1)^t = 0 \quad (113)$$

Also insgesamt bei n Punktpaaren

$$\begin{aligned} u_r^{(1)} u_l^{(1)} f_{11} + u_r^{(1)} v_l^{(1)} f_{12} + u_r^{(1)} f_{13} + v_r^{(1)} u_l^{(1)} f_{21} + v_r^{(1)} v_l^{(1)} f_{22} + v_r^{(1)} f_{23} + u_l^{(1)} f_{31} + v_l^{(1)} f_{32} + f_{33} &= 0 \\ u_r^{(2)} u_l^{(2)} f_{11} + u_r^{(2)} v_l^{(2)} f_{12} + u_r^{(2)} f_{13} + v_r^{(2)} u_l^{(2)} f_{21} + v_r^{(2)} v_l^{(2)} f_{22} + v_r^{(2)} f_{23} + u_l^{(2)} f_{31} + v_l^{(2)} f_{32} + f_{33} &= 0 \\ &(\dots) \\ u_r^{(n)} u_l^{(n)} f_{11} + u_r^{(n)} v_l^{(n)} f_{12} + u_r^{(n)} f_{13} + v_r^{(n)} u_l^{(n)} f_{21} + v_r^{(n)} v_l^{(n)} f_{22} + v_r^{(n)} f_{23} + u_l^{(n)} f_{31} + v_l^{(n)} f_{32} + f_{33} &= 0 \end{aligned} \quad (114)$$

Das Gleichungssystem ist linear, homogen und für $n \geq 8$ Punkte lösbar.

2. Schritt (Gleichungssystem in Matrix Vektor Darstellung überführen)

Das Gleichungssystem (114) lässt sich in Matrix/Vektor Darstellung überführen durch

$$\underbrace{\begin{pmatrix} u_r^{(1)} u_l^{(1)} & u_r^{(1)} v_l^{(1)} & u_r^{(1)} & v_r^{(1)} u_l^{(1)} & v_r^{(1)} v_l^{(1)} & v_r^{(1)} & u_l^{(1)} & v_l^{(1)} & 1 \\ \vdots & & & \vdots & & & & & \vdots \\ u_r^{(n)} u_l^{(n)} & u_r^{(n)} v_l^{(n)} & u_r^{(n)} & v_r^{(n)} u_l^{(n)} & v_r^{(n)} v_l^{(n)} & v_r^{(n)} & u_l^{(n)} & v_l^{(n)} & 1 \end{pmatrix}}_{:=A} \begin{pmatrix} f_{11} \\ f_{12} \\ \vdots \\ f_{32} \\ \underbrace{f_{33}}_{:=x} \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} \quad (115)$$

Und damit

$$Ax = 0 \quad (116)$$

3. Schritt (Gleichungssystem mit SVD lösen)

Für ≥ 8 Punkte folgt jetzt mit Singulärwertzerlegung

$$A = UDV^t \quad (117)$$

und somit ist der Spaltenvektor v von V der zum kleinsten Singulärwert korrespondiert², eine Lösung für $A \underbrace{x}_{:=v} = 0$. Die Lösung v ist nur bis auf einen skalaren Faktor eindeutig.

4. Schritt (Für Lösung mit SVD Eigenschaft einer F Matrix erzwingen)

Setze also für die gesuchte Fundamentalmatrix in erster Näherung

$$F^{(approx)} = \begin{pmatrix} v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 \\ v_7 & v_8 & v_9 \end{pmatrix} \quad (118)$$

² **Wiederholung** : Lösen eines gegebenen homogenen linearen Gleichungssystems $Ax = 0$ durch die Singulärwertzerlegung (SVD) (vgl. **Fehler! Verweisquelle konnte nicht gefunden werden.**): Die Matrix A lässt sich ausdrücken durch

$$A = \underbrace{U}_{m \times m} \underbrace{D}_{m \times n} \underbrace{V^t}_{n \times n} \quad \text{mit} \quad A = \underbrace{\begin{pmatrix} u_{0,0} & \dots & u_{0,m-1} \\ \vdots & & \vdots \\ u_{m-1,0} & \dots & u_{m-1,m-1} \end{pmatrix}}_{\substack{\text{Spalten zueinander} \\ \text{orthogonale Einheitsvektoren}}} \underbrace{\begin{pmatrix} \sigma_0 & \dots & 0 \\ \vdots & & \vdots \\ 0 & \dots & \sigma_{n-1} \end{pmatrix}}_{\substack{\sigma_0 \geq \dots \geq \sigma_{n-1} \geq 0 \\ \text{hei\ss en Singul\ddot arwerte}}} \underbrace{\begin{pmatrix} v_{0,0} & \dots & v_{0,n-1} \\ \vdots & & \vdots \\ v_{n-1,0} & \dots & v_{n-1,n-1} \end{pmatrix}}_{\substack{\text{Spalten zueinander} \\ \text{orthogonale Einheitsvektoren}}}^t$$

Sei jetzt v der Spaltenvektor von V , der zum kleinsten Singulärwert σ_{n-1} korrespondiert. Dann ist v eine Lösung für $Ax = 0$. Die Lösung ist aber nur bis auf einen skalaren Faktor eindeutig (falls x Lösung, dann ist auch $s \cdot x$ eine Lösung).

Mit Kap. 4.5.5 muss für jede Fundamentalmatrix ein Eigenwert (und damit ein Singulärwert) Null sein. Im Allgemeinen hat die in (118) gefundene Matrix $F^{(approx)}$ aufgrund von Überbestimmung und Messfehlern jedoch nicht diese Eigenschaft. Setze deshalb den kleinsten Singulärwert von $F^{(approx)} = UDV^t$ auf Null, und erzwingen so die endgültige Fundamentalmatrix F , also

$$F^{(approx)} = U \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{pmatrix} V^t \Rightarrow F = U \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{pmatrix} V^t \quad (119)$$

4.6.1.1 FAZIT UND AUSBLICK

Der gezeigte **8-Punkte** ist robust ein performant, solange die Punkte nicht auf einer Ebene liegen.

Liegen die Punktkorrespondenzen auf einer Ebene, so kann die Fundamentalmatrix F über eine **Homographie** (also die Abbildung einer Ebene auf eine Ebene) geschätzt werden kann. Es kann gezeigt werden, dass hierzu nur 4 Punkte benötigt werden.

Schließlich gibt es noch den **5-Algorithmus** (vgl. [3]). Hier entfällt die Unterscheidung in planar/nichtplanar, jedoch ist dieser Algorithmus deutlich aufwändiger zu implementieren und auch weniger performant.

4.6.2 RANSAC

Dieser Abschnitt zeigt, wie fehlerhaft zugeordnete Punkte robust detektiert und für die weiteren Berechnungen ausgeschlossen werden können.

4.6.2.1 BEGRIFFE (OUTLIER, INLIER, MODELLGLEICHUNG)

Die nach dem Feature Matching gefundenen korrespondierenden Punkte enthalten oft einen signifikanten Anteil an fehlerhafte Zuordnungen. Diese fehlerhaften, nicht zueinander korrespondierenden Punkte heißen im Folgenden **outlier**, die richtigen Zuordnungen heißen **inlier**.

Um die Punkte in outlier und inlier trennen zu können, muss die Relation der Punkte zueinander bekannt sein. Diese Zuordnung heißt im folgenden **Modellgleichung**.

4.6.2.2 DER RANSAC ALGORITHMUS

Im diesem Abschnitt wird der Ransac Algorithmus vorgestellt, der aus einer gegebenen Menge Q von Punktpaaren outlier herausfiltern kann.

Im Folgenden Abschnitt wird dies zuerst exemplarisch für eine Gerade als Modellgleichung vorgestellt. Im zweiten Abschnitt wird der Algorithmus dann auf die oben formulierte outlier Detektion für ein Stereobildpaar angewendet.

RANSAC für eine Geradengleichung

Gegeben seien n Punktpaare, die über eine Geradengleichung als Modellgleichung in Relation zueinander stehen sollten (siehe Abbildung 29). **Gesucht** ist die Menge der outlier.

Idee: Die Kernidee des Algorithmus basiert darin, in mehreren Durchläufen jeweils mit sowenig zufällig gewählten Punkten wie unbedingt nötig, **(a.)** die gesuchte Geradengleichung zu bestimmen (also mit 2 Punkten) und **(b.)** dann zu überprüfen, ob die so gefundene Geradengleichung für eine möglichst große Anzahl an Punktpaaren erfüllt ist. **(c.)** Ist diese Anzahl größer als bei allen vorherigen Durchläufen, so werden die so detektierten inlier und outlier abgespeichert, ansonsten verworfen.

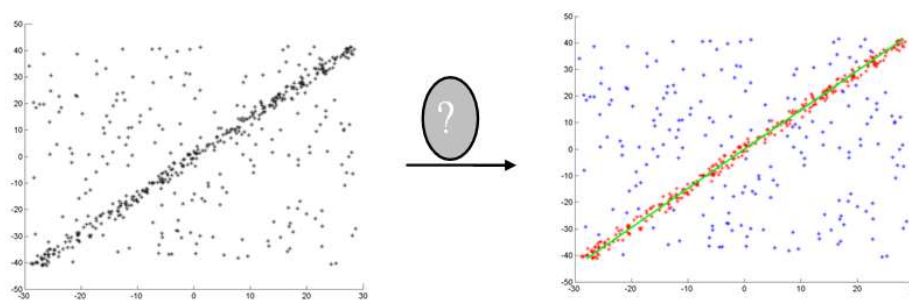


Abbildung 29: Quelle Wikipedia. Die Abbildung zeigt das Ergebnis des RANSAC Algorithmus angewendet auf eine Gerade als Modellgleichung

Die Anzahl der Durchläufe kann entweder einem festen Wert entsprechen, oder sich an statistischen Abbruchkriterien richten. Insgesamt lässt sich folgender Algorithmus formulieren:

Algorithmus RANSAC für Geradengleichung

Input: Menge Q von Punktpaaren

Für jede Iteration k

- 1.) Wähle zufällig eine Teilmenge A_k aus Q . Die Anzahl von A_k sei so groß, wie minimal für die Berechnung der Geradengleichung G_k benötigt wird, also 2
- 2.) Berechne von G_k für die Menge A_k
- 3.) Verwende G_k um für einen gegebenen Schwellwert thr alle Elemente aus Q auf *consensus* zu überprüfen. Also

$$L_k := \{(c, \tilde{c}) \in Q : \|G_k(c) - \tilde{c}\| < thr\}$$

- 4.) Set $L_{\max} := L_k$ if $|L_k| > |L_{\max}|$

Output: Return $L_{\max}$

RANSAC für die F Matrix

Gegeben seien n Punktpaare, die über die F-Matrix als Modellgleichung in Relation zueinander stehen sollten (siehe Abbildung 30). **Gesucht** ist die Menge der outlier.

Algorithmus RANSAC für F-Matrix

Input: Menge Q von korrespondierenden Pixeln

Für jede Iteration k

- 1.) Wähle zufällig eine Teilmenge A_k aus Q . Die Anzahl von A_k sei so groß, wie minimal für die Berechnung der Fundamentalmatrix F_k benötigt wird, (also 8 bei 8-Punkte-Alg. / 4 bei Homography / 5 bei 5-Punkte Algorithmus)
- 2.) Berechne von F_k für die Menge A_k
- 3.) Verwende F_k um für einen gegebenen Schwellwert thr alle Elemente aus Q auf consensus zu überprüfen. Also

$$L_k := \{(c, \tilde{c}) \in Q : \|\tilde{c} F_k c\| < thr\}$$

- 4.) Set $L_{\max} := L_k$ if $|L_k| > |L_{\max}|$

Output: Return $L_{\max}$

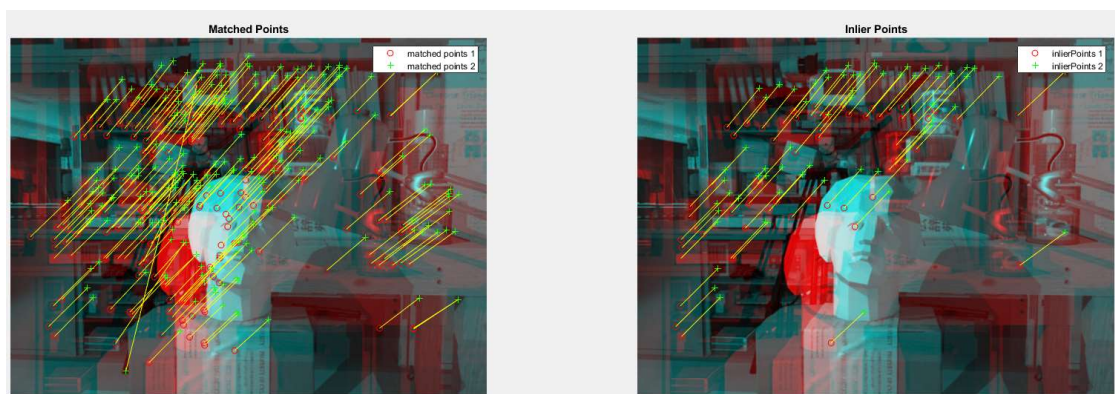


Abbildung 30: Übereinandergelagertes Stereopaar (a.) Ergebnis nach dem Feature Matching (b.) Ergebnis nach der Outlier Detection mit Ransac

4.7 POSE BERECHNEN UND TRIANGULATION

Ziel: Im letzten Kapitel wurde gezeigt, wie die E Matrix berechnet werden kann. In diesem Kapitel wird jetzt gezeigt, wie daraus die pose rekonstruiert werden kann. Das Ergebnis kann hierbei nur bis auf einen skalaren Faktor eindeutig berechnet werden (vgl. hierzu Anmerkung zu Formel (117) und Kap. 2.7.5).

4.7.1 POSE BERECHNEN

Im Folgenden ist die **Essentielle Matrix** E sowie mindestens ein korrespondierendes Punktepaar (p, q) gegeben. **Gesucht** ist die pose.

4.7.1.1 ALGORITHMUS (RECONSTRUCT POSE)

Der Algorithmus lässt sich wie folgt untergliedern (vgl. auch [3] S'258f):

Algorithmus (Reconstruct pose)

Schritt 1: Berechne 4 mögliche Lösungen durch adaptierte Singulärwertzerlegung
Schritt 2: Berechne daraus die einzig geom. mögl. Lösung durch Plausibilitätstest

Schritt 1: (Berechne vier mögliche Lösungen)

Setze mit $\hat{E} = UDV^t$ und (107)

$$\hat{E} = \hat{R}\hat{S} = \underbrace{UW^{-1}V^t}_{\hat{R}} \underbrace{VWDV^t}_{\hat{S}} \quad \text{mit} \quad W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (120)$$

Dann ist (120) eine Lösung. In ähnlicher Weise lassen sich insgesamt vier Lösungen konstruieren:

$$\underbrace{\left(\underbrace{UW^{-1}V^t}_{\hat{R}} ; \underbrace{VWDV^t}_{\hat{S}} \right)}_{\text{Fall 1}} ; \underbrace{\left(\underbrace{UW^{-1}V^t}_{\hat{R}} ; \underbrace{-VWDV^t}_{-\hat{S}} \right)}_{\text{Fall 2}} ; \underbrace{\left(\underbrace{UWV^t}_{\hat{R}} ; \underbrace{VW^{-1}DV^t}_{\hat{S}} \right)}_{\text{Fall 3}} ; \underbrace{\left(\underbrace{UWV^t}_{\hat{R}} ; \underbrace{-VW^{-1}DV^t}_{-\hat{S}} \right)}_{\text{Fall 4}} \quad (121)$$

Beweisskizze (Fall 1): Wenn $(\hat{R}; \hat{S})$ eine Lösung ist, müssen in (120) folgende Eigenschaften gelten

- a.) $\hat{E}$ lässt sich wie oben beschrieben aufspalten
- b.) $\hat{R}$ hat die Eigenschaften einer Rotation (122)
- c.) $\hat{S}$ hat die Struktur der Kreuzproduktdarstellung

zu a.) Mit Anwendung der Singulärwertzerlegung folgt durch „neutrale Ergänzung“ [3] p'257f

$$\hat{E} = UDV^t = UW^{-1} \underbrace{V^t V W}_{I} DV^t \quad (123)$$

Zu b.) Da U, W, V orthogonale Matrizen, ist also auch $\hat{R} = UW^{-1}V^t$ orthogonal. Falls $\det(\hat{R}) = -1$ gelten sollte, so darf o.B.d.A. das Vorzeichen von $\hat{E}$ negiert werden (da nach

Kap. 4.6.1 die E-Matrix nur bis auf einen skalaren Faktor eindeutig berechnet werden kann). Die erneute Berechnung von in $\hat{R}$ (120) wird dann zeigen, dass jetzt $\det(\hat{R}) = \det(UW^{-1}V^t) = 1$ gilt.

Zu c.) Hier muss gezeigt werden, dass $\hat{S}$ nach (9) einer antisymmetrischen Matrix entspricht, d.h.

$$\hat{S}^t = -\hat{S} \Rightarrow (VWDV^t)^t = -VWDV^t \quad (124)$$

Dies gilt mit

$$(WD)^t = \left(\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \sigma & 0 & 0 \\ 0 & \sigma & 0 \\ 0 & 0 & 0 \end{pmatrix} \right)^t = \begin{pmatrix} 0 & -\sigma & 0 \\ \sigma & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}^t = \begin{pmatrix} 0 & \sigma & 0 \\ -\sigma & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = -WD \quad (125)$$

woraus dann insgesamt folgt

$$\hat{S} = (VWDV^t)^t = \underbrace{(V^t)^t}_V \underbrace{(WD)^t}_{-WD} V^t = -VWDV^t \quad (126)$$

Beweisskizze (Fall 2): $(\hat{R}; -\hat{S})$ muss eine weitere Lösung sein, da nach Kap. 4.6.1 die bestimmte E-Matrix $\hat{E}$ nur bis auf einen skalaren Faktor bekannt ist und folglich auch $-\hat{S}$ eine Lösung für $-\hat{E}$ ist.

Beweisskizze (Fall 3-4): Außerdem ergibt sich durch Nachrechnen, dass $((\tilde{R}; \tilde{S}); (\tilde{R}; -\tilde{S}))$ zwei weitere Lösungen sind, falls $W := W^{-1}$ gesetzt wird.

Insgesamt gibt es so vier Möglichkeiten:

$$(\hat{R}, +\hat{S}); (\hat{R}, -\hat{S}); (\tilde{R}, +\tilde{S}); (\tilde{R}, -\tilde{S}) \quad \square$$

2. Schritt (Berechnung pose durch Plausibilitätstest)

Verwende korrespondierende Punkte, sowie die vier möglichen Kombinationen

$$(\hat{R}, +\hat{S}); (\hat{R}, -\hat{S}); (\tilde{R}, +\tilde{S}); (\tilde{R}, -\tilde{S}) \quad (127)$$

und rekonstruiere 3D Koordinaten mit der im folgenden Kap. 4.7.2 beschriebenen Methode für das rechte und linke Bild, also

$$P_l := (X_l, Y_l, Z_l) \quad \text{und} \quad P_r := (X_r, Y_r, Z_r) \quad (128)$$

Nur für **eine der vier möglichen Lösungen** wird $Z_l > 0$ und $Z_r > 0$ gelten [1].

4.7.2 TRIANGULATION

Gegeben seien jetzt Interne und externe Kameraparameter, sowie ein korrespondierendes Punktepaar $(p_i; q_i)$ (in Kamerakoordinaten). **Gesucht** ist der aus den zurückprojizierten Strahlen berechnete 3D Punkt.

4.7.2.1 BEGRIFF (STEREOTRIANGULATION)

Wird der minimale Abstand der zurückprojizierten Strahlen zweier korrespondierender Punkte berechnet, so heißt dieses Verfahren **Stereotriangulation**. Im Allgemeinen sind die Strahlen aufgrund von Rundungseffekten, Messfehlern oder Kalibrierfehlern etc. windschief.

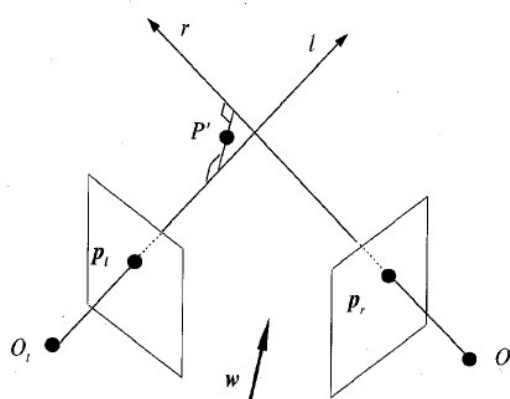


Abbildung 31: Triangulation mit zwei sich nicht schneidenden Strahlen. Im Allgemeinen sind die Strahlen aufgrund von Rundungseffekten, Messfehlern oder Kalibrierfehlern etc. windschief.

4.7.2.2 ALGORITHMUS (STEREOTRIANGULATION)

Der Algorithmus geht kann in folgende zwei Schritte unterteilt werden:

Algorithmus (Triang)

Input: Rotation R, Translation T, korresp. Punkte p1, p2

Schritt 1: Gleichungssystem (lin. und inhom.) berechnet aus Sichtstrahlen Vektoren

Schritt 2: 3D Punkt lässt sich dann aus diesen Vektoren berechnen

Output: Rekonstruierter 3D Punkt P

1. Schritt: (Lineares inhomogenes Gleichungssystem)

Sei

$$l := a p_l \quad (129)$$

der Strahl durch O_l und p_l . Und sei

$$r := B + bRp_r \quad (130)$$

der Strahl durch O_r und p_r beschrieben im linken Kamerakoordinatensystem. Bezeichne mit

$$w := p_l \times Rp_r \quad (131)$$

einen Vektor der senkrecht auf beiden steht. Dann muss mit $a_0, b_0, c_0 \in \mathbb{R}$

$$a_0 p_l - (B + b_0 Rp_r) + c_0 w = 0 \quad (132)$$

gelten (siehe Abbildung 32).

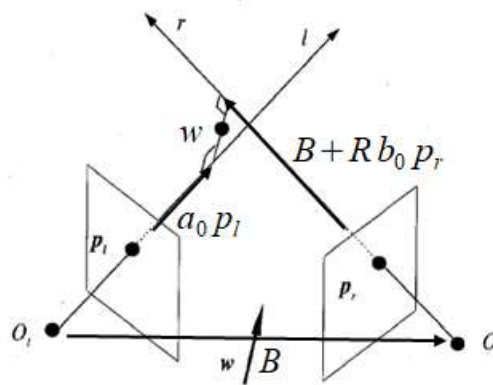


Abbildung 32: Quelle (modifiziert) [1] Triangulation mit zwei sich nicht schneidenden Strahlen

Lösung

Durch Umformen der Gleichung (132) folgt

$$a_0 p_l - (b_0 Rp_r) + c_0 (p_l \times Rp_r) = B \quad (133)$$

Durch Lösen dieses inhomogenen, linearen Gleichungssystems lassen sich die Unbekannten a_0, b_0, c_0 bestimmen.

2. Schritt (3D Punkt berechnen)

Der gesuchte Punkt P' ergibt sich dann durch Vektoraddition wie folgt (vgl. Abbildung 32)

$$P' = a_0 p_r + \frac{1}{2} c_0 (p_l \times Rp_r) \quad (134)$$

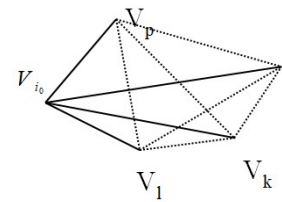
4.7.3 FAZIT UND AUSBLICK

In diesem Kapitel wurde gezeigt, wie aus einer gegebenen E Matrix die pose rekonstruiert werden kann. Die 4 möglichen Lösungen sind nur bis auf einen skalaren Faktor eindeutig; Die einzig geometrisch sinnvolle Lösung ergibt sich dann durch Plausibilitätsüberprüfung.

Das im zweiten Teil eingeführte und für die Plausibilitätsüberprüfung verwendete Verfahren kann auch an anderer Stelle zum Einsatz kommen, beispielsweise wenn ein dichtes 3D Modell berechnet werden soll, das Stereobildpaar jedoch nicht in Standardgeometrie vorliegt.

4.8 BUNDLE ADJUSTMENT

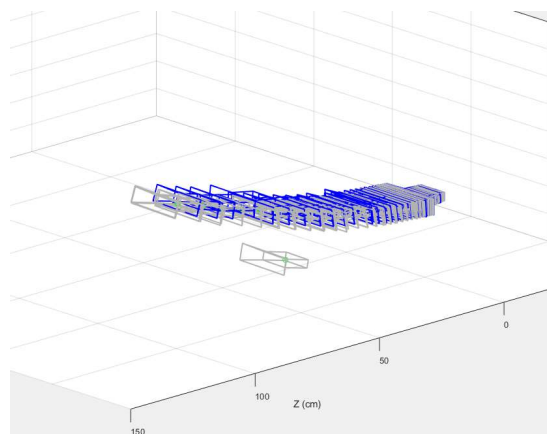
Ziel: In den letzten Kapiteln wurde gezeigt, wie eine Trajektorie aus jeweils einem Bildpaar durch Berechnung der E-Matrix und anschließender Rekonstruktion der Rotation und Translation bestimmt werden konnte. Ziel dieses Kapitels ist es, die Genauigkeit dieser Trajektorie zu verbessern, indem jetzt mehrere Bilder zueinander in Abhängigkeit gebracht werden und ein gemeinsamer Fehler zwischen all diesen Bildern minimiert wird.



4.8.1 PROBLEMSTELLUNG

Gegeben seien im Folgenden also n Bilder sowie die Punktkorrespondenzen zwischen diesen Bildern³. Die innere Kalibrierung der Bilder sei bekannt⁴. Außerdem seien die externen Kameraparameter relativ zu einem Weltkoordinatensystem näherungsweise gegeben.

Gesucht ist dann eine Verbesserung der externen Kalibrierparameter. **Der Ansatz** hierzu ist, dass alle Punktkorrespondenzen zwischen allen Bildern in Relation zueinander gebracht werden und so ein gemeinsamer Fehler minimiert wird.



³ O.B.d.A. kann wiederum angenommen werden, dass die Punktkorrespondenzen in Kamerakoordinaten vorliegen.

⁴ Eine Verallgemeinerung auf nur näherungsweise bekannte innere Kameraparameter wäre leicht zu realisieren

Abbildung 33: Die Abbildung zeigt eine aus einer Bildsequenz geschätzte Kameratrajektorie. **(a.)** vor bundle Adjustment (grau) **(b.)** nach Bundle Adjustment (blau)

4.8.2 BEGRIFFE UND DEFINITIONEN

4.8.2.1 DEFINITION (VIEWPOINT, MASTERVIEWPOINT)

Im Folgenden heie der i -te Kamerastandpunkt **Viewpoint** und der das Weltkoordinatensystem definierende Standpunkt heit **Masterviewpoint** V_{i_0} .

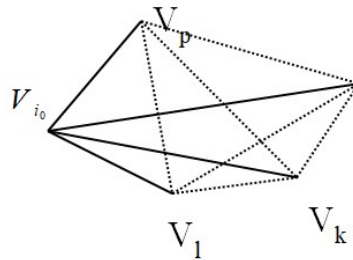


Abbildung 34: Die Abbildung zeigt die Viewpoints und deren Relation zueinander: Ist die Transformation $M^{(i_0)}$ von jedem Viewpoint V_i zum Masterviewpoint V_{i_0} bekannt, so lsst sich auch die Transformation zu jedem beliebigen Viewpoint durch $M^{(ij)} = \left(M^{(j_{i_0})}\right)^{-1} M^{(i_{i_0})}$ konstruieren.

4.8.2.2 BEGRIFF (KORRESPONDIERENDE PUNKTE)

Weiter seien korrespondierende Punkte zwischen V_i und V_j gegeben durch

$$\begin{aligned} p^{(ij)} &:= \left(\left(u_0^{(ij)}, v_0^{(ij)} \right), \dots, \left(u_m^{(ij)}, v_m^{(ij)} \right) \right) \\ p^{(ji)} &:= \left(\left(u_0^{(ji)}, v_0^{(ji)} \right), \dots, \left(u_m^{(ji)}, v_m^{(ji)} \right) \right) \end{aligned} \quad (135)$$

Dann heien $p^{(ij)}$ bzw. $p^{(ji)}$ **Vektoren der korrespondierenden Punkte** zwischen Viewpoint V_i und V_j .

4.8.2.3 BEGRIFF (TRANSFORMATION BER MASTERVIEWPOINT)

Seien zwei Viewpoints V_i und V_j gegeben, sowie die Rotation $R^{(ij)}$ und Translation $T^{(ij)}$ um V_i in V_j zu transformieren. Bezeichne mit

$$M^{(ij)} := \begin{pmatrix} R^{(ij)} & T^{(ij)} \\ 0 & 1 \end{pmatrix} \quad (136)$$

die sich daraus ergebende homogene Matrix. Dann lässt sich $M^{(ij)}$ auch als Transformation über den Masterviewpoint V_{i_0} ausdrücken durch

$$M^{(ij)} = M^{(i_0j)} M^{(i_0)} = \left(M^{(j_0)} \right)^{-1} M^{(i_0)} \quad (137)$$

Und heißt **Transformation über den Masterviewpoint**.

4.8.2.4 FOLGERUNG (ANZAHL DER FREIHEITSGRADE)

Um jeden Viewpoint V_i in einen anderen Viewpoint V_j transformieren zu können, sind unter Verwendung von (137)

$$|DOF| = (n-1)(3+3) \quad (138)$$

Unbekannte (=degrees of freedom, **DOG**) zu bestimmen.

4.8.2.5 DEFINITION (VEKTOR DER EXTERNEN KAMERAPARAMETER)

Setze

$$\lambda_i = \left(\alpha_i, \beta_i, \gamma_i, T_i^{(x)}, T_i^{(y)}, T_i^{(z)} \right) \quad (139)$$

um die in (137) dargestellten homogenen Matrizen $M^{(i_0)}$ aus ihrer jeweiligen parametrisierten Form zu berechnen. Setze weiter

$$\lambda = (\lambda_1, \dots, \lambda_n) \quad (140)$$

und bezeichne λ mit **Vektor der externen Kameraparameter**.

4.8.3 FEHLERMODELLIERUNG

Bezeichnet weiter $I := \{ \dots, (i, j), \dots \}$ die Menge der Links zwischen allen Viewpoints, dann ist

$$\underbrace{\sum_{(i,j) \in I} \sum_{k=0}^{|P^{(i,j)}|-1} \left\| \text{proj} \left(\underbrace{\text{triang} \left(p_k^{(ij)}, p_k^{(ji)}, M^{(ji)} \right)}_{P_k^{(ij)}} \right) - p_k^{(ij)} \right\|^2}_{\Phi(\lambda)} \rightarrow \min \quad (141)$$

ein **Fehlermodell**, welches bei gegebenen korrespondierenden Punkten seinen Fehler also in Abhängigkeit der relativen pose der Kameras ändert. Hierbei handelt es sich um ein nichtlineares Fehlermodell mit

$$|DOF| = (n-1)(3+3) \quad (142)$$

Freiheitsgraden (vgl. Abbildung 35). Die Berechnung erfolgt iterativ durch den **Levenberg-Marquardt Algorithmus** (vgl. Kap. 2.8). Hierzu müssen für λ gute Startwerte $\lambda^{(s)}$ gegeben sein.

Erläuterung

In (141) wird durch **proj** ein mit Kap. 4.7.2.2 (Stereotriangulation) rekonstruierter 3D Punkt in das Kamerakoordinatensystem i zurückprojiziert und dort mit dem korrespondierenden Punkt verglichen.

Um die in Kap. 2.3.2 eingeführten Rotationseigenschaften $\det(R_i)$ und $R_i^{-1} = R_i^t$ einzuhalten, werden in (141) nicht die 9 Einträge der Rotationen R_i direkt, sondern deren **parametrisierte Formen** λ_i optimiert. Insgesamt ergibt sich dann

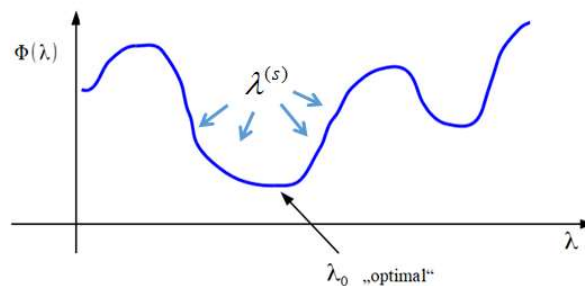


Abbildung 35: Die Abbildung zeigt eine schematische Darstellung des Fehlerverlaufes. Sind gute Startwerte bekannt („d.h. der Fehler $\Phi(\lambda^{(s)})$ muss im selben Tal liegen wie $\Phi(\lambda_0)$ “), so lässt sich der optimale Parametervektor λ_0 mit Hilfe des Levenberg-Marquardt Algorithmus bestimmen

Folgerung

Ist der in (140) eingeführte Vektor λ der externen Kameraparameter „optimal“ bestimmt, dann ist (141) minimal.

4.8.4 ALGORITHMUS (BUNDLE ADJUSTMENT)

Algorithmus (Bundle Adjustment)

Input: Punktkorrespondenzen zwischen n Bildern, internen Kameraparameter, ungefähr bekannte externen Kameraparameter $\lambda^{(s)}$

- Optimiere $\lambda^{(s)}$ in (141) durch **Levenberg-Marquardt**

Output: Optimierter Vektor λ_0 der externen Kameraparameter

4.9 MODIFIKATIONEN

4.9.1 REFINEMENT DURCH BEKANNTE STÜTZSTELLEN

Gegeben sei der Vektor der externen Kameraparameter λ , im i -ten Bild rekonstruierte 3D Punkte $\tilde{P}^{(i)}$ (beschrieben im n -ten Bild) sowie eine hierzu korrespondierende Landmarke $P^{(i_0)}$ (beschrieben im Masterviewpoint i_0). Ziel ist es jetzt

Dann bietet das folgende Fehlermodell die Möglichkeit

$$\left\| \underbrace{\left(M^{(1,i_0)} \cdot \dots \cdot M^{(n,n-1)} \tilde{P}^{(n)} \right)}_{\tilde{\Phi}_\lambda(\Delta)} - P^{(n)} \right\|^2 \rightarrow \min$$

4.9.2 WORKFLOWS

4.9.2.1 EGO MOTION AUTO

Sie wollen die Ego Motion eines Autos berechnen. Entwickeln Sie einen workflow in Pseudocode!

- W-KOS = eye(4);
- For all Images j
 - Img[j] = readImage
 - Img[j] = undistortImages
 - initTracker (img[j])
 - feat[j] = extractHarrisFeatures(img[j]);
 - for i = j : j+10
 - Img[i] = readImage
 - Img[i] = undistortImages
 - feat[i] = track(feat[j], img[i]);
 - F Matrix berechnen mit 8 Pkte Alg und Ransac
 - Daraus E Matrix mit Kameramatrix
 - Daraus Pose (rel zu img j)
 - vSet → add Features, i etc, pose
 - vSet → add Connection (j, i, pose)
- XYZ = Triangulate(tracks)
- BundleAdjustment
- Update vSet
- W-KOS = W-KOS * T_i2j
- XYZ = Triangulate(tracks) → gibt dünnes 3D Modell zu img j
- XYZ_w = W-KOS * XYZ → gibt dünnes 3D Modell zur welt
- Pose ins WeltKos transformieren

4.9.2.2 HAUSFASSADE

Sie wollen mit einer Spiegelreflexkamera ein 3D Modell (als Punktwolke) einer Hausfassade generieren. Entwickeln Sie einen workflow in Pseudocode!

- W-KOS = img 0;
- For all Images i
 - readImage
 - undistortImages
 - extractSurfFeatures();
 - For all Images j
 - F Matrix berechnen zw i und j
 - Daraus E Matrix mit Kameramatrix
 - Daraus Pose (zw i und j)
 - Daraus Pose zu img 0
 - vSet → add Features, id etc
 - vSet → add Connection (0, i, pose)
- XYZ = Triangulate(tracks)
- BundleAdjustment
- Update vSet
- XYZ = Triangulate(tracks) ergibt dünnes 3D Modell
- Dann z.B. noch Vermascher

4.9.3 WORKFLOW 3 (STEREOKAMERA)

Sie wollen mit einem mobilen Stereokamerasystem eine 3D Punktwolke generieren (dessen Bildaufnahmefrequenz relativ hoch ist). Entwickeln Sie hierzu einen workflow in Pseudocode.

- Schritt 1:
 - Wie Aufgabe 1
- Schritt 2
 - % beide Bild
 - For all Images
 - Rectify
 - Disparity
 - Paarweises dichtes 3D Modell
 - Transformation über pose in Welt
 - end

5 Literatur

- [1] A. Trucco und E. Verri, Introductory Techniques for 3D Computer Vision, Prentice Hall, 1998.
- [2] T. Abmayr, F. Härtl, G. Hirzinger, D. Burschka und C. Fröhlich, „A Correlation based Targetfinder for Terrestrial Laserscanning,“ *Journal of applied Geodesy*, pp. 31-38, Sep. 2008.
- [3] A. Hartle und R. Zisserman, Multiple View Geometry in Computer Vision, Cambridge University Press, 2003.
- [4] J. Herzberger , K. Lingemann und A. Nüchter, Mobile Roboter, Springer, 2011.
- [5] G. Kaehler und A. Bradski, „Learning OpenCV,“ O'Reilly Verlag, 2008.
- [6] T. Luhmann, Nahbereichsphotogrammetrie, Herbert Wichmann Verlag, 2003.
- [7] T. Abmayr, „Calibration and Registration Framework for Multisensor Panoramic Color Scanning,“ Shaker Verlag, 2010.
- [8] P. Corke, Robotics, Vision and Control – Fundamental Algorithms in Matlab, Springer, 2011.
- [9] E. T. a. A. Verri, Introductory Techniques for 3D Computer Vision, Prentice Hall, 1998.

6 FAQs

6.1 MOBILE MAPPING

6.1.1 EINFÜHRUNG (WOCHE 1)

Zu Kap. 0.1.3

Wie lassen sich folgende Begriffe erläutern?

1. Intrinsischen und extrinsische Kameraparameter
2. Standardstereogeometrie
3. Levenberg Marquardt und Bündelblockausgleichung
4. Kamerakalibrierung nach Zhang
5. Disparitätsbild ("disparity")
6. SIFT
7. Epipolargeometrie berechnen
8. dense 3D Modell
9. sparse 3D Modell
10. Bildrektifizierung ("rectify images")
11. Triangulation ("triangulate")
12. Singulärwertzerlegung
13. RANSAC
14. Korrespondenzsuche
15. ply und pld Datenformat
16. Pose und Homogene Matrix

Zu Kap. 1

1. Wofür steht der Begriff Lokalisation?
2. Wofür steht der Begriff 3D Rekonstruktion?
3. Welche Komponenten hat ein Mobile Mapping System?

6.1.2 GEWEGUNG IM $\mathbb{R}^3$, KOORDINATENSYSTEME (WOCHE 2)

Zu Kap. 2.1, Kap. 2.2, Kap. 2.3, Kap. 2.4, Kap. 2.5

1. Wie ist das Skalarprodukt definiert?

2. Wie kann der Winkel zwischen zwei Vektoren berechnet werden?
3. Wie kann überprüft werden, ob zwei Vektoren rechtwinklig (=orthogonal) aufeinander stehen?
4. Wie ist das Vektorprodukt definiert?
5. Wie ist der Zusammenhang zwischen "rechter Hand" und Vektorprodukt?
6. Wie lautet die Matrix/Vektor Form des Vektorproduktes?
7. Welche Eigenschaften hat eine Rotation?
8. Wie lautet der Unterschied zwischen einer Rotation und einer Spiegelung? (nicht im Skript)
9. Gegeben sei eine 3 x3 Matrix. Wie kann überprüft werden, ob es sich um eine Rotation handelt?
10. Wie kann eine Translation und Rotation als homogene Matrix dargestellt werden?
11. Welche Vorteile hat das Rechnen mit homogenen Matrizen? (nicht im Skript)
12. Wie kann eine homogene Matrix als Koordinatensystem interpretiert werden?
13. Wie lässt sich aus einer homogenen Matrix der Koordinatenursprung, sowie die Achsen berechnen? (nicht im Skript)
14. Was versteht man unter einem affinen, orthogonalen und orientierungstreuen Koordinatensystem?
15. Was versteht man unter der RPY Darstellung einer Drehung?
16. Wie lassen sich aus der RPY Darstellung einer Drehung die drei Winkel zurückrechnen?
17. Welche anderen Darstellungen für Drehungen gibt es noch?

6.1.3 KAMERA-KALIBRIERUNG NACH ZHANG (WOCHE 3)

Zu Kap. 3.1, Kap. 3.2; Kap. 3.3

1. Welche Transformation ist gesucht?
2. Wie lässt sich eine Kamera einfach beschreiben?
3. Wofür steht der Begriff
 - Lochkameramodell?
 - Zentralprojektion
 - tangential- und radiale Verzeichnung?
 - externe und innere Kameraparameter?
 - Bild- Kamera- und Weltkoordinatensystem?
 - Brennweite?
 - Bildebene (reale)
 - normalisierte Bildebene
 - Bildhauptpunkt

- DOF
- 4. Wie lautet die Abbildung (inkl. Anzahl der DOF) nach dem Modell nach Zhang
 - von Weltkoordinaten in das Kamerakoordinatensystem
 - auf die Bildebene
 - wie wird die Verzeichnung korrigiert
 - in das Bildkoordinatensystem
- 5. Wie lautet die Gesamtabbildung
- 6. Wie kann das der Kalibrierung zugrundeliegende Fehlermodell beschrieben werden?
- 7. Wenn lauten die praktischen Schritte zur Durchführung einer Kalibrierung?
- 8. Was sind die inneren und externen Kameraparameter eines Stereokamerasystems (inkl. Anzahl DOF) ?

6.1.4 STANDARDSTEREOGEOMETRIE (WOCHE 4)

zu Kap. 4.1 (Einführung)

1. In welchen Ihnen bekannten Anwendungen wird 3D Rekonstruktion durch Kameras benötigt? (Internet)
2. Welches Ziel wird bei der 3D Rekonstruktion durch Kameras verfolgt?
3. Wie lauten die zwei Probleme der Stereobildverarbeitung?

zu Kap. 4.2 (Die Standardstereogeometrie)

1. Welche Modellannahmen werden an die Standardstereogeometrie gestellt?
2. Wie lautet das geometrische Grundprinzip zur 3D Rekonstruktion eines Punktes (bei vorliegender Standardstereogeometrie)? Wie lassen sich die Formeln herleiten?
3. Wie lässt sich der Begriff Disparität erläutern?
4. Was bedeutet Basisabstand?
5. Kann die Standardstereogeometrie in der Realität verwendet werden?

zu Kap. 4.2.5 (Konvergente Stereogeometrie)

1. Wie lauten die Unterschiede zwischen Standardstereogeometrie und konvergenter Stereogeometrie hinsichtlich ihrer Modellannahmen?
2. Kann die konvergente Stereogeometrie in der Realität verwendet werden?

6.1.5 KORRESPONDENZANALYSE I – DISPARITÄTSBILD (WOCHE 5)

zu Kap. 4.4.1, Kap. 4.4.2, Kap. 4.4.3

1. Was bedeutet Korrespondenzanalyse?
2. Wie lassen sich flächenbasierte Verfahren und wie lassen sich merkmalsbasierte Verfahren definieren?
3. Wie lässt sich Disparitätsbild definieren?
4. Welche Einschränkungen können die Suche nach Punktkorrespondenzen vereinfachen?
5. Wie lauten die Schritte beim Blockmatching Algorithmus?
6. Wie lässt sich der Begriff korrelationsbasiertes Matching erläutern?
7. Wie lassen sich die unterschiedlichen Ähnlichkeitsmetriken beschreiben?

6.1.6 KORRESP.-ANALYSE II - MERKMALSBASIERTE VERF. (WOCHE 6)

zu Kap. 4.4.4

1. Wie lautet der prinzipielle Ablauf und das Ziel bei der merkmalsbasierten Korrespondenzsuche?
2. Wie lassen sich folgende Begriffe erläutern:
 - Feature Point, Corner Point
 - Feature Point Deskriptor, Keypoint Descriptor
 - Feature Matching
 - Point Detection, Feature Point Extraction
 - Template Matching
3. Wie lässt sich der Movarec Operator erläutern?
4. Welche Alternativen zum Movarec Operator gibt es?
5. Wie lässt sich Template Matching erläutern? (z.B. als Pseudocode)?
6. Wie lauten die Schritte des SIFT Algorithmus?
 - (zu Schritt 1) Wie lassen sich die Teilschritte erläutern?
 - (zu Schritt 2) Wie werden die lokalen Orientierungen berechnet?
 - (zu Schritt 3) Wie werden beim SIFT Algorithmus die Deskriptoren berechnet?
 - (zu Schritt 4) Wie werden Deskriptoren miteinander verglichen?

6.1.7 EPIPOLARGEOMETRIE, E MATRIX UND F MATRIX (WOCHE 7)

zu Kap. 4.5.1, Kap. 4.5.3, Kap. 4.5.4, Kap. 4.5.5

1. Wie lauten die Modellannahmen für die Epipolargeometrie?
2. Wie lassen sich folgende Begriffe definieren?
 - Epipolarebene?
 - Epipolarlinie?

- Epipol?
- 3. Wie lautet das epipolar constraint?
- 4. Welche besondere Eigenschaft hat der Epipol bzgl. der Epipolarlinien?
- 5. Wie lässt sich ein Sichtstrahl der rechten Kamera (und damit im rechten KOS beschrieben) in das Koordinatensystem der linken Kamera transformieren?
- 6. Wie lautet die Komplanaritätsbedingung? Wie lässt sich die Komplanaritätsbedingung herleiten bzw. geometrisch erläutern?
- 7. Welchen Zusammenhang liefert die Essentielle Matrix?
- 8. Wie lässt sich diese aus der Komplanaritätsbedingung herleiten?
- 9. In welchem Koordinatensystem müssen die Daten für die E Matrix vorliegen?
- 10. In welchem Koordinatensystem müssen die Daten für die F Matrix vorliegen?
- 11. Welche Voraussetzung wird an die Kameras in Hinblick auf die Kalibrierung beim Aufstellen der F Matrix angenommen?
- 12. Wie wird vorgegangen, damit Bilder diese Voraussetzung erfüllen?
- 13. Welche Struktur hat die M Matrix (=Kameramatrix)? Was ist die Aufgabe der Kameramatrix?
- 14. Wie lautet der Zusammenhang zwischen E Matrix und F Matrix?
- 15. Wie können Sie testen, ob es sich um eine E-Matrix oder F-Matrix oder keine von beiden handelt?

6.1.8 WIEDERHOLUNG (WOCHE 8)

Bewegung im $\mathbb{R}^3$

1. Wie kann überprüft werden, ob zwei Vektoren rechtwinklig (=orthogonal) aufeinander stehen?
2. Wie kann ein Vektor konstruiert werden, der rechtwinklig auf zwei Vektoren steht?
3. Gegeben sei eine 3×3 Matrix. Wie kann überprüft werden, ob es sich um eine Rotation handelt?
4. Wie ist der Zusammenhang zwischen einem Koordinatensystem und einer Bewegung?

Das Kameramodell von Zhang

1. Welche Einheiten liegen jeweils dem Kamera- Welt- Bildkoordinatensystem zugrunde?
2. Wie lauten die Abbildungen von Weltkoordinaten in das Bildkoordinatensystem der Kamera nach dem Modell von Zhang
3. Wie wird eine Kamera nach dem Verfahren von Zhang kalibriert?
4. Was wird mit der Belichtungsautomatik eingestellt und hat das Auswirkungen auf die Kalibrierparameter?

Kalibrierung eines Stereokamerasystems

1. Wie viele innere Kameraparameter hat ein Stereokamerasystem?
2. Wie viele externe Kameraparameter hat ein Stereokamerasystem?

Die Standardstereogeometrie

1. Welche Modellannahmen werden an die Standardstereogeometrie gestellt?
2. Wie lautet hier das geometrische Grundprinzip zur 3D Rekonstruktion eines Punktes?
3. Wie lässt sich der Begriff Disparität erläutern?

Die Epipolargeometrie

1. Wie lauten die Modellannahmen für die Epipolargeometrie?
2. Wie lautet die Komplanaritätsbedingung?
3. Wie ist der Zusammenhang zwischen F Matrix und E Matrix?
4. Was sind notwendige Bedingungen für eine E Matrix bzw. F-Matrix?

Korrespondenzanalyse

1. Wann werden flächenbasierte Verfahren, wann merkmalsbasierte Verfahren benötigt?
2. Wie lauten die Schritte beim Blockmatching Algorithmus?
3. Welche Ähnlichkeitsmetriken können beim Blockmatching oder Template Matching verwendet werden?
4. Wie lautet der prinzipielle Ablauf bei der merkmalsbasierten Korrespondenzsuche?
5. Wie lauten die Schritte beim Template Matching Algorithmus?
6. Welche Vorteile und Nachteile hat SIFT im vgl. zum Template Matching?
7. in welche Teilschritte lässt sich SIFT unterteilen?

6.1.9 8-PUNKTE ALGORITHMUS UND RANSAC (WOCHE 9)

zu Kap 4.6.1

1. Was ist das Ziel des 8-Punkte Algorithmus?
2. (Schritt 1) Gegeben seien korr. Pixel. Wie lautet der Ansatz um daraus ein Gleichungssystem zum Berechnen der Fundamentalmatrix aufzustellen?
3. (Schritt 1) Welche Art von Gleichungssystem liegt dem 8 Punkte Algorithmus zugrunde? Linear/nichtlinear // Homogen/inhomogen?
4. (Schritt 2) Wie schaut das Gleichungssystem in Matrix Vektor Darstellung aus?

5. (Schritt 3) Wieviele korrespondierende Punkte (Pixel) werden minimal benötigt, um das Gleichungssystem zu lösen?
6. (Schritt 3: Wie lässt sich das Gleichungssystem mit Hilfe des SVD lösen (und damit der Parametervektor f berechnen) und wie lautet die Lösung in Pseudocode?
7. (Schritt 3) Wie lässt sich der Begriff Singulärwertzerlegung erläutern?
8. (Schritt 3) Wie lässt sich der Begriff kleinster Singulärwert erläutern?
9. (Schritt 4) Wie kann aus der gewonnenen 3×3 Matrix die F-Matrix F berechnet werden? Aufgrund von welcher Eigenschaft von F gilt dies?
10. Wie kann der 8 Punkte Algorithmus so modifiziert werden, dass daraus direkt die E Matrix berechnet wird?
11. Was sind mögliche Alternativen zum 8-Punkte Algorithmus und welche Vor- und Nachteile haben diese?

Zu Kap. 4.6.2

1. In Kombination mit welchem Algorithmus können beim 8-Punkte Algorithmus Ausreißer eliminiert werden?
2. Warum sind herkömmliche statistische Verfahren zur Elimination von Ausreißern meist ungenügend?
3. Wie lauten die Schritte des RANSAC Algorithmus, falls als Modellgleichung eine Geradengleichung gesucht ist?
4. Wie viele korr. Punkte werden minimal benötigt, falls als Modellgleichung eine Geradengleichung gesucht ist?
5. Wie lauten die Schritte des RANSAC Algorithmus, falls als Modellgleichung eine F-Matrix gesucht ist?
6. Wie viele korr. Punkte werden minimal benötigt, falls als Modellgleichung eine F-Matrix gesucht ist?

6.1.10 POSE BERECHNEN UND TRIANGULATION (10. WOCHE)

Zu Kap. 4.7.1

1. Wie gut lässt sich ein 3D Modell rekonstruieren, falls die E Matrix bekannt sind?
2. Wie lautet der Ansatz mit Hilfe der Singulärwertzerlegung, um die pose zu rekonstruieren?
 - Welche Eigenschaften muss hierbei die Matrix $\hat{R}$,
 - welche die Matrix $\hat{S}$ erfüllen?
3. Ist die Lösung eindeutig?

4. Wie wird vorgegangen um aus den vier mathematisch möglichen Lösungen die geometrisch einzig sinnvolle Möglichkeit zu bestimmen?

Zu Kap. 4.7.2

1. Wie lauten die Kernidee zur Rekonstruktion eines 3D Punktes, falls alle Kameraparameter bekannt sind? Wie heißt dieses Verfahren?
2. Schritt 1:
 - Wie lautet die Idee, die zum beschriebenen Gleichungssystem führt?
 - Wie lässt sich das Gleichungssystem formulieren und aufstellen?
 - Aus wie vielen Gleichungen besteht das Gleichungssystem? Ist es linear / nichtlinear / homogen / inhomogen / überbestimmt / nicht überbestimmt?
3. Schritt 2:
 - Nachdem das Gleichungssystem aus Schritt 1 aufgestellt und gelöst wurde: Wie lässt sich jetzt der gesuchte 3D Punkt berechnen?
4. Wofür kann der vorgestellte Algorithmus noch verwendet werden?

6.1.11 EINFÜHRUNG IN DAS BUNDLE ADJUSTMENT (11. WOCHE)

Zu Kap. 4.8

1. Wie lauten die Eingabedaten für das im Skript vorgestellte Bundle Adjustment? An welcher Stelle in Ihrem workflow (z.B. im gerade zu bearbeitenden Praktikum) würde er zum Einsatz kommen?
2. Was ist gesucht?
3. Wie lässt sich also "Bundle Adjustment" allgemein formulieren?
4. Wie lassen sich folgende Begriffe definieren?
 - korrespondierende Pixel
 - Viewpoint, Masterviewpoint
 - Transformation über Masterviewpoint
5. Warum wird die Transformation über den Masterviewpoint verwendet?
6. Wie viele Freiheitsgrade gibt es bei der Berechnung zu berücksichtigen?
7. Wie lautet das Fehlermodell?
8. Wofür wird der Levenberg-Marquardt Algorithmus benötigt?

6.1.12 BILDREKTIFIZIERUNG (12. WOCHE)

Zu Kap. 4.3

1. Was lässt sich unter dem Begriff Bildrektifizierung verstehen?
2. Welche Annahmen werden getroffen?
3. Wie können diese Annahmen für reale Bilder auch eingehalten werden?
4. Wie lautet die Kernidee beim Bildrektifizieren?
5. Welche Vorteile haben rektifizierte Kameras?
6. Zur Konstruktion der Rotationsmatrix: In welchen drei Schritten wird die Rotationsmatrix konstruiert?
7. Algorithmus (Rectification)
 - (Schritt 1) Wie lassen sich jetzt aus der so gewonnenen Matrix die Transformationen R_l und R_r berechnen?
 - (Schritt 2 und 3): Wie lassen sich jetzt so die Bildinhalte neu berechnen?

6.1.13 WORKFLOWS UND WIEDERHOLUNG (13. WOCHE)

Aufgabe (workflows)

Sie wollen die Ego Motion eines Autos berechnen. Entwickeln Sie einen workflow in Pseudocode!

1. Sie wollen mit einer Spiegelreflexkamera ein 3D Modell (als Punktwolke) einer Hausfassade generieren. Entwickeln Sie einen workflow in Pseudocode!
2. Sie wollen mit einem UAV ein 3D Modell einer Kirche generieren, indem Sie mehrere GPS Koordinaten anfliegen und von dort ein Photo auslösen. Generieren Sie einen workflow in Pseudocode!
3. Sie wollen mit einem mobilen Stereokamerasystem eine 3D Punktwolke generieren (dessen Bildaufnahmefrequenz relativ hoch ist). Entwickeln Sie hierzu einen workflow in Pseudocode.

Wiederholung

1. Rotation
2. parametrisierte Form einer Drehung. Wozu wird diese benötigt?
3. Zhang
4. E Matrix, F Matrix, Zusammenhang zwischen beiden
5. Epipolarebene, Epipolarlinie
6. Standardstereogeometrie
7. Bundle Adjustment
8. Shift, Surf

9. Feature Tracking
10. Ransac
11. Homographie
12. Epipol
13. Levenberg-Marquardt
14. density Map, SGM, Blockmatching
15. dichtes und dünnes 3D Modell
16. Arten der 3D Rekonstruktion
17. Rectify

7 Praktika

7.1 MOBILE MAPPING (MATLAB)

7.1.1 EINFÜHRUNG IN MATLAB (WOCHE 1)

7.1.1.1 BILDER EINLESEN

Ziel der Aufgabe ist es, ein Matlab Skript zu erstellen, so dass Sie das auf Moodle bereitgestellte Bild *HM_Karlstr.jpg* aus einer Datei einlesen und darstellen können. Geben Sie außerdem an, wie viele Zeilen, Spalten und Farben das Bild hat. Verwenden Sie hierzu die folgenden Funktionen:

- `imread`
- `size`
- `imshow`

7.1.1.2 BILD ANALYSIEREN UND AUFBEREITEN

Ziel ist es, das gegeben Bild zu manipulieren, indem bestimmte Farbwerte geändert werden.

Grauwertbild

- Laden Sie jetzt das in Aufgabe 1 genannte Bild mit *imread*
- wandeln Sie es mit *rgb2gray* in ein Graustufenbild
- Verwenden Sie den Matlab-Befehl *figure*, um zu bewirken, dass die Darstellung des Farbbildes nicht vom Grauwertbild überschrieben wird

Bildhistogramm

- Berechnen jetzt für das Grauwertbild das Bildhistogramm mit der Funktion *imhist*
- Stellen Sie dieses mit den Matlab-Funktionen *plot* und *bar* dar. Verwenden Sie wiederum den *figure* Befehl, um die Darstellungen nicht zu überschreiben.
- Berechnen Sie jetzt das Histogramm für das Farbbild und interpretieren bzw. dokumentieren Sie Ihre Ergebnisse als Kommentar (Zeichen %) im Programm Code.

7.1.1.3 BILD ALS MATRIX INTERPRETIEREN

- Verwenden Sie jetzt den Befehl *imresize*, um das Farbbild auf 10% seiner ursprünglichen Größe zu skalieren.
- Ein Bild lässt sich als eine Matrix der Dimension Bildbreite * Bildhöhe * Farbtiefe interpretieren (dies kann wieder mit der Matlab Funktion *size* abgefragt werden). Ziel der Aufgabe ist es,
 - in der Bildmitte, sowie
 - in den vier Ecke

das Grauwertbild, sowie das Farbbild jeweils mit der Farbe schwarz zu belegen. Implementieren Sie dies geeignet.

7.1.1.4 MIT BILDERN ARBEITEN

Ziel ist es, die 9 Bilder der auf Moodle bereitgestellten Bildsequenz PeterCorke_Traffic.zip (*.png) mit Hilfe einer *for* Schleife einzulesen und anzuzeigen. Laden Sie hierzu die Bilder in Ihr Arbeitsverzeichnis und implementieren Sie eine *FOR* Schleife mit 9 Durchläufen,

- generieren Sie sich mit dem *SPRINTF* Befehl die richtigen Filenamen zweier aufeinanderfolgender Bilder (vgl Matlab Hilfe bzw. >> help sprintf)
- *BEIPIEL*: `sprintf('000%d.png',3)` ergibt den Pfadnamen 0003.png
- laden Sie diese mit der *IMREAD* Methode
- Stellen Sie die Bilder mit der *imshow* Methode dar;
- Speichern Sie Ihre Bilder mit *IMWRITE* in Ihr Homeverzeichnis ab
- Stellen Sie die Bilder außerdem mit der *MONTAGE* Methode in einem Rechteckgitter dar
- implementieren Sie eine Verzögerung von 1 Sekunde zwischen zwei Schleifendurchläufen mit Hilfe des *PAUSE* Befehls (vgl. Matlab Hilfe bzw. >> help pause)
- Verwenden Sie *VIDEOWRITER* (unter Verwendung von *IM2FRAME* oder *GETFRAME*), um die Bildsequenz wieder als Video abzuspeichern. Setzen Sie die Parameter so, dass eine Framerate von 1 sek eingestellt ist.
- Verwenden Sie *IMPLAY*, um Ihre Bildsequenz als Video zu spielen

7.1.1.5 PUNKTWOLKEN DARSTELLEN

Gegeben sei eine Punktwolke PO (vgl. 'teapotOut.asc'). Laden Sie die Punktwolke mit *pcread* und stellen Sie diese mit *pcshow* dar.

7.1.2 KOORDINATENSYSTEMEN (WOCHE 2)

Erstellen Sie ein Skript zum Abarbeiten der folgenden Schritte:

7.1.2.1 ARBEITEN MIT KOORDINATENTRANSFORMATIONEN

- Definieren Sie sich die Orientierung eines Koordinatensystems mit *angles*=[0,0,0] (Methoden beispielsweise *eul2rotm*, vgl. Robotic Operating System Toolbox). Verwenden Sie dann *rotm2tform*, um die 3x3 Matrix in eine homogene Matrix zu überführen. Geben Sie das Ergebnis auf der Konsole aus.
- Definieren Sie sich die *Location* eines Koordinatensystems mit *t* = [0,0,0]. Verwenden Sie dann *trvec2tform*, um den 1x3 Vector in eine homogene Matrix zu überführen. Geben Sie das Ergebnis auf der Konsole aus.

- Berechnen Sie sich jetzt das Koordinatensystem aus *Verschiebung* und *Drehmatrix* durch Hintereinanderausführung der beiden berechneten homogenen Matrizen, wie in der Lehrveranstaltung eingeführt.
- Stellen Sie dieses Koordinatensystem unter Verwendung der Parameter '*Orientation*' und '*location*' grafisch dar (Methode *plotCamera*)⁵
- Verwenden Sie *hold on* um Ihren *plot* nicht zu überschreiben. Verwenden Sie *grid on*, um Gitterlinien zu zeichnen, sowie *axis equal*, um gleiche Achsskalierung zu bekommen

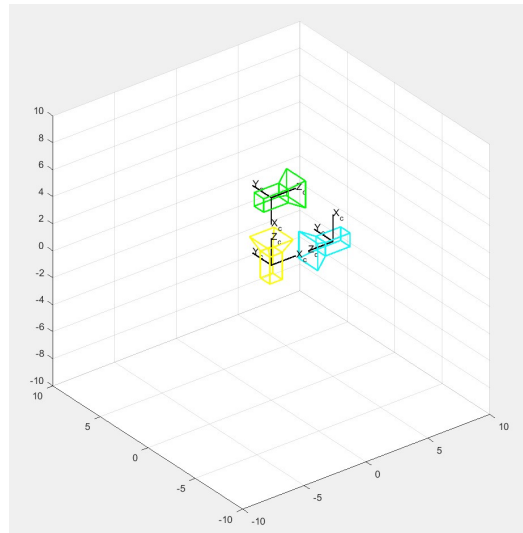


Abbildung 36: Die Abbildung zeigt das Ergebnis der drei Kameras beschrieben im Weltkoordinatensystem

- Setzen Sie den Wertebereich Ihres Koordinatensystems in einem Bereich von +/- 10 m für jede Achse (Methoden *xlim* bzw. *ylim* bzw. *zlim*)
- Definieren Sie sich ein zweites Koordinatensystem *K2*, welches 90 Grad um die Y-Achse gedreht ist und in 5 [m] entlang der z-Achse zum Ursprung verschoben ist.
- Definieren Sie sich ein drittes Koordinatensystem *K3*, welches -90 Grad um die Y-Achse gedreht ist und in 5 [m] entlang der x-Achse zum Ursprung verschoben ist

⁵ **Hinweis:** Aufgrund der Matlab Notation muss die Orientierung bei *plotCamera* als Transponierte der aus dem Koordinatensystem ableitbaren Orientierung übergeben werden.

Weitere Informationen siehe

https://de.mathworks.com/help/vision/ref/cameraposextrinsics.html#bvcpjz_-1-location

- Definieren Sie sich die Transformation T_{23} zwischen den beiden Koordinatensystemen K_2 und K_3 als homogene Matrix (Hinweis: Die Inverse einer Matrix M wird in Matlab mit $inv(M)$ berechnet)
- Extrahieren Sie aus T wieder die Translation und schreiben Sie diese auf die Konsole (Methode `trform2trvec`)
- Extrahieren Sie aus T die Eulerwinkel und schreiben Sie diese auf die Konsole (Methode `tform2eul`)

7.1.2.2 DIE RPY DARSTELLUNG (FREIWILLIG)

- Implementieren Sie eine Matlab Funktion mit dem Namen `setRPY`, die aus drei Winkeln eine 3x3 Matrix berechnet. Hierbei solle die roll, pitch, yaw Darstellung für Rotationen verwendet werden (vgl Skript).
- Implementieren Sie eine Matlab Funktion `getRPY`, die aus einer 3x3 Matrix die drei Winkel der RPY Darstellung extrahiert

7.1.2.3 CHECK IF ROTATION

Implementieren Sie eine Methode `isRot`, die überprüft, ob es sich bei einer gegebenen 3x3 Matrix um eine Rotation handelt.

7.1.2.4 PUNKTWOLKEN TRANSFORMIEREN (FREIWILLIG)

Gegeben sei eine Punktwolke mit `pc` (vgl. 'teapotOut.asc') sowie eine Transformation in ein neues Koordinatensystem durch

```
M = [
    cos(pi/4) sin(pi/4) 0 0;
    -sin(pi/4) cos(pi/4) 0 0;
    0 0 1 5;
    0 0 0 1
];
```

Zum Lösen der Aufgabe gehen Sie wie folgt vor:

- Laden Sie die Punktwolke mit `pcread` und stellen Sie diese mit `pcshow` dar
- Überprüfen Sie mit Ihrer Funktion `isRot`, ob M eine Rotation beinhaltet
- geben Sie die rpy Winkel der Rotation mit Ihrer Funktion `getRPY` aus
- Transformieren Sie jetzt `pc` mit M in das neue Koordinatensystem,
 - indem Sie mit Hilfe einer for Schleife jeden Punkt aus `pc` mit M transformieren und als neuen Punkt `pc_new` abspeichern
 - Das Ergebnis mit `pcshow` in grün darstellen
- Speichern Sie die neue Punktwolke `pc_new` als `ply` mit der Methode `pcwrite`.

7.1.3 KAMERA-KALIBRIERUNG NACH ZHANG (WOCHE 3)

7.1.3.1 KALIBRIERPATTERN UND BILDAUFNAHME

Ziel der Aufgabe ist es, ein eigenes Kalibrierpattern anzufertigen und mit dem Handy sinnvolle Kalibrieraufnahmen durchzuführen. Gehen Sie hierzu wie folgt vor

- Fertigen Sie sich ein Kalibrierpattern an. Vorlagen hierzu finden Sie über die Matlab Hilfe
- Nehmen Sie mit Ihrer Handykamera Bilder des Musters auf. Anordnung vgl. Skript Abschnitt 3.2 und Abbildung 36. Wichtig ist ein Objektiv mit fixer Brennweite, welches während der Bildaufnahme nicht verändert werden darf

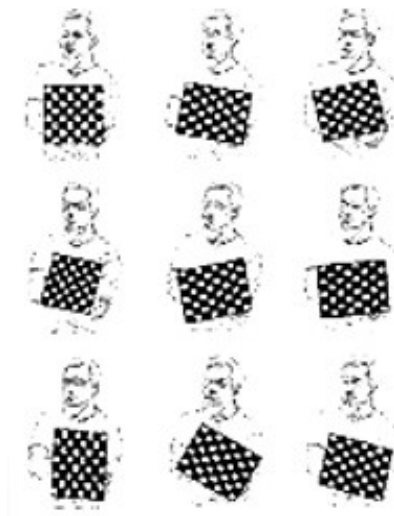


Abbildung 37: Quelle O'Reilly. Die Abbildung zeigt schematisch die Aufnahme des Kalibrierpatterns. Hierzu sind unterschiedliche Entfernungen und Blickwinkel notwendig, jedoch muss das Pattern immer vollständig im Bild erkennbar sein.

7.1.3.2 KALIBRIERUNG EINES MONOSYSTEMS

Führen Sie jetzt eine Monobildkalibrierung mit der in Matlab bereitgestellten *cameraCalibratorApp*⁶ durch (siehe auch Abbildung 38 (a.)). Verwenden Sie hierzu die in der Computer Vision Toolbox bereitgestellten Monobilder⁷. Speichern Sie die Kameraparameter ab und dokumentieren Sie Ihre Ergebnisse in einem pdf und so dass die **Ergebnisbilder eingebunden** werden.

Hinweise:

- Hinterfragen Sie die bereitgestellten Bildsequenzen hinsichtlich ihrer Güte für die Kalibrierung!

⁶ eine online Hilfe siehe unter <http://de.mathworks.com/help/vision/ref/cameracalibrator-app.html>

⁷ → siehe ..\toolbox\vision\visiondata\calibration\

- Die in obiger Aufgabe aufgenommenen Handybilder eignen sich prinzipiell nicht für eine eigene Kalibrierung, da die Hersteller im Allgemeinen schon mit eigenen Methoden eine Korrektur durchführen. Trotzdem könnten Sie den Ablauf aus Übungszwecken auch mit diesen Bildern durchführen und Ihre Ergebnisse interpretieren.

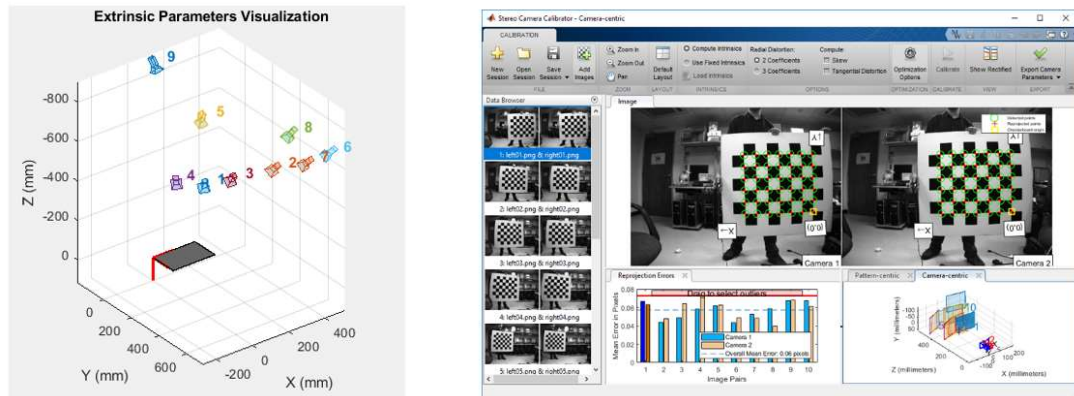


Abbildung 38: Quelle Matlab (a.) Anordnung der Bilder nach Kalibrierung (b) von Matlab bereitgestellte App zur Stereo Kalibrierung

7.1.3.3 STEREOBILDKALIBRIERUNG

Ziel der Aufgabe ist es, die Kalibrierung eines Stereokamerasystems mit der in Matlab bereitgestellten *StereoCameraCalibratorApp*⁸ durchzuführen (vgl. Abbildung 38 (b.)). Gehen Sie hierzu in folgenden Schritten vor:

- Verwenden Sie hierzu die in der Computer Vision Toolbox bereitgestellten Stereobilder⁹
- Speichern Sie die extrinsische Kameraparameter ab
- Dokumentieren Sie Ihre Ergebnisse in einem pdf und so dass die **Ergebnisbilder eingebunden** werden. Laden Sie dieses auf Moodle

7.1.3.4 BILDER ENTZERREN

Legen Sie ein neues Skript an und gehen Sie wie folgt vor

- Laden Sie die Kalibrierparameter Ihrer Stereobildkalibrierung pro Kamera jeweils in die Matlab Datenstruktur *cameraParameters*
- Laden Sie dann beide *CameraParameters* Objekte in die *stereoParameters* Struktur von Matlab

⁸ Eine online Hilfe siehe unter <http://de.mathworks.com/help/vision/ug/stereo-camera-calibrator-app.html>

⁹ → siehe ..\toolbox\vision\visiondata\calibration\

- Verwenden Sie die *undistortImage* Methode, um Ihre Bilder in verzerrungsfreie Bilder überzuführen (also die Umkehrung der in Kap. 3.2.2.5 beschriebenen *world_2_Pixel* Abbildung) und stellen Sie das Ergebnis mit *imshow* dar.
- Verwenden Sie *subplot* oder *montage*, um die verzerrungsfreien Bilder als Tabelle anzuordnen und darzustellen
- Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode und so dass die **Ergebnisbilder** eingebunden werden. Laden Sie dieses auf Moodle

7.1.3.5 WORLD_2_PIXEL UND UNDIST (FREIWILLIG)

(In Pseudocode):

- Implementieren Sie die Funktion *world_2_pixel* (vgl. Absatz 3.2.2.5)! Überlegen Sie sich hierzu geeignete Übergabe- und Rückgabeparameter.

```
[ .. ] = world_2_pixel( .. )
```

- (In Pseudocode): Implementieren Sie die Funktion *undist* unter Verwendung obiger *world2pixel* Abbildung. Überlegen Sie sich hierzu geeignete Übergabe- und Rückgabeparameter.

```
[ .. ] = undist( .. )
```

7.1.4 DIE STANDARDSTEREOGEOMETRIE (WOCHE 4)

Ziel der Aufgabe ist es, die geometrischen Zusammenhänge der Standardstereogeometrie besser zu verstehen.

7.1.4.1 WIEDERHOLUNG

Die Transformation von Welt- nach Kamerakoordinaten (=externen Kameraparameter) wird in Matlab wie folgt berechnet ¹⁰

$$\begin{array}{c} [x \ y \ z] \\ \text{camera coordinates} \end{array} = \begin{array}{c} [X \ Y \ Z] \\ \text{world coordinates} \end{array} R + t \quad \begin{array}{l} \text{translation vector} \\ \text{rotation matrix} \end{array}$$

Für die von z.B. der *plotCamera* Funktion benötigte *Location* und *Orientation* gilt dann

```
orientation = R' % Orientierung im W-KOS
location = -t * R' % Lage im W-KOS
```

Dies folgt aus

¹⁰ Vgl. https://de.mathworks.com/help/vision/ref/cameraposeparameters.html#bvcpjz_1-location

$$X^{(c)} = X^{(w)}R + t \rightarrow X^{(w)} = (X^{(c)} - t) * R'$$

Dabei lässt sich die *orientation* sowie die *location* in eine homogene Matrix überführen und als Transformation $K^{(c)}$ vom **Kamera- in das Welt-KOS** interpretieren.

7.1.4.2 KOORDINATENSYSTEME UND KAMERAS DEFINIEREN

- Definieren und visualisieren Sie sich ein Koordinatensystem W ("**Weltkoordinatensystem**") wie folgt
 - mit dem Ursprung $T=[0, 0, 0]$
 - ohne weiteren Drehwinkel
 - zum Visualisieren verwenden Sie die Methode `plotCamera`. Implementieren Sie hierbei folgende Eigenschaften: Farbe schwarz, Name World, Achsen sichtbar.
- Definieren Sie sich ein Koordinatensystem für eine Kamera K1 ("**1. Kamera Koordinatensystem**") wie folgt
 - mit dem Ursprung $T=[-0.5, 0, 1]$ [m] (Methode `trvec2tform`)
 - mit keiner Drehung um irgendeine Achse
 - zum Visualisieren verwenden Sie die Methode `plotCamera`. Implementieren Sie hierbei folgende Eigenschaften: Farbe *rot*, Name **K1**, Achsen sichtbar.
 - Die Brennweite der Kamera solle 0.002 [m] betragen; die Kamera sei verzeichnungsfrei.
- Definieren Sie sich ein Koordinatensystem für eine Kamera K2 ("**2. Kamera Koordinatensystem**") wie folgt
 - mit dem Ursprung $T=[0.5, 0, 1]$ [m] (Methode wie oben)
 - mit keiner Drehung um irgendeine Achse
 - zum Visualisieren verwenden Sie die Methode `plotCamera`. Implementieren Sie hierbei folgende Eigenschaften: Farbe *blau*, Label **K2**, Achsen sichtbar
 - Die Brennweite der Kamera solle 0.002 [m] betragen; die Kamera sei verzeichnungsfrei

7.1.4.3 DARSTELLUNG

- Verwenden Sie die `hold on` Eigenschaft, um alle Visualisierungen in einer Grafik darzustellen
- Wählen Sie die Matlab Optionen `grid on` und `axis` um Ihren Arbeitsraum besser darzustellen (mit $x=-3 \dots 3$, $y=-3 \dots 3$, $z=0 \dots 4$)
- Definieren Sie sich die Transformation T zwischen den beiden Koordinatensystemen als homogene Matrix (Hinweis: Die Inverse einer Matrix M wird in Matlab mit `inv(M)` berechnet)

7.1.4.4 3D PUNKTE SIMULIEREN UND AUF BILDEBENE PROJIZIEREN

Ziel der Teilaufgabe ist es, 3D Punkte zu generieren und diese auf die Bildebenen der jeweiligen Kamera zu projizieren. Gehen Sie hierzu in folgenden Schritten vor:

- Simulieren Sie sich 80 3D Punkte P unter Verwendung einer *for* Schleife (Syntax siehe Matlab Hilfe) und visualisieren Sie diese mit *scatter3*. Die Punkte sollen entlang einer Linie mit Richtungsvektor $[1 \ 1 \ 0]$ liegen.
Hinweis: Ein guter Startpunkt der Linie könnte z.B. der Punkt $\mathbf{po} = [-4 \ -4 \ 3]$ und ein guter Endpunkt könnte der Punkt $\mathbf{eo} = [4 \ 4 \ 3]$ sein
- Projizieren Sie jetzt die Punkte P auf die realen Bildebenen beider Kameras (Verwenden Sie als Brennweite die oben genannte Brennweite). Implementieren Sie also die Teilschritte
 - Transformation der Punkte P in das jeweilige Kamerakoordinatensystem $K1$ bzw $K2$
 - perspektivische Projektion auf die reale Bildebene der jeweiligen Kamera *cam1* bzw *cam2* (diese Berechnung können Sie z.B. aus den in der Lehrveranstaltung besprochenen Beziehungen der kongruenten Dreiecke ableiten, vgl. Vorlesung).
 - Speichern Sie diese Punkte in einem Vektor $\mathbf{pl}$ und $\mathbf{pr}$

7.1.4.5 PUNKTE 3D REKONSTRUIEREN

Ziel der Teilaufgabe ist es, aus den in obiger Teilaufgabe simulierten korrespondierenden Punkt $\mathbf{pr}$ und $\mathbf{pl}$ wieder 3D Punkte zu rekonstruieren. Gehen Sie hierzu in folgenden Schritten vor:

- implementieren Sie jetzt die 3D Rekonstruktion zweier in der realen Bildebene der Kameras beschriebenen korrespondierender Punktpaare (vgl. Vorlesung).
- Testen Sie Ihr Ergebnis, indem Sie es mit den oben vorgegebenen 3D Punkten vergleichen
- Modifizieren Sie jetzt Ihre korrespondierenden Pixel $\mathbf{pl}$ und $\mathbf{pr}$, indem Sie kleine zufallsverteilte Fehler auf die Pixel addieren (mit der Matlab Funktion *rand*)
- Wiederholen Sie die Berechnung der 3D Punkte und vergleichen Sie wieder Ihre berechneten Werte mit den simulierten Daten

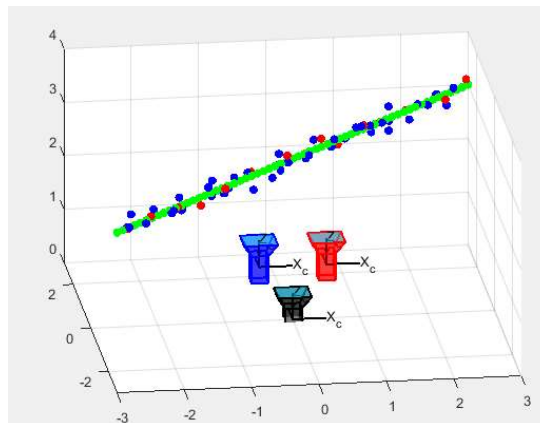


Abbildung 39: Grün: Simulierte 3D Punkte, Blau: 3D Punkte + Noise, Rot: Aus Ansatz für Stereogeometrie rekonstruierte 3D Punkte

7.1.4.6 DOKUMENTATION

Dokumentieren Sie Ihr Programm, indem Sie kurze Kommentare anfügen, sowie ggf die Foliennummer im Skript mit anfügen. Verwenden Sie die *publish* Option von Matlab!

7.1.5 DISPARITÄTSBILD UND 3D REKONSTRUKTION (WOCHE 5)

7.1.5.1 AUFGABE (DISPARITÄTSBILD UND 3D REKONSTRUKTION)

Ziel der Aufgabe ist es aus zwei Bildern in Standardgeometrie bereitgestellten Bildern eine 3D Punktwolke zu generieren. Gehen Sie wie folgt vor:

- Laden Sie zuerst die von Matlab bereitgestellten Bilder *scene_l.jpg* und *scene_r.jpg* als Grauwertbilder mit Hilfe der Funktionen (Funktionen: *imread* und *rgb2gray*)
- Visualisieren Sie die Bilder als übereinandergelegte Bilder (Methode: *stereoAnaglyph*)
- Berechnen Sie ein Disparitätsbild (Methode: *disparity*). Testen Sie die Funktion mit unterschiedlichen Parametern.

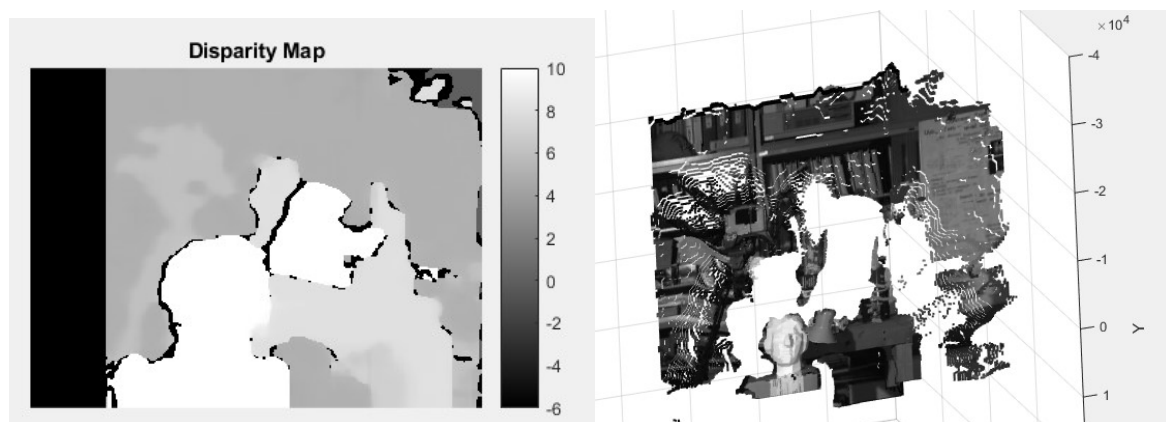


Abbildung 40: (a.) DisparityMap und (b.) 3D Punktwolke

- Berechnen Sie jetzt daraus eine 3D Punktwolke, mit Hilfe der in der Vorlesung besprochenen Abbildungsgleichungen. **Hinweis:**
 - Führen Sie Ihre Berechnungen zur Vereinfachung in Pixeleinheiten durch:
 - $\Rightarrow$ Verwenden Sie als Brennweite den Wert $f = 0.08 * 2000$ [pix]
 - $\Rightarrow$ Verwenden Sie als Basisabstand den Wert $0.4 * 2000$ [pix]
 - Beachten Sie, dass der Ursprung eines Bildkoordinatensystems in Matlab in der linken oberen Ecke angegeben ist, das in der Lehrveranstaltung verwendete Koordinatensystem jedoch seinen Ursprung in der Bildmitte hat (und damit jedes Pixel noch um den Bildmittelpunkt verschoben werden muss)
 - Die Rekonstruktion wird auch fehlerhafte Punkte beinhalten. Filter Sie deshalb alle Punkte weg, die „außerhalb Ihres gewünschten Bereiches liegen“
- Stellen Sie Ihre Punktwolke mit *pcshow* dar!
- Dokumentieren Sie Ihr Programm, indem Sie kurze Kommentare anfügen, sowie ggf. Folien Nummer im Skript mit anfügen. Verwenden Sie die *publish* Option von Matlab

7.1.5.2 AUFGABE (FREIWILLIG)

In einem matlabähnlichen Pseudocode:

- Wie lässt sich eine *DisparityMap* unter Verwendung der SSD Ähnlichkeitsmetrik berechnen?
- Wie lautet der Algorithmus zum Erstellen einer 3D Punktwolke aus einer *DisparityMap* (vgl. auch Teilaufgabe aus obiger Aufgabe)?

7.1.6 KORR.- ANALYSE II – MOVAREC/HARRIS UND SIFT (WOCHE 6)

7.1.6.1 AUFGABE (EXTRACT AND MATCH FEATURE)

DetectFeatures

Ziel der Aufgabe ist es, markante Punkte in einem Bild zu extrahieren (ähnlich zum Movarec Operator aus der Lehrveranstaltung). Zum Lösen der Aufgabe gehen Sie in folgenden Schritten vor:

- Laden Sie die Bilder 'eiffel2-1.jpg' und 'eiffel2-2.jpg' und transformieren Sie diese in Grauwertbilder
- Verwenden Sie die Funktion *detectHarrisFeatures*, um markante Punkte zu extrahieren
- Stellen Sie diese im Bild mit der plot Methode dar (vgl. hierzu *cornerPoints* object in der Matlab Hilfe)
- Extrahieren Sie jetzt aus allen Merkmalen nur die 20 stärksten Merkmale und
 - stellen Sie diese mit der plot Methode der Klasse *cornerPoints* dar

- schreiben Sie diese auf die Konsole



Abbildung 41: Quelle Peter Corke. Harris Corner Detection

ExtractAndMatchFeatures (vgl. SIFT im Skript)

Ziel der Aufgabe ist es markante Punkte in einem Bildpaar zu extrahieren und zu Matchen (vgl. SIFT im Skript, *Bildquelle:* Peter Corke). Zum Lösen der Aufgabe gehen Sie in folgenden Schritten vor:

- Pro Bild:
 - Verwenden Sie die Funktion `detectSURFFeatures`, um markante Punkte in jedem Bild zu extrahieren.
 - Beschränken Sie die Anzahl mit einer geeigneten Option auf 200 (siehe `SurfPoint` Object in der Matlab Hilfe).
 - Verwenden Sie die Funktion `extractFeatures`, um aus den markanten Punkten Merkmale zu extrahieren
 - Stellen Sie das Ergebnis dar

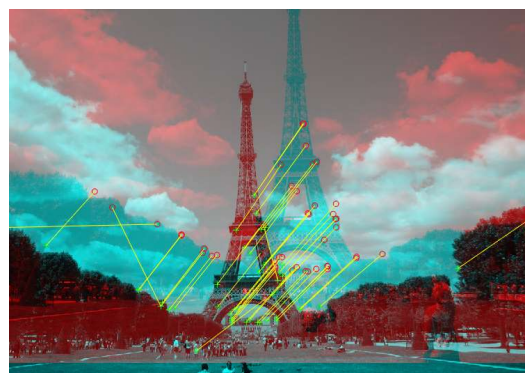
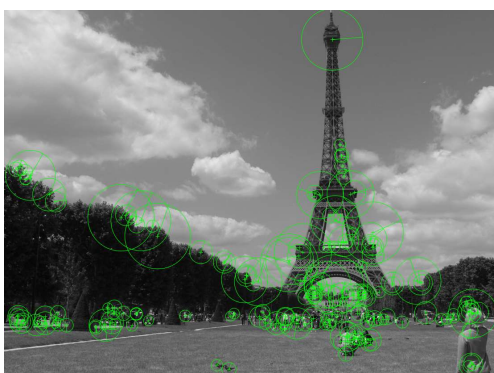


Abbildung 42: Quelle Peter Corke. Ergebnisse des SURF Operators nach feature extraction (a.) und feature matching (b.)

- Verwenden Sie die Funktion *matchFeatures*, um Punktkorrespondenzen zwischen beiden Bildern zu suchen und stellen Sie diese dar!
- Verwenden Sie die Klasse *vSet*, sowie die in ihr enthaltenen Methoden *addView* und *addConnection*, um Ihre Aufnahmestandpunkte und die korrespondierenden Punkte zu verwalten (*Hinweis*: Nehmen Sie als Orientierung die Einheitsmatrix und als Location den Nullvektor; die Berechnung erfolgt in einer der nächsten Lehrveranstaltungen).
- Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode **und so dass die Ergebnisbilder eingebunden werden**. Laden Sie dieses auf Moodle

7.1.7 EPIPOLARGEOMETRIE – F UND E MATRIX (WOCHE 7)

Ziel der Aufgabe ist es, die Epipolargeometrie zweier Kameras zu visualisieren, um ein besseres Verständnis der geometrischen Zusammenhänge zu bekommen (vgl. auch Aufgabe 7.1.2 und Aufgabe 7.1.4).

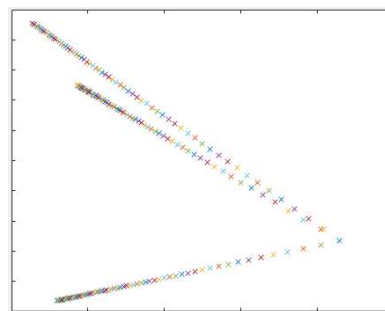
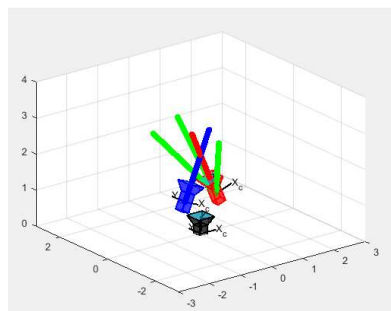
Gehen Sie in folgenden Schritten vor:

7.1.7.1 KAMERAS IN EPIPOLARGEOMETRIE

Koordinatensysteme definieren

Definieren und visualisieren Sie sich ein Koordinatensystem **W** ("Weltkoordinatensystem") wie folgt und in der Grafik dargestellt:

- mit dem Ursprung $T=[0, 0, 0]$ und ohne weiteren Drehwinkel
- zum Visualisieren verwenden Sie die Methode *plotCamera*¹¹. Implementieren Sie hierbei folgende Eigenschaften: Farbe *schwarz*, Name *World*, Achsen sichtbar



¹¹ **Hinweis**: Aufgrund der Matlab Notation muss die Orientierung bei *plotCamera* als Transponierte der aus dem Koordinatensystem ableitbaren Orientierung übergeben werden. Weitere Informationen siehe https://de.mathworks.com/help/vision/ref/cameraposeToExtrinsics.html#bvcpjz_-1-location

Abbildung 43: (a.) Die Abbildung zeigt die Anordnung der Kameras sowie die exemplarisch zu berechnenden Strahlen **(b.)** Ergebnis der auf die Bildebene der Kamera zurückprojizierten Strahlen

Definieren Sie sich ein Koordinatensystem **K1** ("1. Kamera Koordinatensystem") wie folgt

- mit dem Ursprung $T = [-0.5, 0, 1]$ [m] (Methode *trvec2tform*)
- mit einer Verkippung um die y-Achse um 0.4 rad (Methode *eul2tform*) und mit einer Verkippung um die x-Achse um 0.1 rad (Methode *eul2tform*)
- zum Visualisieren verwenden Sie die Methode *plotCamera*¹¹. Implementieren Sie hierbei folgende Eigenschaften: Farbe *rot*, Name *K1*, Achsen sichtbar

Definieren Sie sich ein Koordinatensystem **K2** ("2. Kamera Koordinatensystem") wie folgt

- mit dem Ursprung $T=[0.5, 0, 1]$ [m] (Methode wie oben)
- mit einer Verkippung um die y-Achse um -0.4 rad (Methode wie oben) und mit einer Verkippung um die x-Achse um -0.1 rad (Methode wie oben)
- zum Visualisieren verwenden Sie die Methode *plotCamera*¹¹ und implementieren Sie hierbei folgende Eigenschaften: Farbe *blau*, Label *K2*, Achsen sichtbar

Kameras definieren

Definieren Sie sich jetzt die internen und externen Parameter einer Kamera **cam1** wie folgt

- die Kamera habe die mit Zhang erstellten Kameraparameter
 - $(cx,cy) = (500, 350)$ und $(fx,fy) = (700, 700)$
 - keine Verzeichnungsparameter
- Die externen Kameraparameter sollen aus **K1** abgeleitet werden

Definieren Sie sich eine zweite Kamera **cam2** wie folgt

- die Kamera habe die mit Zhang erstellten Kameraparameter
 - $(cx,cy) = (500, 350)$ und $(fx,fy) = (700, 700)$
 - keine Verzeichnungsparameter
- Die externen Kameraparameter sollen aus **K2** abgeleitet werden

Visualisierung

Zum Visualisieren führen Sie die folgenden Schritte durch:

- Verwenden Sie die *hold on* Eigenschaft, um alle Visualisierungen in einer Grafik darzustellen

- Wählen Sie die *Matlab* Optionen *grid* on und *axis* um Ihren Arbeitsraum besser darzustellen (mit $x = -3 \dots 3$, $y = -3 \dots 3$, $z = 0 \dots 4$)
- Definieren Sie sich die Transformation T zwischen den beiden Koordinatensystemen als homogene Matrix. **Hinweis:** Die Inverse einer Matrix M wird mit $\text{inv}(M)$ berechnet

Strahlen berechnen und darstellen

Ziel der Teilaufgabe ist es, die optischen Achsen beider Kameras, sowie drei exemplarische "Sichtstrahlen" der Kamera **cam2** zu visualisieren. Gehen Sie hierzu in folgenden Schritten vor:

- Definieren Sie sich eine Linie entlang der optischen Achse $[0,0,1]$ von **cam1** und **cam2** und transformieren Sie diese in das W-Kos
- Definieren Sie sich Sichtstrahlen entlang der Richtungsvektoren $[0.1 \ 0.5 \ 1]$, $[-0.3 \ 0.5 \ 1]$ und $[0.1 \ -0.5 \ 1]$ und transformieren Sie diese in das W-Kos
- Stellen Sie das Ergebnis graphisch dar

Strahlen in Kamera cam1 projizieren

Ziel der Teilaufgabe ist es, die Sichtstrahlen in das Bildkoordinatensystem der Kamera **cam1** zu projizieren (vgl. hierzu auch den Anhang Kap. 7.1.7.3). Implementieren Sie also die in (74) formulierte Funktion *World2Pixel* mit folgenden Teilschritten

- Transformation der Punkte in das Kamerakoordinatensystem **K1**
- Perspektivische Projektion der Punkte auf die normalisierte Bildebene der Kamera **cam1**
- (*Kameras wurden verzeichnungsfrei angenommen* → keine Verzeichnungskorrektur)
- Transformation in das Bildkoordinatensystem mit f_x , f_y , c_x und c_y

Stellen Sie die berechneten Pixel in einem Plot mit Hilfe der *plot* Funktion dar. **Hinweis:** Die projizierten Strahlen sollten sich alle in einem Punkt schneiden (vgl. Abbildung 42).
Welchem?

E-Matrix und epipolar constraint

Ziel der Teilaufgabe ist es, die *epipolar constraint* zu überprüfen (vgl. Kap. 4.5.2 und Kap. 4.5.3). Gehen Sie hierzu in folgenden Schritten vor:

- Berechnen Sie die Rotation R und Translation T zwischen beiden Kamerastandpunkten
- Berechnen Sie die E-Matrix aus R und S sowie die F Matrix
- Überprüfen Sie die Richtigkeit Ihrer berechneten *F-Matrix*, indem Sie die *epipolar constraint* mit Hilfe der korrespondierenden Pixel überprüfen

Dokumentation

Dokumentieren Sie Ihr Programm, indem Sie kurze Kommentare anfügen, sowie ggf. die Foliennummer im Skript mit anfügen. Verwenden Sie die *publish* Option von *Matlab*!

7.1.7.2 KORRESPONDIERENDE PUNKTE MIT F MATRIX ÜBERPRÜFEN (FREIWILLIG)

Gegeben seien ein Bildpaar und eine dazugehörige *F-Matrix* (mit dem linken Bild als Referenzkoordinatensystem) sowie mit *SIFT* extrahierte Matches zwischen beiden. Verwenden Sie das *epipolar constraint*, um aus den Matches die *outlier* zu extrahieren! Implementieren Sie dies in in Pseudocode und unter Verwendung der Computer Vision Toolbox von Matlab (Daten siehe Moodle).

7.1.7.3 ANHANG

Wie wird ein Punkt $P^{(c)}$ aus *einer Kamera* im *Welt-Kos* dargestellt? Im Folgenden wird dies exemplarisch (und ohne zu stark auf mathematische Formalismen Wert zu legen) für einen Punkt der optischen Achse $(0,0,\lambda)$ gezeigt. Gegeben seien hierzu das Weltkoordinatensystem $K^{(w)}$, sowie die Transformation $K^{(c)}$ (als homogene Matrix) vom Kamera- in das Weltkoordinatensystem. Folgende drei Ansätze:

- **Interpretation als Bewegung im $\mathbb{R}^3$** , siehe Kap. 2.3.3: Hier wird das Objekt mittels $K^{(c)}$ in das neue Koordinatensystem (=das Welt-KOS) bewegt. Das ist der Zugang, wie er auch in der mobilen Robotik und unter Verwendung von homogenen Matrizen verwendet wird.

Also:

$$P^{(w)} = K^{(c)} P^{(c)} \quad (143)$$

- **„Sichtstrahl konstruieren“**, siehe Hier ist die Idee, den Sichtstrahl geometrisch als Linearkombination aus den drei Achsen X, Y, Z und dem Ursprung T des neuen Koordinatensystems (siehe Kap. 2.4) zu konstruieren. Der in die Welt transformierte Punkt $P^{(w)}$ ergibt sich dann aus der Transformation $P^{(w)} = (0 \cdot X + 0 \cdot Y + \lambda \cdot Z) + T$ also

$$P^{(w)} = \begin{pmatrix} 0 * K_{11}^{(c)} + 0 * K_{12}^{(c)} + \lambda * K_{13}^{(c)} \\ 0 * K_{21}^{(c)} + 0 * K_{22}^{(c)} + \lambda * K_{23}^{(c)} \\ 0 * K_{31}^{(c)} + 0 * K_{32}^{(c)} + \lambda * K_{33}^{(c)} \end{pmatrix} + \begin{pmatrix} K_{14}^{(c)} \\ K_{24}^{(c)} \\ K_{34}^{(c)} \end{pmatrix} = K^{(c)} P^{(c)} \quad (144)$$

- **Die Welt bleibt dieselbe und es wird ein Basiswechsel (und damit ein anderer Blickwinkel) durchgeführt**: Das ist der Ansatz den Mathematiker haben und der aus der linearen Algebra kommt; Seien die Punkte $P^{(w)}$ und $P^{(c)}$ derselbe Punkt, jedoch in unterschiedlichen Basen $B^{(w)}$ und $B^{(c)}$ beschrieben. Damit gilt $B^{(w)} P^{(w)} = B^{(c)} P^{(c)}$. Die

jeweilige Basis ergibt sich aber gerade aus den Spaltenvektoren der Koordinatensysteme.

Also gilt für $P^{(w)}$:

$$P^{(w)} = B^{(w)} P^{(c)} = \underbrace{K^{(w)}}_{= \text{Einheitsmatrix}} P^{(c)} \quad \text{und} \quad P^{(c)} = B^{(c)} P^{(w)} = K^{(c)} P^{(w)} \quad (145)$$

7.1.8 WIEDERHOLUNG (WOCHE 8)

7.1.8.1 PROJEKT DENSE MATCHING

Siehe → Aufgabe 7.2.3 Teil 2/6 (Dense Matching - Multiple Views)

7.1.9 8-PUNKTE ALGORITHMUS UND RANSAC (WOCHE 9)

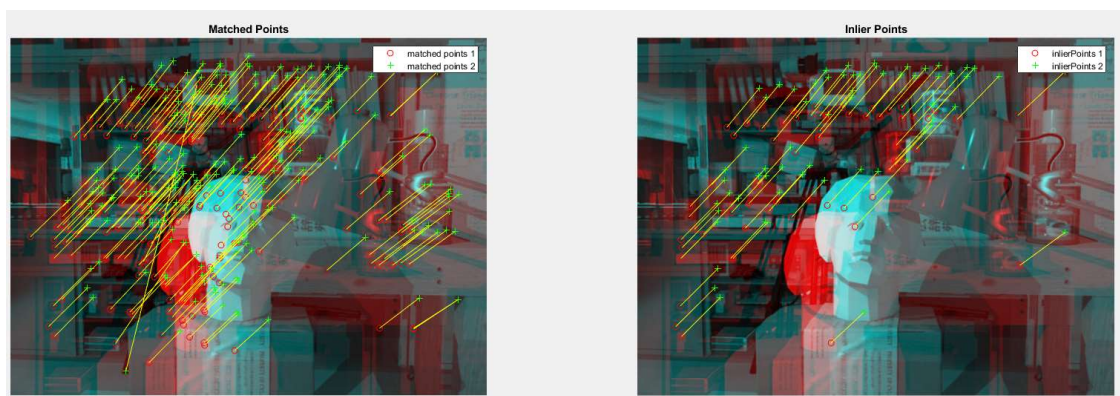


Abbildung 44: Übereinandergelagertes Stereopaar **(a.)** Ergebnis nach dem Feature Matching
(b.) Ergebnis nach der Outlier Detection mit Ransac

7.1.9.1 PROJEKT

- Siehe → Aufgabe 7.2.4 Teil 3/6 (8-Punkte Algorithmus, RANSAC)

7.1.9.2 8 PUNKTE ALGORITHMUS

- Implementieren Sie den 8-Pkte Algorithmus in einem Matlab ähnlichen Pseudocode.

7.1.10 POSE BERECHNEN UND TRIANGULATION (10. WOCHE)

7.1.10.1 PROJEKT DENSE MATCHING

- Siehe → Angabe 7.2.5 Teil 4 / 6 (Pose aus E Matrix berechnen)

7.1.10.2 PSEUDOCODE

- **Stereotriangulation:** Implementieren Sie einen Algorithmus zur Stereotriangulation in einem Matlab ähnlichen Pseudocode.

- **pose_From_E_Matrix:** Implementieren Sie in einem Matlab ähnlichen Pseudocode, wie die Rotation und Translation aus einer gegebenen E Matrix berechnet werden kann.

7.1.11 PROJEKT: BUNDLEADJUSTMENT (11. WOCHE)

7.1.11.1 PROJEKT DENSE MATCHING

Siehe → Aufgabe 7.2.6 Teil 5/ 6 (Bundle Adjustment)

7.1.12 RECTIFY (12. WOCHE)

Ziel der Aufgabe ist es, zwei Bilder zu rektifizieren und daraus ein 3D Modell zu berechnen.



Abbildung 45: Quelle Matlab: (a.) rektifizierte Bilder (b.) Disparity Map (c.) Rekonstruierte 3D Punktwolke

7.1.12.1 AUFGABE (RECTIFY AND RECONSTRUCTSCENE)

Zum Lösen der Aufgabe gehen Sie in folgenden Schritten vor:

- Laden Sie die Datei `'webcamsSceneReconstruction.mat'` mit `load`, um die drin gespeicherten Stereoparameter verwenden zu können.
- Laden Sie die Bilder `'sceneReconstructionLeft.jpg'` und `'sceneReconstructionRight.jpg'` und stellen Sie diese dar
- Überführen Sie beide in Standardstereogeometrie mit `rectifyStereolImages`
- Berechnen Sie die Disparität mit `disparity` und stellen Sie das Disparitätsbild dar
- Berechnen Sie daraus ein dichtes 3D Modell mit `reconstructScene`
- Stellen Sie alle Punkte in einem Bereich von 3200 mm bis 3700 mm dar!

7.1.12.2 AUFGABE (ZUSAMMENFASSUNG ERSTELLEN)

Erstellen Sie sich eine Zusammenfassung der Vorlesungsinhalte auf einem beidseitig handbeschriebenem DIN A4 Blatt!

Achtung: Laden Sie dieses Blatt nicht auf Moodle, es dient zur Prüfungsvorbereitung!

7.1.12.3 PROJEKT DENSE MATCHING

Siehe → Aufgabe 7.2.7 Teil 6/6 (Gegebene Groundtruth)

7.2 PROJEKT: DENSE MATCHING (MATLAB)

7.2.1 ÜBERSICHT WORKFLOW

Gegeben sei eine Stereobildsequenz¹²; das Ziel ist die Berechnung der Trajektorie, um daraus ein dichtes 3D Modell zu generieren. Der **workflow** inklusive Angaben der Wochen ist (grob) wie folgt aufgebaut (bitte beachten Sie auch die Wochenangabe **Wo X** in blau):

```
% Schritt 0: init
load images → Teil 1,2 (Wo. 6,8)
init viewSet → Teil 1 (Wo. 6)
load Traj real → Teil 2 (Wo. 8)
load intrinsics → Teil 1 (Wo. 6)

% Schritt 1: Berechnung der Trajektorie → pairwise
1. Über alle Bilder (left ); immer zum ersten Bild (= Ref. Kos)

    (estimate Traj)
    1. undistort cameras → Teil 1 (Wo. 6)
    2. Extract and Match Features → Teil 1 (Wo. 6)
    3. add / update viewset → Teil 1 (Wo. 6)
    4. Detect Outlier, Berechne E Matrix / F Matrix → Teil 3 (Wo. 9)
    5. calc relative pose (Extraktion externe Kameraparameter) → Teil 4 (Wo 10)
    6. viewset, set connections → Teil 4 (Wo 10)

    (visualize Traj est and real)
    7. Visualize Traj (estimated) → Teil 4 (Wo 10)
    8. Visualize Traj (real) → Teil 4 (Wo 10)
    9. (Sparse Matching → Teil 6 (Wo 12)

    (transform Traj real)
    10. Transf. GroundTruth ins Ref.-Kos (= 1. Frame linke Kam.) → Teil 6 (Wo 12)
    11. add / update viewset → Teil 6 (Wo 12)

% Schritt 2: refinement → all images
2. Über alle Bilder (left) zueinander
    1. Match Features all images → Teil 5 (Wo. 11)
    2. add / update viewset → Teil 5 (Wo. 11)

Bundle Adjustment (nur estimated traj.) → Teil 5 (Wo 11)
```

¹² Data: Tsukuba University's CVLAB

[1] "Realistic CG Stereo Image Dataset with Ground Truth Disparity Maps", Martull et al., TrackMark 2012.

[2] "Towards a Simulation Driven Stereo Vision System", Peris et al., ICPR2012.

```
% Schritt 3: Dichtes 3D Modell → pairwise
3. Über Alle Bilder
    • rectify stereopair → obsolet, da synthetisch; Teil 2 (Wo. 8)
    • Dense Matching stereopair → Teil 2 (Wo. 8)
    • Transf. der Punktwolke in das Referenz-Kos → Teil 2 (Wo. 8) (mit geg. Traj)
```

7.2.2 TEIL 1/6 (EXTRACT AND MATCH FEATURES, VIEW_SET)

Ziel der Aufgabe ist es, aus der gegebenen Bildsequenz immer zum ersten Bild korrespondierende Punkte zu erzeugen¹³.

```
% Schritt 0: init
load images → Teil 1,2 (Wo. 6, 8)
init viewSet → Teil 1 (Wo. 6)
load intrinsics → Teil 1 (Wo. 6)
( .. )

% Schritt 1: Berechnung der Trajektorie → pairwise
1. Über alle Bilder (left ); immer zum ersten Bild (= Ref. Kos)

    (estimate Traj)
    • undistort cameras → Teil 1 (Wo. 6)
    • Extract and Match Features → Teil 1 (Wo. 6)
    • add viewset → Teil 1 (Wo. 6)
    • ( .. )
( .. )
```

Adaptieren Sie Praktikum 7.1.6 wie folgt:

7.2.2.1 IN SCHRITT 0 INIT ,

- Verwenden in *Schritt 1* Trajektorie berechnen
- Initialisieren Sie unter Verwendung der Methode *cameraParameters* ein Kameraobjekt. Verwenden Sie hier die Option '*IntrinsicMatrix*' mit den inneren Parametern

$$\mathbf{K} = [615 \ 0 \ 320; 0 \ 615 \ 240; 0 \ 0 \ 1];$$
- Verwenden Sie die *undistortImage* Methode, um das aktuelle Bild verzeichnungsfrei zu rechnen (*Hinweis*: Da es sich um synthetische Bilder handelt, ist es für die gegebene Bildsequenz eigentlich nicht notwendig)
- Verwenden Sie eine for Schleife, um mit *SURF* korrespondierende Punkte vom i-ten Bild zum ersten Bild zu berechnen
- Verwenden Sie *showMatchedFeatures*, um die gefundenen Feature Punkte geeignet darzustellen

¹³ **Materialien** zum download: *Bildseq_Tsukuba_lab.zip*

- Verwenden Sie *viewSet* (Methoden: *addView* und *addConnection*), um Ihre Matches und Standpunkte zu verwalten (Hinweis: Nehmen Sie auch hier wieder als ‚Orientation‘ die Einheitsmatrix und als ‚location‘ den Nullvektor; die Berechnung erfolgt in einer der nächsten Themenblöcke).
- Erweitern Sie Ihr Programm mit dem *VideoPlayer* Object, um Bilder besser darstellen zu können und (später) als Movie abspeichern zu können
- Sie das *imageDatastore* Object, um Ihre Bilder zu laden (vgl. Matlab Hilfe)

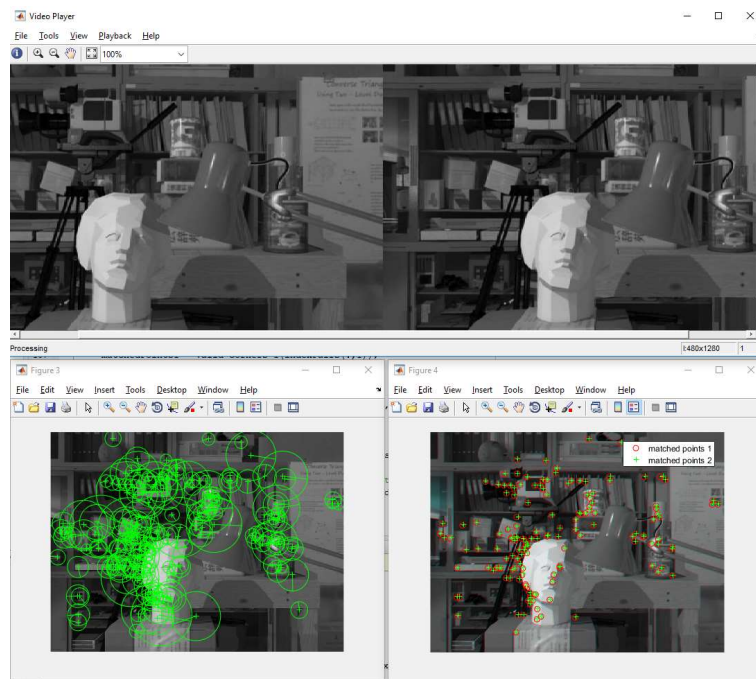


Abbildung 46: Quelle Tsukuba lab. Feature Extraction and Matching

7.2.2.2 DOKUMENTATION

Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode **und so, dass die Ergebnisbilder eingebunden werden**. Laden Sie dieses auf MoodleTeil 2: Dense Matching mit geg. GroundTruth (Woche 8)

7.2.3 TEIL 2/6 (DENSE MATCHING - MULTIPLE VIEWS)

Ziel der Aufgabe ist es, ein dichtes 3D Modell aus der gegebenen Bildsequenz zu erzeugen. Die Trajektorie ist hierbei bereits gegeben¹⁴.

¹⁴ **Materialien** zum download: **(a.)** *vSet_real.mat* (Vorab berechnete Trajektorie) **(b.)** *Bildseq_Tsukuba_lab_10_images.zip* (Stereo bildsequenz mit Disparitätsbild).

Ergänzen Sie Ihren Code aus Aufgabe 7.2.2, indem Sie den workflow um die folgenden Schritte erweitern:

```
% Schritt 0: init
load images → Teil 1,2 (Wo. 6, 8)
load Traj real → Teil 2 (Wo. 8)

( ... )

% Schritt 3: Dichtes 3D Modell → pairwise
1. Über Alle Bilder
    • rectify stereopair → obsolet, da synthetisch; Teil 2 (Wo. 8)
    • Dense Matching stereopair → Teil 2 (Wo. 8)
    • Transf. der Punktwolke in das Referenz-Kos → Teil 2 (Wo. 8) (mit geg. Traj)
    • ( ... )

( ... )
```

7.2.3.1 IN SCHRITT 0 „INIT“

- Verwenden Sie das *imageDatastore* Object um die Bilder aus Folder `\left\disp\right` der auf Moodle bereitgestellte Bildsequenz zu laden
- Initialisieren Sie unter Verwendung der Methode *cameraParameters* ein Kameraobjekt. Verwenden Sie hier die Option '*IntrinsicMatrix*' mit den inneren Parametern

$$\mathbf{K} = [615 \ 0 \ 320; 0 \ 615 \ 240; 0 \ 0 \ 1];$$
- Laden Sie das auf Moodle bereitgestellte viewSet *vSet_real.mat*.

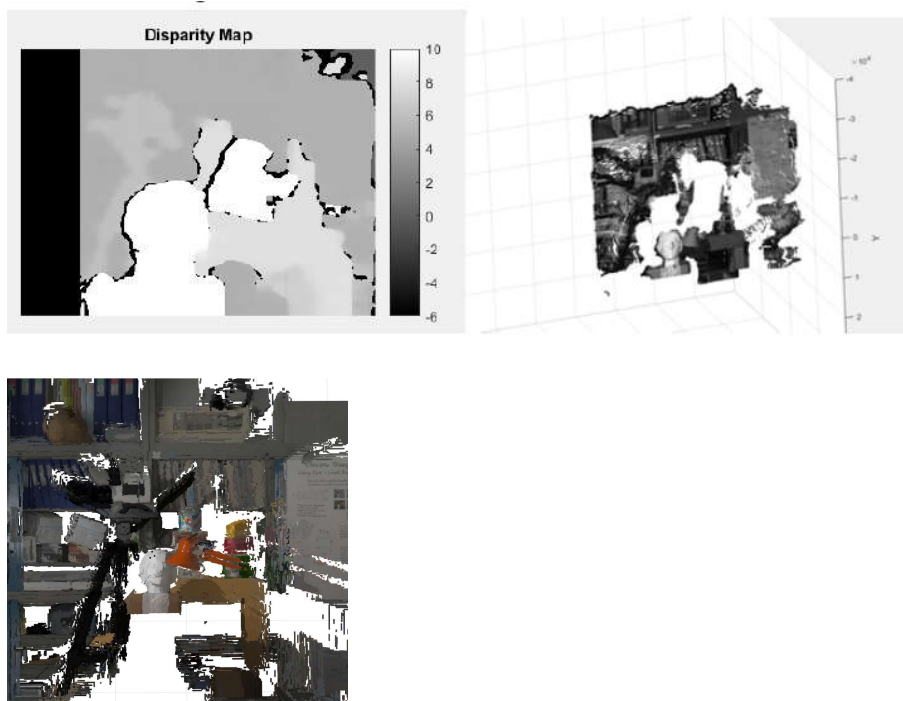


Abbildung 47: Quelle Tsukuba lab. Die Abbildung (b.) zeigt das Ergebnis des dense matchings unter Verwendung der gegebenen Ground Truth, sowie der gegebenen Disparitätsbilder

7.2.3.2 IN SCHRITT 3 „DENSE MATCHING“

Modifizieren Sie Ihre in Aufgabe 7.1.5.1 entwickelte Methode, so dass Sie aus den vorliegenden Stereopaaren eine dichte 3D Punktwolke berechnen können

- Verwenden Sie im ersten Schritt, die auf Moodle bereitgestellten und schon prozessierten Disparitätsbilder (Order \disp)
- Implementieren Sie eine Methode mit dem Namen *denseMatching_helper*, um zuerst paarweise ein dense matching durchzuführen, *Hinweis*: Sie könnten das Ergebnis bspw. unter Verwendung eines cell Arrays als *pointCloud* pc{i} mit *pcwrite* speichern, und vorab Störpunkte mit *pcdenoise* eliminieren. Der Methodenkopf könnte lauten:

```
function pc /*Punktwolke pc pointCloud*/ =
    helper_reconstructScene(
        disp /* DispImage */,
        K /* innere CamParams */, img /*Bild f. Grauwerte */,
        zlimit /*Begrenzung um Outlier zu elimin.*/,
        B /* Base */
    )
```

- Verwenden Sie die gegebene Trajektorie, um die mit *denseMatching_helper* prozessierten Punkte in das Referenzkoordinatensystem zu transformieren.

7.2.3.3 DOKUMENTATION

Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode **und so, dass die Ergebnisbilder eingebunden werden**. Laden Sie dieses auf Moodle.

7.2.4 TEIL 3/6 (8-PUNKTE ALGORITHMUS, RANSAC)

Ziel der Aufgabe ist es, aus den in Teil 1 extrahierten Matches, die outlier zu extrahieren, um daraus die E Matrix berechnen zu können¹⁵.

Ergänzen Sie Ihren Code aus Aufgabe 0, indem Sie den workflow um die folgenden Schritte erweitern:

```
( .. )
% Schritt 1: Berechnung der Trajektorie → pairwise
1. Über alle Bilder (left ); immer zum ersten Bild (= Ref. Kos)
    (estimate Traj)
        • ( .. )
        • Detect Outlier, Berechne E Matrix / F Matrix → Wo 9
        • ( .. )
( .. )
```

¹⁵ **Materialien** zum download: (a.) *Bildseq_Tsukuba_lab_10_images.zip*

7.2.4.1 IN SCHRITT 1 (TRAJEKTORIE BERECHNEN)

- Verwenden Sie die Methode *estimateFundamentalMatrix*
 - mit der Option *Ransac*, um die Fundamentalmatrix zu berechnen
 - Verwenden Sie den Rückgabeparameter *inlierIndex*, um daraus Ihre *outlier* zu eliminieren
- Verwenden Sie *showMatchedFeatures*, um die inlier (in grün) geeignet darzustellen
- Überprüfen Sie die Eigenschaften der F-Matrix (vgl. Skript), mit Hilfe der Methoden *rank* und *eig*

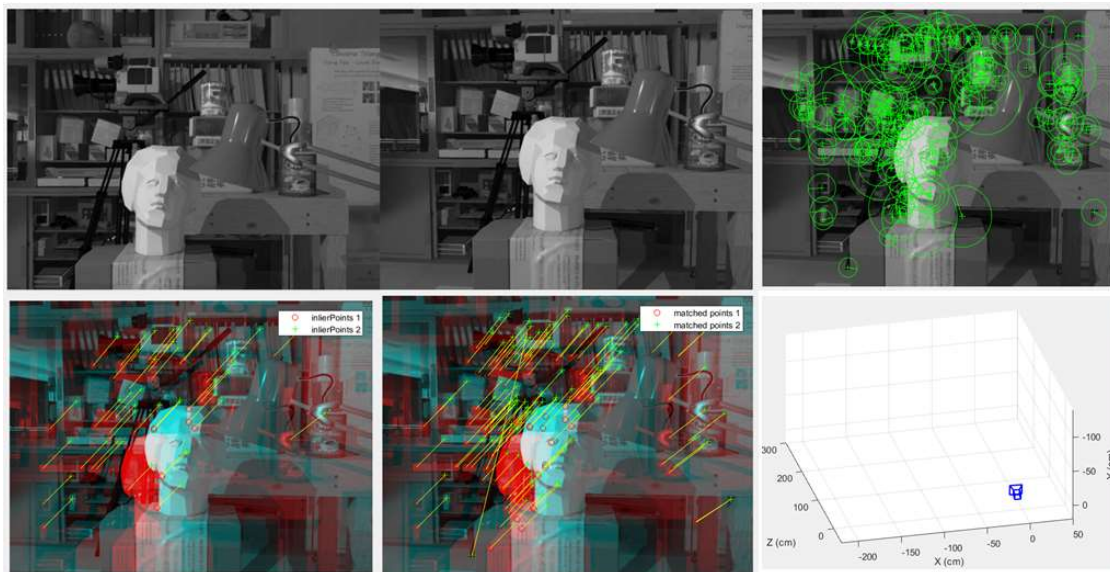


Abbildung 48: Quelle Tsukuba lab. Die Abbildung zeigt die Ergebnisse nach der outlier Detection durch RANSAC

- Überprüfen Sie, ob Sie die *outlier* richtig detektiert haben, mit Hilfe des *epipolar constraint* (vgl. Aufgabe 7.1.7.2)
- (Verwenden Sie die Methode *intrinsicMatrix* der Klasse *cameraParameters*, um aus der *F* Matrix die *E* Matrix zu rekonstruieren → für den workflow nicht notwendig, aber für das Verständnis sinnvoll)
- Überprüfen Sie Ihr Ergebnis, ob die berechnete Matrix die Eigenschaften einer E-Matrix hat (*rank* und *eig*, vgl. Skript)
- Berechnen Sie jetzt die E-Matrix mit Hilfe der Methode *estimateEssentialMatrix*
- Überprüfen Sie die Eigenschaften der berechneten E-Matrix mit *rank* und *eig*
- Aktualisieren Sie Ihren viewSet *vSet_est* („estimated Trejektorie“) mit berechneten *inliern*

7.2.4.2 DOKUMENTATION

Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode **und so, dass die Ergebnisbilder eingebunden werden**. Laden Sie dieses auf Moodle

7.2.5 TEIL 4 / 6 (POSE AUS E MATRIX BERECHNEN)

Ziel der Aufgabe ist es, aus der in Teil 3 berechneten E Matrix die pose zu rekonstruieren¹⁶.

Ergänzen Sie Ihren Code aus Aufgabe 7.2.4, indem Sie den workflow um die folgenden Schritte erweitern:

```
( ... )
% Schritt 1: Berechnung der Trajektorie → pairwise
1. Über alle Bilder (left ); immer zum ersten Bild (= Ref. Kos)

    (estimate Traj)
    • ( ... )
    • calc relative pose (Extraktion externe Kameraparameter) → Wo 10
    • viewset, set connections → Wo 10

    (visualize Traj est and real)
    • Visualize Traj (estimated) → Wo 10
    • Visualize Traj (real) → Wo 10
( ... )
```

Bearbeiten Sie die folgenden Teilaufgaben:

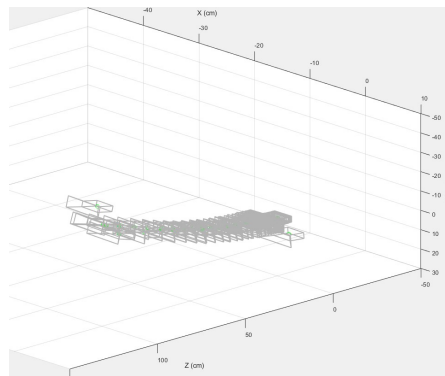


Abbildung 49: Die Abbildung zeigt die berechnete Trajektorie für 30 Standpunkte vor der Nachiteration durch das Bundle Adjustment

7.2.5.1 IN SCHRITT 1 (TRAJEKTORIE BERECHNEN)

- Rekonstruieren Sie aus der Essentiellen Matrix E die Rotation und Translation (Methode *relativeCameraPose*). Wofür werden die Übergabeparameter *matchedPoints* benötigt?

¹⁶ **Materialien** stehen wie folgt zur Verfügung: **(a.)** *Bildseq_Tsukuba_lab_10_images.zip*.

(b.) Zum Vergleich: *vset_est.mat*

Hinweis: Die Translation kann nur bis auf einen skalaren Faktor berechnet werden (vgl. Vorlesung). Zur Berechnung der Skalierung kann üblicherweise beispw. die Odometrie oder GPS Koordinaten verwendet werden. Berechnen Sie sich in diesem Projekt diesen skalaren Faktor aus der Groundtruth (aus der Differenz der Translation).

- Aktualisieren Sie Ihren viewSet *vSet_est* („estimated Trejektorie“) mit den berechneten Transformationen
- Stellen Sie auch die berechnete Trajektorie („estimated“) mit *plotCamera* bzw. *scatter3* graphisch dar
- Stellen Sie auch die auf Moodle bereitgestellte „Groundtruth“-Trajektorie *vSet_real* mit *plotCamera* bzw. *scatter3* graphisch dar. **Hinweis:** Die für die Orientierungen R_i gegebenen 3×3 Matrizen werden i.A. nicht den Eigenschaften einer Rotation entsprechen (warum?). Erzwingen Sie diese Eigenschaften durch Anwendung der Singulärwertzerlegung mit folgende Transformation (was bewirkt diese Transformation?)

$$\tilde{R}_i := U \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(UV') \end{pmatrix} V' \quad \text{wobei } [U, D, V] = \text{svd}(R_i) \quad (146)$$

7.2.5.2 DOKUMENTATION

Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode und so, dass die Ergebnisbilder eingebunden werden. Laden Sie dieses auf Moodle

7.2.6 TEIL 5/ 6 (BUNDLE ADJUSTMENT)

Ziel der Aufgabe ist es, die in Teil 4 berechneten *posen* durch Anwendung von bundle adjustment zu verbessern¹⁷.

Ergänzen Sie Ihren Code aus Aufgabe 7.2.5, indem Sie den workflow aus Aufgabe 7.2.1 um die folgenden Schritte erweitern:

```
( .. )

% Schritt 2: refinement → all images
4. Bundle Adjustment (nur estimated traj.) → Wo 11

( .. )
```

¹⁷ **Materialien** zum download: *Bildseq_Tsukuba_lab_10_images.zip* ; **(b.)** *vset_est.mat*

7.2.6.1 IN SCHRITT 0 („BUNDLE ADJUSTMENT“)

Verwenden Sie jetzt die berechneten initialen Posen als input Parameter für ein *bundleAdjustment* und gehen Sie hierzu wie folgt vor:

- Verwenden Sie die *findTracks* Methode Ihrer Klasse *viewSet*, um aus den bereits gefundenen korrespondierenden Punktepaares zum ersten View jetzt für die übrigen Bilder zueinander korrespondierende Punkte zu extrahieren (Methodik ähnlich zu Template Matching Kap. 4.4.4.3).
- Verwenden Sie die *triangulateMultiview* Methode, um aus diesen Tracks 3D Punkte *xyzPoints* und den reprojection error *reprojErr* zu berechnen (diese Methode entspricht der in der Vorlesung in Kap. 4.7.2 besprochenen *StereoTriangulation*, jedoch zur Steigerung der Genauigkeit adaptiert auf eine Folge von korrespondierenden Punkte)
- Verwenden Sie *pcshow*, um die Punkte aus *xyzPoints* darzustellen
- Rufen Sie das *BundleAdjustment* mit der Methode *bundleAdjustment* auf. Verwenden Sie hier geeignete Optionen
- Verwenden Sie *updateView*, um Ihr *viewSet* mit den Ergebnissen des BundleAdjustment zu aktualisieren
- Visualisieren Sie Ihre verfeinerte Trajektorie und Ihre korrespondierenden Punkte mit *plotCamera*, *scatter3* und *pcshow*

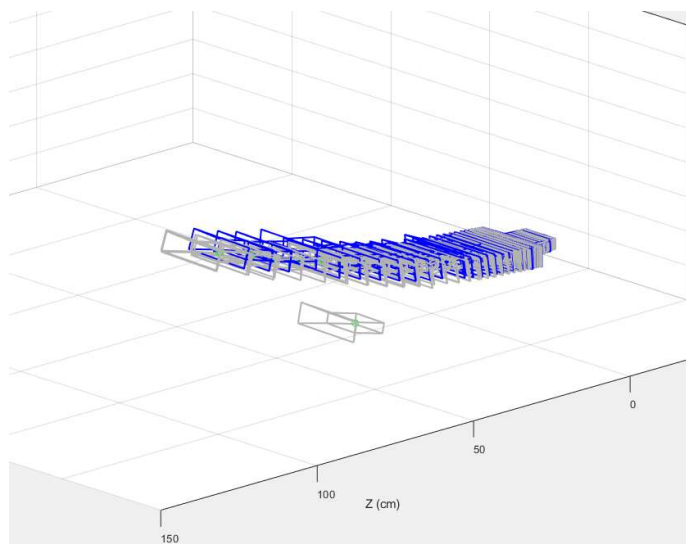


Abbildung 50: Die Abbildung zeigt die aus der Bildsequenz geschätzte Kameratrajektorie. (a.) vor bundle Adjustment (grau) (b.) nach Bundle Adjustment (blau)

7.2.6.2 DOKUMENTATION

Dokumentieren Sie Ihre Ergebnisse in einem .pdf unter Verwendung der Matlab-publish Methode **und so, dass die Ergebnisbilder eingebunden werden**. Laden Sie dieses auf Moodle

7.2.7 TEIL 6/6 (GEGEBENE GROUNDTRUTH)

Zwischen geschätzter Trajektorie („estimated“) und dem vom cvlab der Tsukuba University bereitgestellten Trajektorie *GroundTruth_ii* gibt es einen Versatz. Dieser kommt daher, dass die *Groundtruth_ii* relativ zum Mittelpunkt des ersten Frames des Stereokamera Systems angegeben wird.

Definieren Sie sich deshalb eine Transformation, um die *GroundTruth_ii* Daten ebenfalls zum ersten Frame der linken Kamera berechnen und darstellen zu können und visualisieren auch diese Trajektorie.

```
( ... )

(transform Traj real)
12.Transf. GroundTruth ins Ref.-Kos (= 1. Frame linke Kam.) → Teil 6 (Wo 12)
13.add / update viewset → Teil 6 (Wo 12)

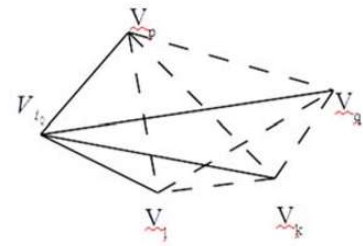
( ... )
```

7.2.8 MODIFIKATIONEN

7.2.8.1 SLIDING WINDOW

Verbessern Sie jetzt Ihr Ergebnis durch Modifikation des workflows (siehe *listing*):

(*Sliding Window*): In diesem Schritt wird immer nur ein Verband aus N Bildern mittels BundleAdjustment lokal zueinander optimiert und anschließend die berechneten Rotationen und Translationen in das Referenz-KOS $img_{N/2}$ transformiert.



```
% Schritt 0: init
( ... )

% Schritt 4: SlidingWindow
Über alle Bilder k ; Start = N/2 → Teil 7 (Wo P12)

    % Schritt 1: Berechnung der Trajektorie nur über N Bilder im sliding window
    ( ... )

    % Schritt 2: refinement → all images
    ( ... )

end slidingWindow

% Schritt 3: Dichtes 3D Modell → pairwise
( ... )
```

7.2.8.2 STABILISIEREN DURCH STÜTZSTELLEN

(*Stabilisierung durch Stützstellen*): Verwenden Sie eine/mehrere Stützstellen (bei uns Koordinaten aus der Ground Truth) und verbessern Sie ihr Ergebnis mit Levenberg-Marquardt durch Lösung eines nichtlinearen Optimierungsmodells.

```
% Schritt 0: init
( .. )

% Schritt 4: SlidingWindow

    % Schritt 1: Berechnung der Trajektorie nur über N Bilder im sliding window
    ( .. )

    % Schritt 2: refinement → all images
    ( .. )

% Schritt 5: Stabilisierung durch Stützstellen
Optimiere nichtlineares Fehlermodell mit Levenberg-Marquardt Teil 7

% Schritt 3: Dichtes 3D Modell → pairwise
( .. )
```

7.3 MOBILE MAPPING (PYTHON/OPENCV)

Im Folgenden wird gezeigt, wie die Computer Vision Bibliothek OpenCV unter Python verwendet werden kann¹⁸.

7.3.1 EINFÜHRUNG IN PYTHON (WOCHE 1)

7.3.1.1 GETTING STARTED

Vorbereitung:

Installieren von **Python** (Empfohlene Python Versionen 3.4 bis 3.8 zur Nutzung der Packages). Für die Erstellung dieses Tutorials wurde 3.8 verwendet. Mit der neuesten Version kann es zu Problemen mit den notwendigen Packages kommen. Siehe

- www.python.org

Für Windows 64bit kann beispielsweise

- <https://www.python.org/ftp/python/3.8.10/python-3.8.10-amd64.exe>

verwendet werden. In der Installer-Version ist die *IDLE* und *pip* enthalten. Außerdem sollte Python zum „path“ hinzugefügt werden.

¹⁸ Die Entwicklung des Praktikums unter Python/OpenCV entstand im Rahmen des HM Förderprojektes „Innovative Lehre“ und unter maßgeblich Beteiligung von an der B. Eng. Sven Ruckriegel.